

Computer organization & Architecture

→ prerequisite
 ↳ How a Computer works internally → Digital logic
 Circuits & Design
 ↳ CPU, RAM, I/O → Key board, Mouse, disk
 Intel i3, i5, ...

4GB, 8GB, ...

IOT → Micro-Controllers

Smart phone

Laptop

Smart watch

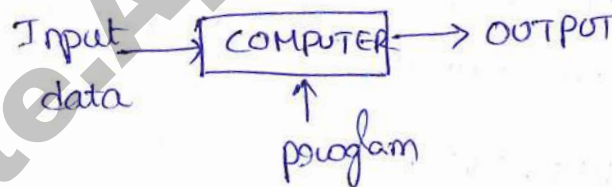
COA → Hardware

OS → Software

CD, DB

Micro-processors

What's a Computer: programmable Computer
 CPU



Neumann-Architecture

stored program Computer



program: Sequence of Instructions

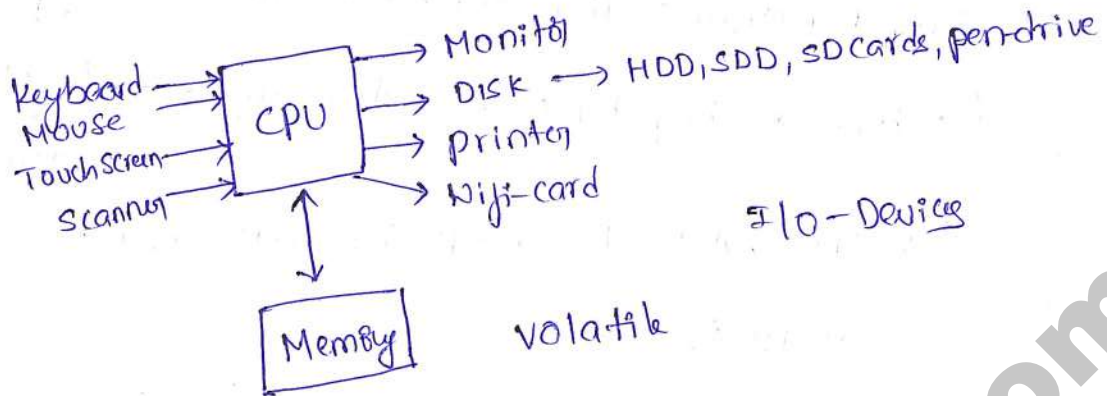
Compiler
 C → MLL

Instruction: Binary Sequence / String

data : Binary - seq / String

Block Diagram:

Ph: 844-844-0102



When the power is switched off all the data in main memory is removed.

RAM - Random-Access Memory

ROM - Read only Memory

Mother-board:

ROM

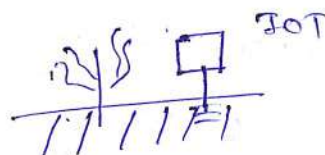
- CPU slot (Micro processor)
- PCI slot (can connect sound cards etc)
- Slots for HDD, DVD-ROMS
- RAM (Memory) slots
- Graphics card slot
- Back panel with connections for peripherals and USB Memory.

Micro Controllers

ESP-32 chip

ARDUINO boards

Raspberry pi boards



Every thing inside a chip (All Hardware of Computer) is a Micro Controller chip.

Ph: 844-844-0102

Basics of Memory organization:

- All the programs are in Main Memory.
- CPU performs memory-requests

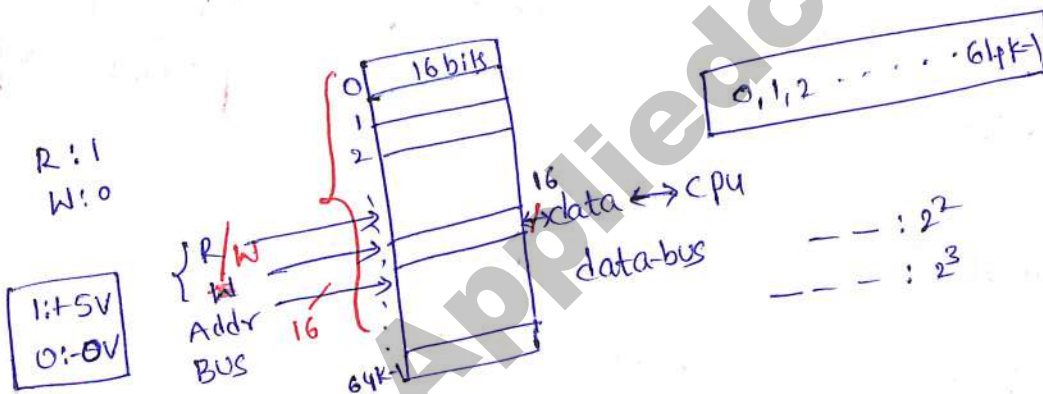
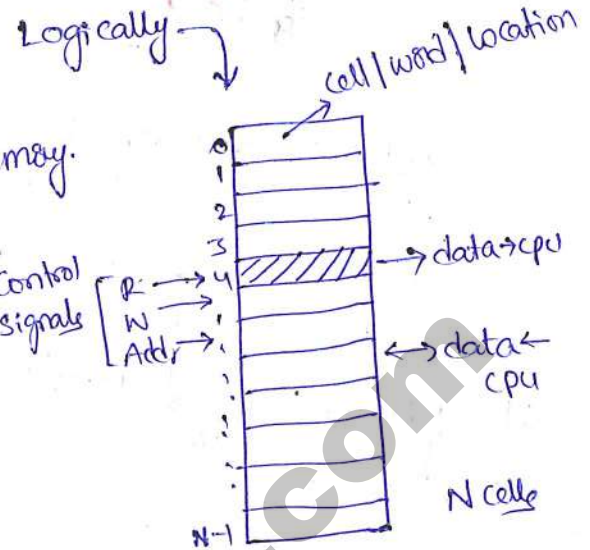
Eg: 6264 Memory chip

Datasheet 6264

64K X 16

N = # Cells

Size of each cell in bits



$$1K = 2^{10} = 1024 \approx 10^3$$

$$1M = 2^{20} \approx 10^6$$

$$1G = 2^{30} \approx 10^9$$

$$1T = 2^{40} \approx 10^{12}$$

$$64K \Rightarrow 2^6 \times 2^{10} = 16 \text{ bits}$$

$$2^{16} \text{ cells/locations}$$

$$K \text{ bits} \rightarrow 2^K$$

$$16 \text{ bits} \rightarrow 2^{16} \text{ unique addresses}$$

$$64K \times 32 \downarrow \downarrow \text{databus} = 32 \text{ bits}$$

$$\downarrow \text{Address bus}$$

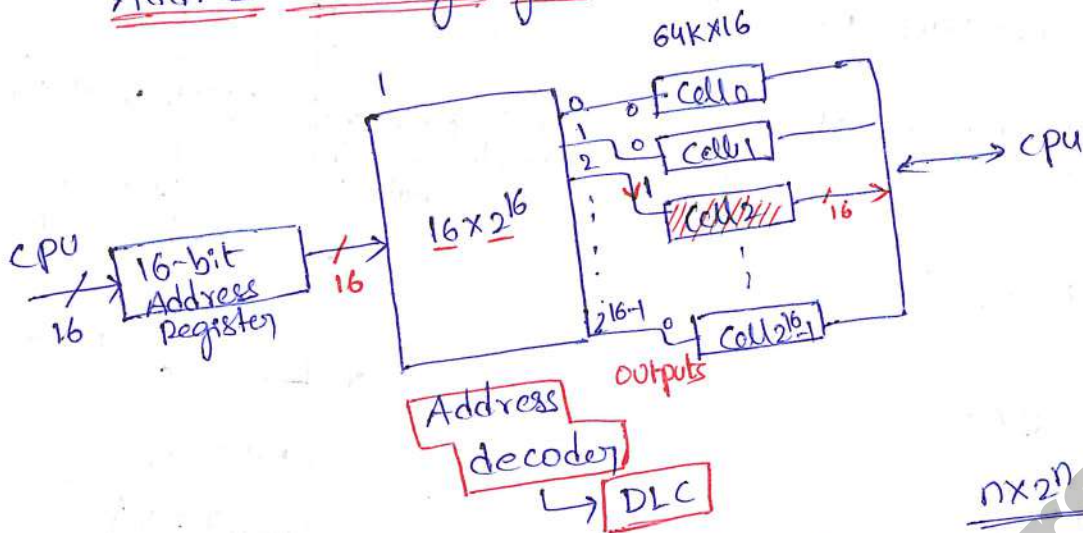
$$(1011\ 1100\ 0001\ 1100)_2$$

$$\downarrow$$

$$(B\ C\ 1\ C)_{16}$$

Bin & Hex
 $\downarrow$
 C, DLC

Address - decoding Logic:



$n \times 2^n$: Decoder

At any time only one output is active / high

$(0000\ 0000\ 0000\ 0010)_2$
 16 bits $\rightarrow$ 0, 1, 2, ..., $2^{16}-1$

high: 1.5V
 Low: 0.0V

Pin-diagram of 6264 :

VCC: Power

A₁₂ ... A₀: Address Bus

D₇ ... D₀: Data Bus

GND: Ground 0V

R/W: 1=R 0=W

NC: Not Connected

OE: Enable output

CE1: chipselect1

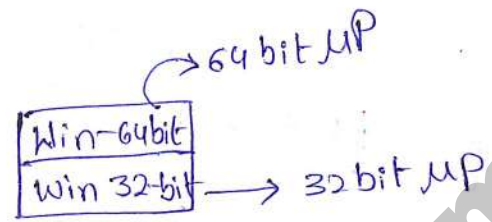
13-bit Address
 A₀ ... A₁₂ $\Rightarrow 2^{13}$
 $\Rightarrow 2^3 \times 2^{10}$
 $\Rightarrow 8K$

8Kx8

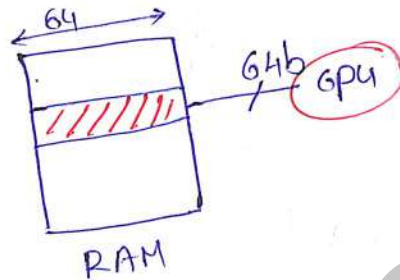
} Control pins

Note: No coherent address will be stored. They are corresponding to the output lines of address decoder.

HLW 8085, 8086
8-bit microprocessor
16-bit microprocessor
32-bit microprocessor
64-bit microprocessor
 Most modern processor



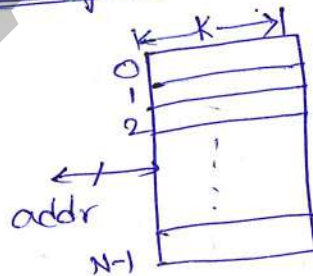
Word-size 64bits



Memory Addressability:

Byte and word - Addressability

Physical Address Space:



2^n cells / words

0 to $2^n - 1$

$$n = \log_2 N$$

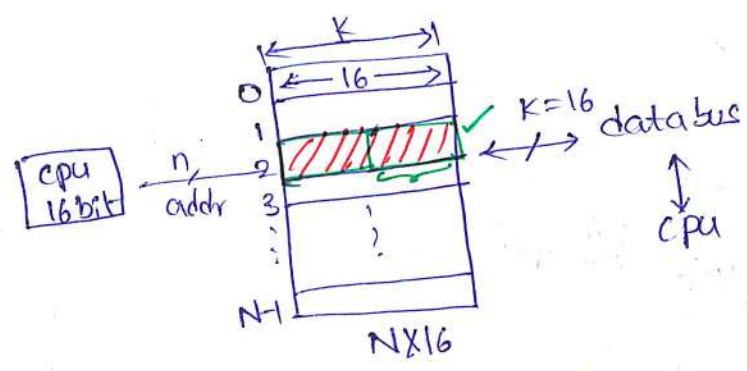
Byte Addressable: one byte using

$\rightarrow$ 8bits one Address if $N \times 8$

Nibble: 4bits

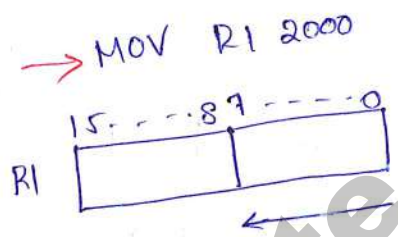
$1W = 1B \quad (N \times 8)$
 $1W = 2B \quad (N \times 16)$
 $1W = 3B \quad (N \times 24)$ $\rightarrow$ One Address corresponds to 2B.

Up: Byte Addr

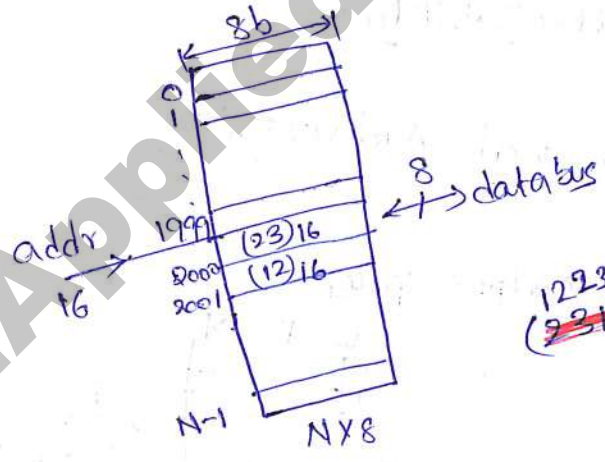


Ignore the first Byte

16-bit processor:

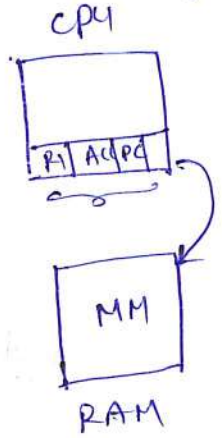
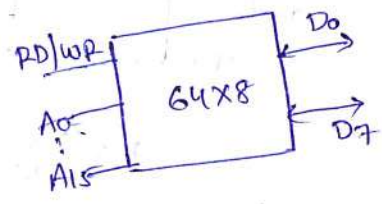


$x = x + 1;$
 $\downarrow$
2B [2000, 2001]

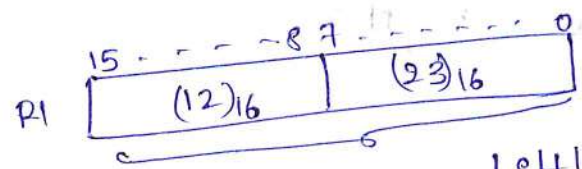


$(1223)_{16}$
 ~~$(2212)_{16}$~~

Register $\downarrow$ Memory inside CPU

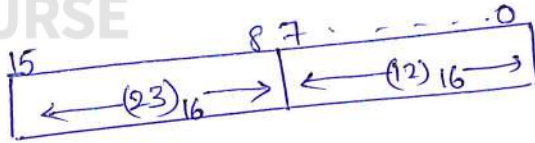


MOV R1 2000

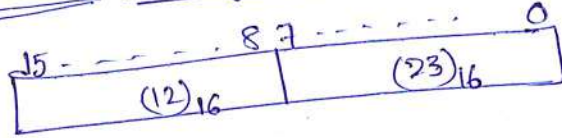


Little Endian

Big-endian System:



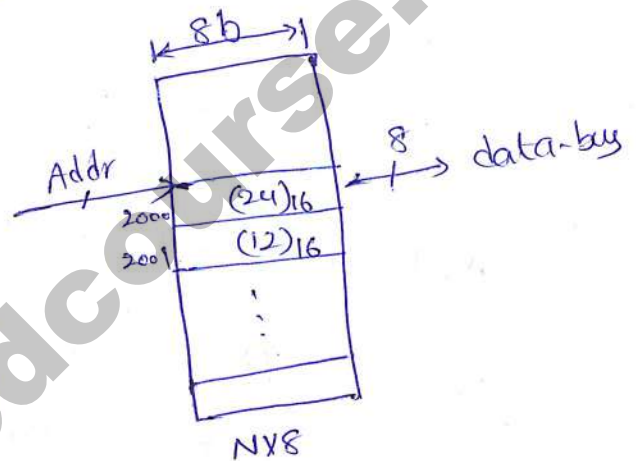
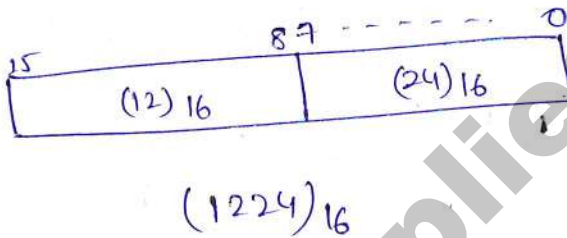
Little-Endian System:



MOV R1 2000 $\xrightarrow{\text{ALL}} \underline{\text{MLL}}$

{ 10 11 010  ... }

MOV R1 2000
INCR R1
MOV 2000 R1



CPU-Memory Interfacing:

Pin-Structure:

① Active-low $\rightarrow 0$

② Active-High $\rightarrow 5V$
 $\downarrow$
1

ALE

8085 A
8-bit

8086
16bit

$\left. \begin{array}{l} \overline{RD} \quad RD' \\ \overline{WR} \quad WR' \\ \overline{INTA} \end{array} \right\} \text{Active when they are low}$

③ Time-multiplexed - Pin:

8085A: AD0 - - - AD7
 A8 - - - A15

8 bit

1W=1B

16-addressing $\Rightarrow 2^{16}$ cells

$$= 2^{16} B$$

= 64KB size of MM

A0 - - - A7 A8 - - - A15 $\leftarrow$ addr
D0 - - - D7 $\leftarrow$ data

ALE: Address Latch Enable

$\left\{ \begin{array}{l} \text{ALE} = 1 \text{ (A0 - - - A7)} \\ \text{ALE} = 0 \text{ (D0 - - - D7)} \end{array} \right.$
 Address Lines
 Data Lines

cpu $\leftrightarrow$ RAM

- ① Send the address
- ② R & W data

8086 CPU: 16 bit MP

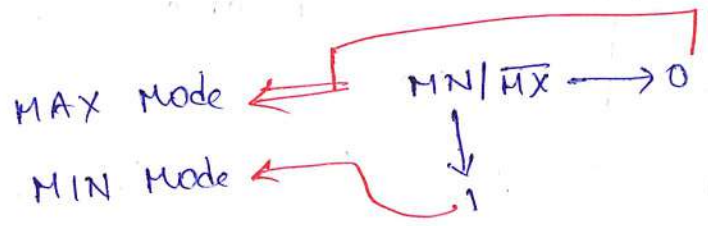
AD0 - - - AD15 A16 - - - A19

(0-19) 20 bit Address

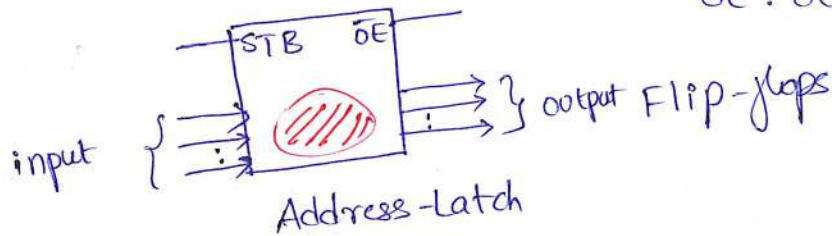
Data $\Rightarrow$ 16 bits

$$2^{20} \text{ cells} = 1 \text{ MW} \\ \downarrow \\ 2B$$

$\Rightarrow$ 2MB



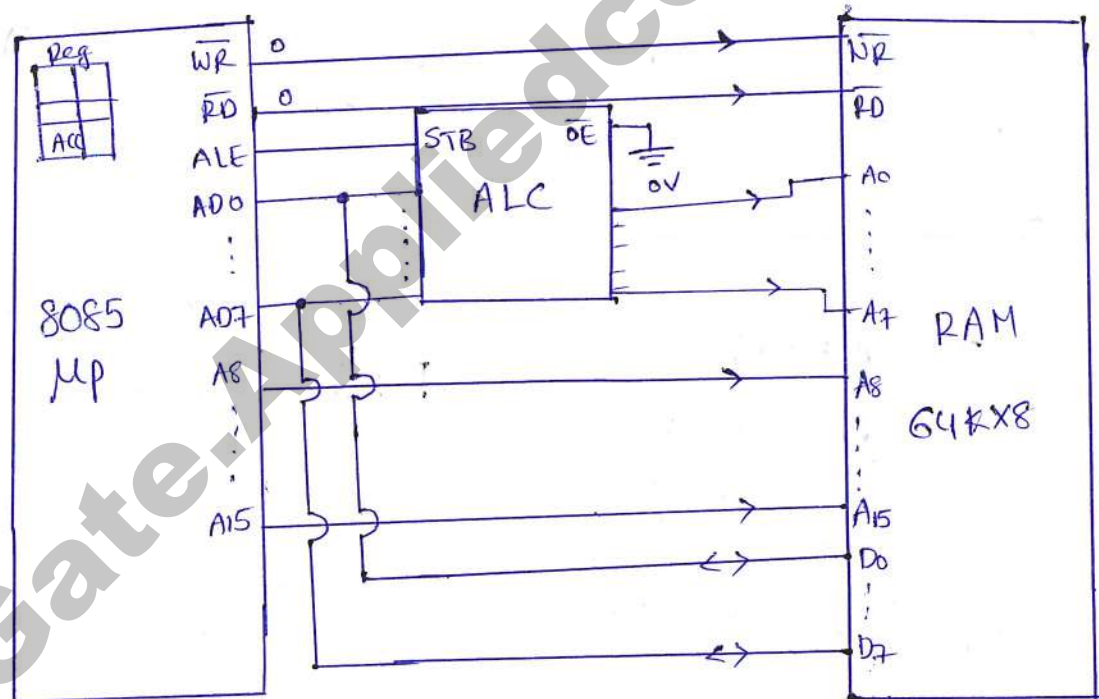
STB: Strobe
OE: output Enable



STB $\begin{cases} 0 \Rightarrow \text{Input is disabled.} \\ 1 \Rightarrow \text{Input is Enabled} \end{cases}$

OE $\begin{cases} 0 \Rightarrow \text{output is Enabled.} \\ 1 \Rightarrow \text{output is disabled.} \end{cases}$

Cpu-Memory Interfacing:



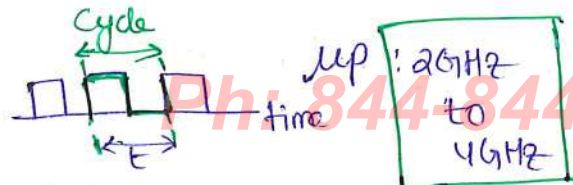
when ALE 0 Data pins are connected to RAM

More the pins, the complexity of the chip will increase.

① clock-cycle:

$$\text{cycle-time} = \frac{1}{f}$$

$$= \frac{1}{\text{freq of operation of } \mu p}$$



Speed of μp

$$2 \text{ GHz} = f$$

$$2 \times 10^9 \text{ Hz} = f$$

$$2 \times 10^9 = f$$

$$2 \times 10^9 \text{ Hz}$$

$$2 \times 10^9 \text{ cycles/sec}$$

$$t = \frac{1}{2 \times 10^9} \text{ sec}$$

$$= 0.5 \times 10^{-9}$$

$$= 0.5 \text{ ns}$$

$$\text{ms} = 10^{-3}$$

$$\mu\text{s} = 10^{-6}$$

$$\text{ns} = 10^{-9}$$

② Machine (M/C) cycle:

of Cycles required to complete an operation.

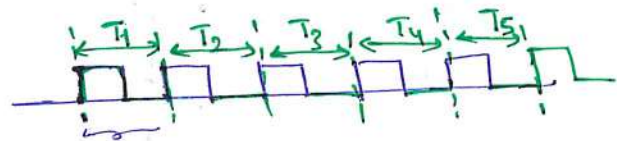
one instruction LDA (2080)₁₆

Load into accumulator Register

① Send the address

T₁: ALE to 1

$\left. \begin{array}{l} \text{AD}_{15} - \text{AD}_0 : (80)_{16} \\ \text{A}_{15} - \text{A}_8 : (20)_{16} \end{array} \right\}$



② Send Control Signals:

T₂: $\left\{ \begin{array}{l} \text{ALE} = 0 \\ \overline{\text{RD}} = 0 \\ \overline{\text{WR}} = 1 \end{array} \right.$

③ Read the data to ACC:

T₃:

T₄:

μp : 2GHz - 4GHz

RAM: 1444 MHz $\approx$ 1.4 GHz

MM

3GHz Very Exp

8GB
16GB
32GB

Slower than μp

$D_0 \dots D_7$

Ph: 844-844-0102 slower than up
RAM clock cycle relatively

$\begin{Bmatrix} T_3: \\ T_4: \end{Bmatrix} D_0 \dots D_7$

once the data is read from Memory to Data line, processor will store them inside Accumulator.

INCR ACC

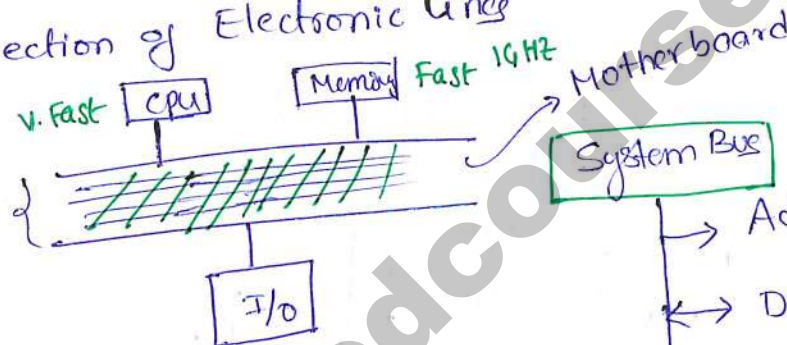


control unit
Arithmetic logic unit

System Bus Configuration:

BUS: Collection of Electronic Lines

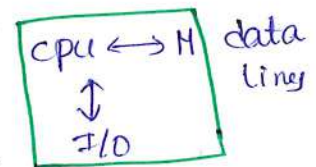
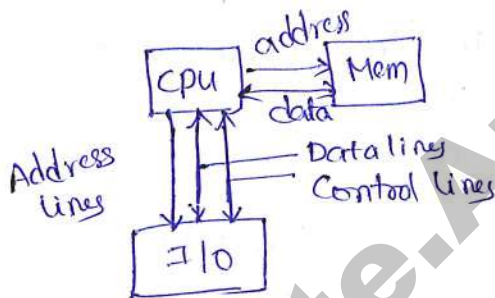
24Hz v. Fast 14Hz Fast



System Bus

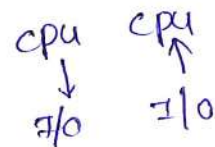
Address Lines
Data Lines
Control Lines

Monitor, Keyboard,
printer,
Mouse,
DISK



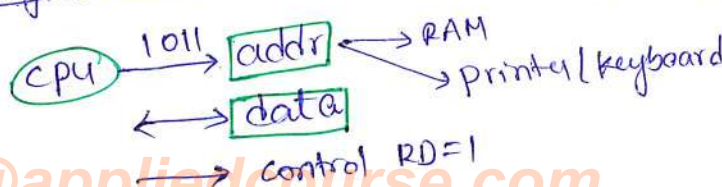
printer: page are over

need to send the interrupt signal to cpu
to display a message to the user



Ambiguity problem:

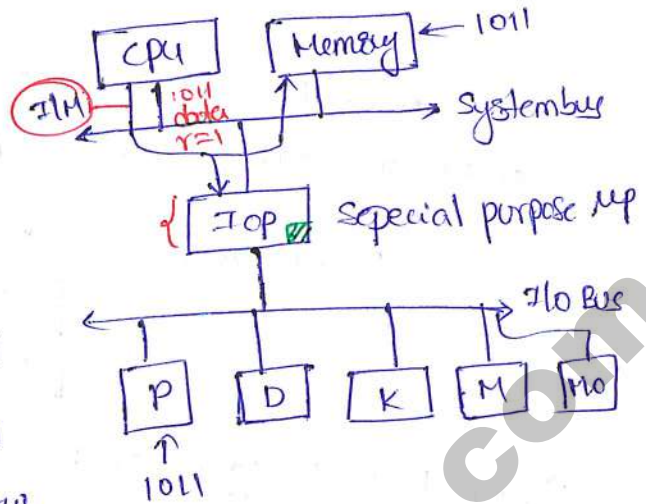
if keyboard address = 1011



Bus Configuration : I/O processor ①

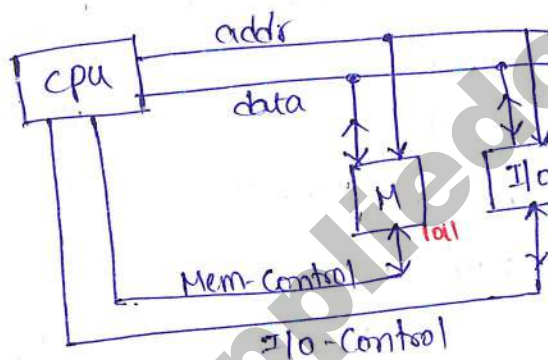
High performance Computing
NVIDIA IOP

- Common Control Signals
- Common Address Space
- Different Buses
- I/O - Memory direct access
- Expensive



when I/O is active, M is low
M is active, I/O is low

Bus Configuration : Isolated I/O ②



Eg: 8085A

Pin 34

I/O M

1 → I/O

0 → Mem

we can have multiple I/O devices

let addr = 1011, RD=1, I/O M Signal

Each I/O device have unique address

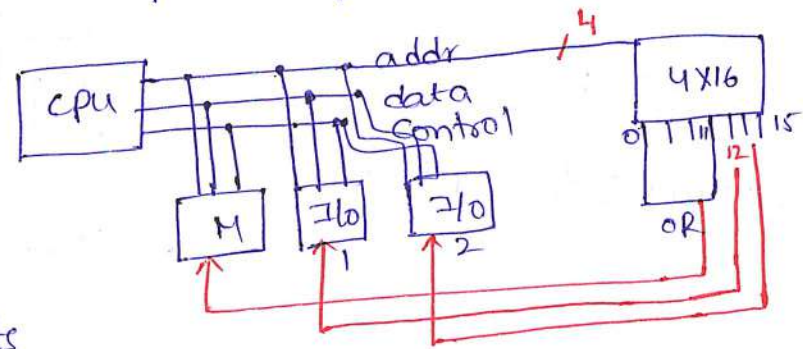
No requirement of I/O processor.

I/O M	RD	
1	0	I/O Read
0	0	Mem Read

Bus Configuration : Memory mapped I/O ③

Ph: 844-844-0102

→ shared address space using decoders

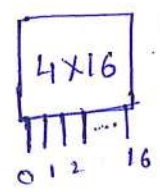
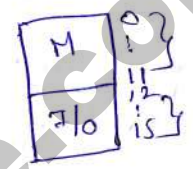


addr : 4 bits

1 1 - -

24 addr 00 - - }
01 - - } → Mem-addr
10 - - }
11 - - }

⇒ 11 - - } → I/O device addr



② μp that supports 128MW memory

Memory-mapped I/O

128MW
Connection point $2^7 \times 2^{20} W$

3HSB core ⇒ assign to I/O ports / devices ⇒ 2^{27}

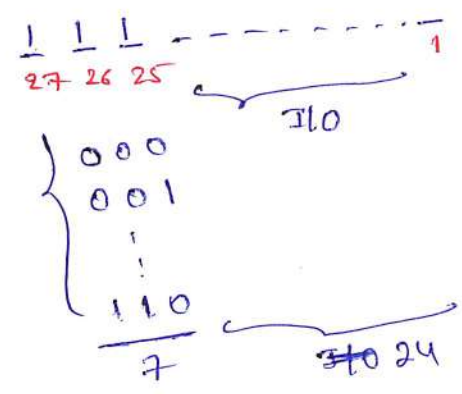
Address: 27 bits

I/O addresses: ?

Memory addresses: ?

I/O Addresses : 2^{24}

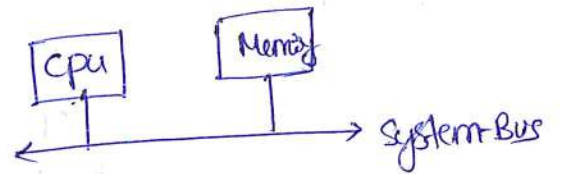
Memory Addresses
 7×2^{24}



Instruction cycle: Sequence of Execution Steps

① Fetch-Cycle:

② Execute-cycle: cpu-orig



T_1 : LDA 2080 8085-Syntax

Fetch-Cycle:

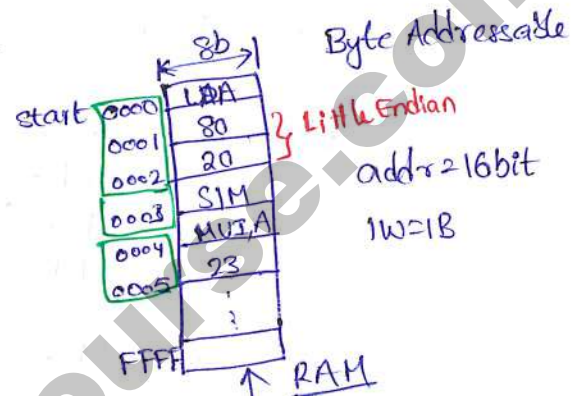
Load Accumulator

$(23)_H$ 0x23

Program:

T_1 : LDA (2080)₁₆ $\rightarrow 3B$
 T_2 : SIM $\rightarrow 1B$
 T_3 : MVI A, 23H $\rightarrow 2B$
 Move Immediate

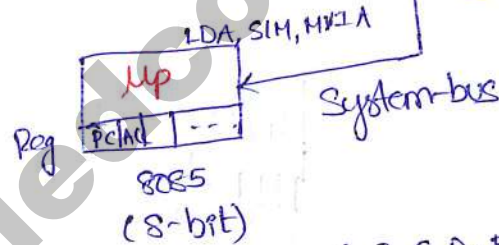
SIM
Set Interrupt
Mask



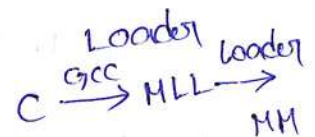
Program Counter: 0000

Step-Size: 3B

Stores the address of the next instruction to be executed.



A, B, C, D, E, H, L



μP Fetches LDA 2080
goes to Execute cycle

Program counter: 0003
Step-size: 1B

μP Fetches SIM
Next goes to Execute cycle

Program counter: 0004

Step-size: 2B

If Instruction is Executed

Program counter: 0006

Fixed-length Instruction CPUs:

↓
Step size is fixed

All instructions are fixed Length
Ph: 844-844-0102

Variable-length instruction CPU: ↓

(8085, 8086 - - -)

I₇ I₉

Stepsize = f (opCode)

Q1 32-bit $\mu p \Rightarrow 1W = 4B$

$I_1: 2W \quad 234 \text{ to } 23B$ starting memory addr = 0×234
 $I_2: 1W \quad 23C \Rightarrow$ value of PC when I_6 is being executed? $(258)_{16}$
 $I_3: 1W \quad 240$ Memory is Byte addressable.
 $I_4: 2W \quad 244$
 $I_5: 1W \quad 24C$
 $I_6: 2W \quad 250$
 $I_7: 1W \quad 258$
 $\quad \quad \quad 25C$

$$\begin{array}{r} 234 \\ 8 \\ \hline 23C \end{array}$$

$$\Rightarrow \begin{array}{r} 23C \\ 14 \\ \hline 240 \end{array}$$

$$\begin{array}{r} 240 \\ 4 \\ \hline 244 \\ 8 \\ \hline 24C \\ 8 \\ \hline 250 \\ 8 \\ \hline 258 \end{array}$$

0, 1, 2 - - 9, A, B, C, D, E, F
Hexadecimal

Q2 Memory is word addressable (Same as above)

$1W = 32b = 4B$

program-starting-addr = $(2340)_{10}$

PC-value when I_6 is being executed.

(16)

 $(2346)_{10}$

Start

Ph: 844-844-0102

 $I_1: 2346$ $I_2: 2348$ $I_3: 2349$ $I_4: 2350$ $I_5: 2352$ $I_6: 2353$

PC $\rightarrow$ $I_7: 2355$ $(2355)_{10}$
 $I_8: 2356$

Q3

cpu with 24-bit instructions program:

Fixed-len-instruction

-cpu

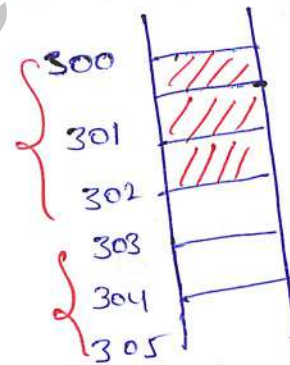
program-load-address = $(300)_{10}$

which of these is a valid value of pc.

24bit = 3B

- ☒ 400 ☒ 500 ☒ 600 ☒ 700
☒ 400 ☒ 500 ☒ 600 ☒ 700

By default Memory is
Byte addressable



PC: 300

PC: 303

PC: 306

: 309

!

 $300 + i \times 3$ $300 + 100 \times 3$ Execution-Cycle:

LDA 2080
 opcode operand

Instruction format $\rightarrow$ 1B
 $\rightarrow$ 2B
 $\rightarrow$ 3B...

$\rightarrow$ cpu-organization

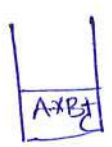
cpu organization: Stack cpu

Ph: 844-844-0102

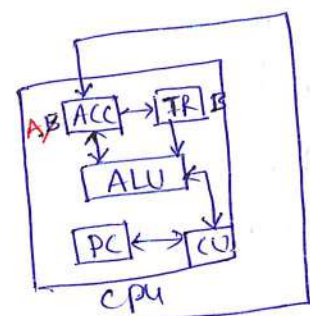
Stack-cpus: (Zero address Instructions)

- I₁ : MOV A, 2080
- I₂ : MOV B, 2090
- I₃ : MOV C, 2096

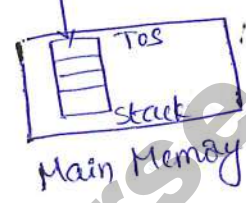
- I₄ : PUSH A
- I₅ : PUSH B
- I₆ : MUL opcode, No addr
- I₇ : PUSH C
- I₈ : ADD
- I₉ : POP D



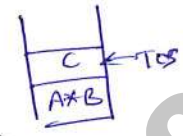
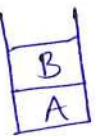
D = A*B + C



TR - Temporary Register



TOS
Top of the Stack

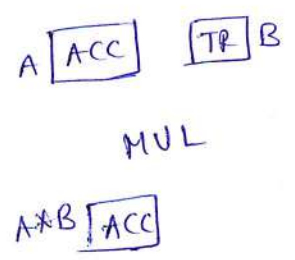


ALU: Arithmetic and Logic unit

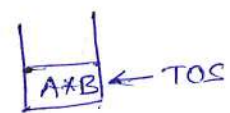
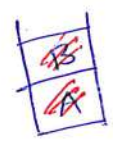
- Zero-address Instruction Format ← ALU instr
- Non-All instructions need not be 0-address-instr
- Instruction-cycle: MUL (I₆)

Fetch: Read MUL from memory to cpu based on PC by the Control unit.

Execute: Interpret the opcode

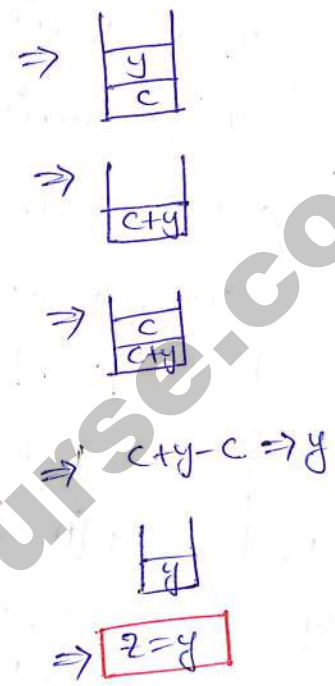
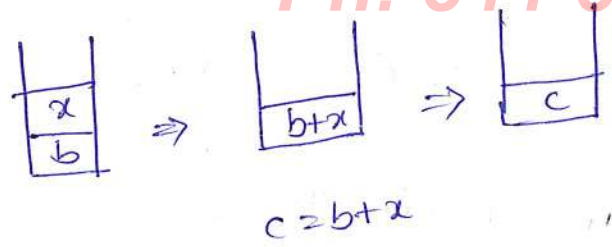


- pop ← 2nd op
- pop ← 1st op
- MUL ←
- PUSH →



Q Stack-cpu:

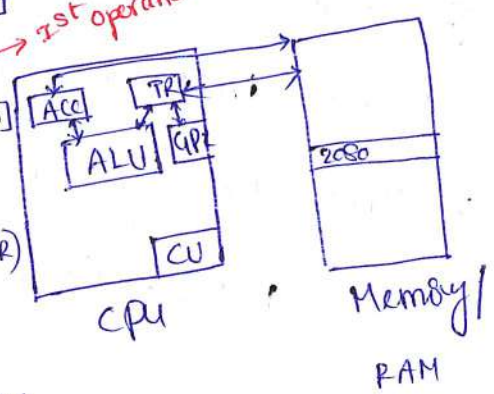
- I₁: Push b
- I₂: push x
- 3: Add
- 4: pop c
- 5: push c
- 6: push y
- 7: ADD
- 8: push c
- 9: Sub
- 10: pop z



CPU organization: Accumulator CPU

1- address Instruction

- I₁: Load 2080; $ACC \leftarrow M[2080]$
- I₂: MUL 2090; $ACC \leftarrow ACC \times M[2090]$
- I₃: ADD 2096; $ACC \leftarrow ACC + c$



Instruction Cycle: I₂

- (a) Fetch: MUL 2090 read from Memory.
- (b) Execute: Interpret

$TR \leftarrow M[2090]$

Multiply

$ACC \leftarrow m[2080] + m[2090]$

$(a * b) + c$

Examples: ENIAC (1945) had 20 ACC

Ph: 844-844-0102

{ IBM 701 (1952)
IBM 650
PDP 8

General register cpu

one-addr - Instructions : opcode Addr memory/register

$X \leftarrow (A+B) / (C+D)$

(P1)

- 1: load A
- 2: Add B
- 3: STORE T
- 4: load C
- 5: Add D
- 6: Store Y
- 7: load T
- 8: DIV Y
- 9: Store X

$ACC \leftarrow A+B$

$T \leftarrow A+B$

$ACC \leftarrow C+D$

$Y \leftarrow C+D$

$ACC \leftarrow (A+B)$

$ACC \leftarrow (A+B) / (C+D)$

X

(P2)

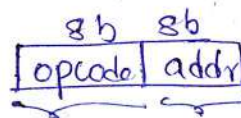
- 1: load C
- 2: ADD D $ACC \leftarrow C+D$
- 3: ~~load~~ STORE T $T \leftarrow C+D$
- 4: load A
- 5: ADD B $ACC \leftarrow A+B$
- 6: DIV T $ACC \leftarrow (A+B) / (C+D)$
- 7: STORE X

Expand opcode - technique:

up with fixed-length instructions (16b)

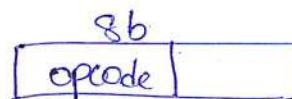
①

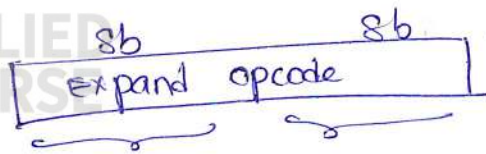
1-addr instruction:



②

0-addr instruction:





Ph: 844-844-0102

0000 0000
1111 1111

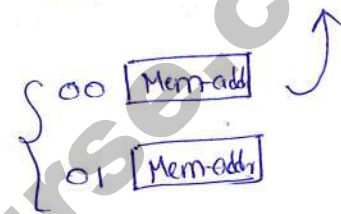
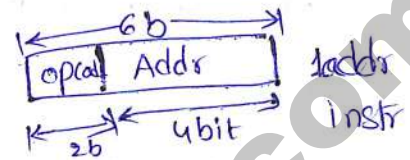
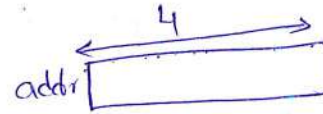
Q up that supports 0-addr-instr & 1-addr-instr

Memory size: $16W = 24W$

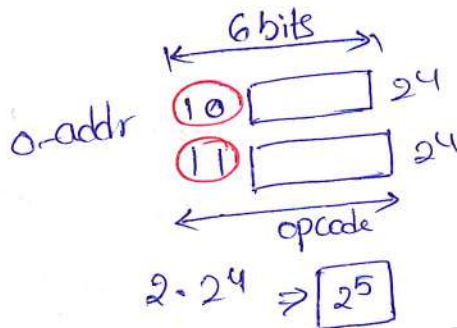
fixed-sized instr: 6 bits = $1W$

Two 1-addr instructions

possible zero-addr instructions = ?



0-addr



Q

32-bit up:

256 MW memory (max-supported)

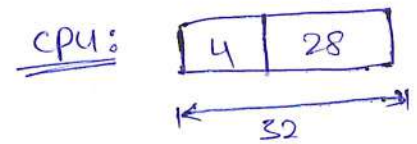
fixed-size addr-instr: $1W$

Eight 1-addr instr

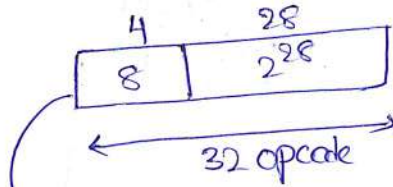
possible 0-address instr = ?

$$256 \text{ KW} \Rightarrow 2^8 \times 2^{20} \text{ W}$$

$$\Rightarrow 2^{28} \text{ W}$$

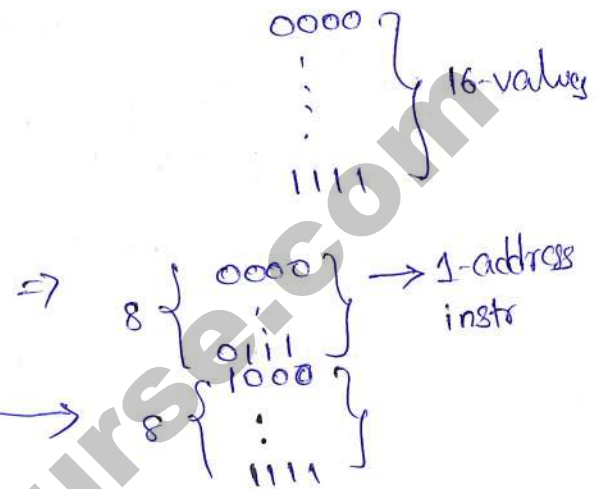


0-address instruction:



$$\Rightarrow 2^3 \times 2^{28}$$

$\Rightarrow 2^{31}$ possible 0-address instructions.

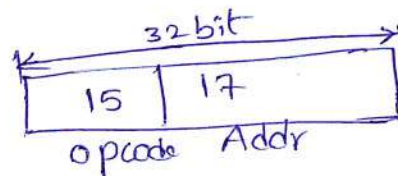


① up with 0-addr & 1-addr instruction

fixed-sized-instr: 32b

$$\text{Memory: } 128 \text{ KW} = 2^{17} \text{ W}$$

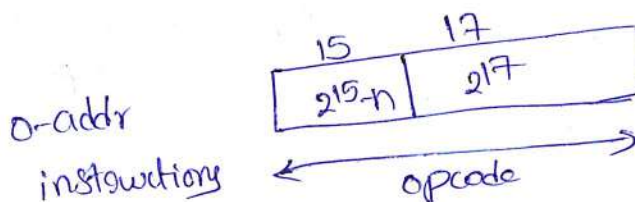
If n 1-addr instruction exist, # 0-address-instr = ?



$$2^{15}$$

$$\Rightarrow 2^{15}$$

n 1-addr instructions



cpu organization: General Register cpu

Ph: 844-844-0102

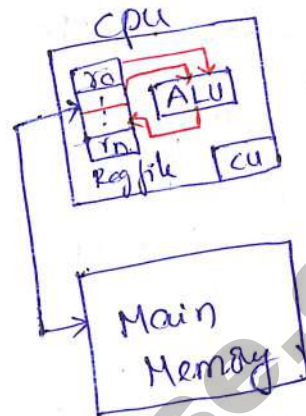
- Reg - Mem Reference architecture
- Reg - Reg Reference Architecture.

Reg-Reg Reference cpu:

I_1 : opcode dest op1 op2
 $\text{ADD } r_0, r_1, r_2 \quad r_0 \leftarrow r_1 + r_2$

- 3- Address - instruction

- load & store: $\text{Reg} \leftrightarrow \text{Mem}$



Eg: $X = (A+B) * (C+D)$

load r_0, A (2-addr-instruction)

load r_1, B

add r_0, r_0, r_1

load r_1, C

load r_2, D

add r_1, r_1, r_2 (3-address-instruction) $r_1 \leftarrow C+D$

MUL r_0, r_0, r_1

store X, r_0

r_0 ☐

r_1 ☐

$r_0 \leftarrow A+B$

$r_1 \leftarrow C+D$
 $r_0 \leftarrow (A+B) * (C+D)$

4- Address instructions:

I_1 : opcode dest-addr src1 src2
 op1 op2

addr-next-instr
 $\text{addr}(I_{20})$

I_2

$\vdots$

I_{20}

→ Not used widely

PC →

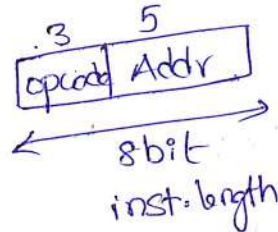
Address of the next instruction.

Ph: 844-844-0102

Instruction - Execution Sequence:

Accumulator - CPU:

↓
1-Address instruction

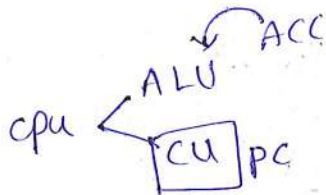


Fetch:

① PC → CPU generates mem-request

Memory

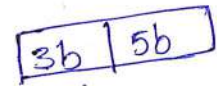
↓
Instruction Register



Design-time binding

② Instruction decode

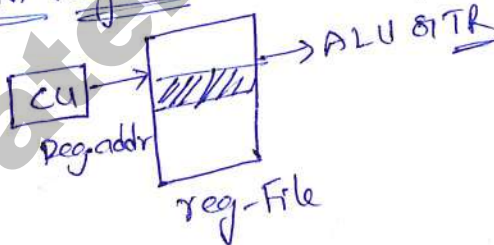
000 - Add
001 - MUL
010 - Sub
011 - LOAD
⋮



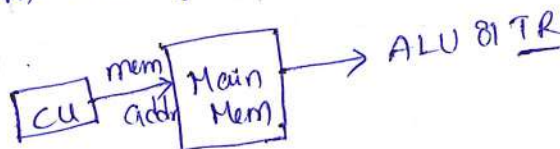
↓
cu Psw
Flag Reg

③ operand Fetch:

① From Register

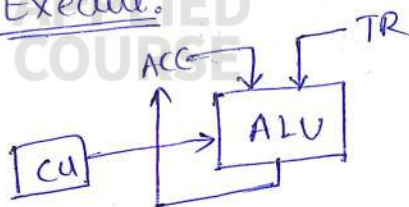


② From Memory



program status word: will store all miscellaneous information about the currently executed instruction.

④ Execute:



Ph: 844-844-0102

Reg-Mem Reference Architectures

MUL $r_0, 2080$
 ↑ ↘
 operand1 Memory Location
 operand2

in Reg-Reg Reference

MUL r_0, r_1
 ↓ ↓
 Reg Reg

→ $T_1: \text{ADD } r_0, 2080$ (with a red checkmark)
 (with a red arrow pointing to r_0 and a red arrow pointing to 2080)

→ 2-addr-format

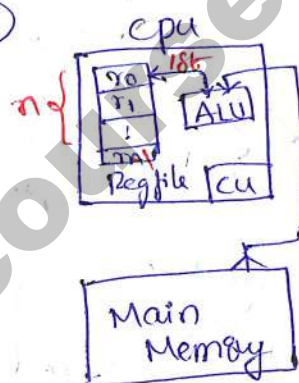
→ Fetch

→ Execute: Interpret

Fetch $m[2080]$

ADD $r_0 + m[2080]$

$r_0 \leftarrow r_0 + m[2080]$



Reg-File
 ↓
 collection
 of
 Registers

$$X = (A+B) * (C+D)$$

1: MOV r_0, A

2: Add r_0, B

3: MOV r_1, C

4: Add r_1, D

5: MUL r_0, r_1

6: MOV X, r_0

$r_0 \leftarrow A$

$r_0 \leftarrow A+B$

$r_1 \leftarrow C$

$r_1 \leftarrow C+D$

$r_0 \leftarrow r_0 * r_1$

$\left. \begin{array}{l} \text{MOV } T, r_1 \\ \text{MUL } r_0, T \end{array} \right\}$

① μp That supports 2-addr, 1-addr & 0-addr instructions.

Ph: 844-844-0102

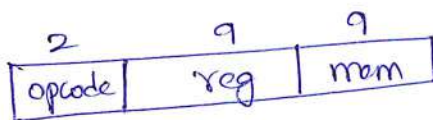
20-bit instructions

512W memory

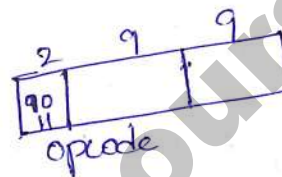
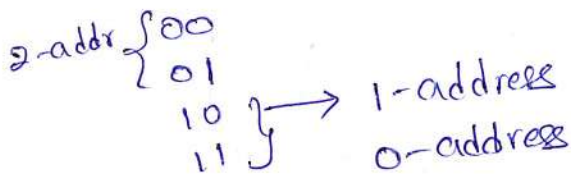
Two 2-addr-instr

150 1-addr-instr

possible 0-addr-instructions = ?



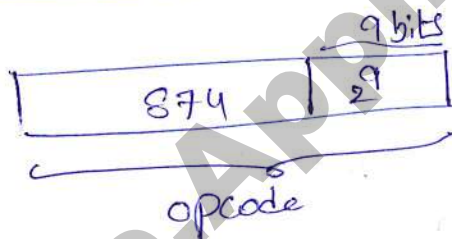
Memory
512W



$$2 \times 2^{10} \Rightarrow 2^{10} \Rightarrow 1024$$

- given 150-1addr instructions

zero-address Instructions:



$\Rightarrow 874 \times 2^9$ zero-address instructions possible.

②

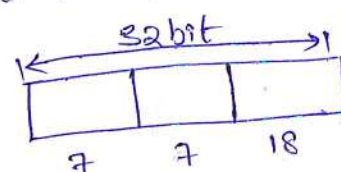
instruction-format $\langle \text{opcode}, \text{reg-addr}, \text{mem-addr} \rangle$

registers = 128 $\Rightarrow 2^7$

256 MW Memory

32-bit instructions

$$256 \Rightarrow 2^8 \text{ MW} \Rightarrow 2^{18} \text{ W}$$



③ n 2-addr instructions

1-addr instr = ?

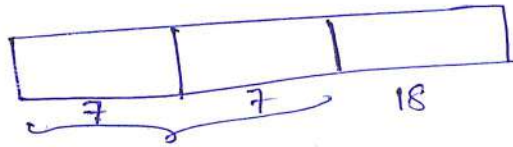
27 possible 2-address Instr

(26)

Given n -address instructions.

Ph: 844-844-0102

$2^7 - n$
1-address instructions



$\Rightarrow (2^7 - n) \times 2^7 \Rightarrow$ 1-address instructions.

(b) n 2-addr instr

m 1-addr instr

0-addr instructions = ?



$\frac{(2^7 - n) \times 2^7}{\text{possible 1-address instructions}}$

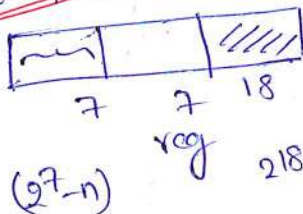
Given m -address instructions

$(2^7 - n) \times 2^7 - m$

$\Rightarrow$ # of possible zero address instructions.

$[(2^7 - n) \times 2^7 - m] \times 2^{18}$ opcode Memory

opcode, reg



No memory

$[(2^7 - n) \times 2^{18} - m] \times 2^7$

③ Auxiliary-carry:

Set if $\exists$ a carry from LN to UN

$$\begin{array}{r} 2A \\ + 34 \\ \hline ACC = 5E \end{array}$$

AC=0

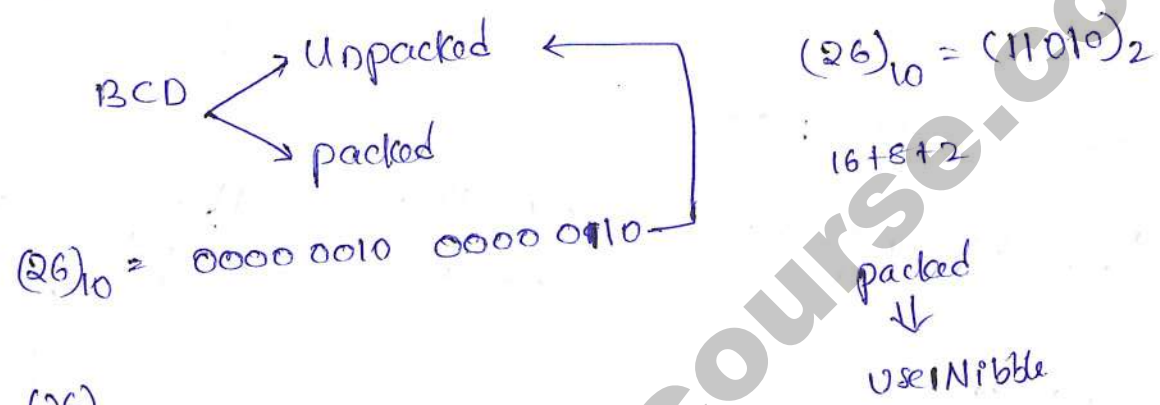
ubits: Nibble

BCDEF

$$\begin{array}{r} 2A \\ 2A \\ \hline ACC = 54 \\ \hline AC=1 \end{array}$$

$$\begin{array}{r} 10 \\ 10 \\ \hline 20 \end{array}$$

Binary-coded-Decimal-Arithmetic: (BCD)



④ $(26)_{10}$

$$\begin{array}{r} 26 = 0010\ 0110 \\ 27 = 0010\ 0111 \\ \hline 53 = 0100\ 1101 = ACC \end{array}$$

CY=0
AC=0

Rule: If $LN > 9$ then add 6 to Accumulator & AC=0

$$\begin{array}{r} 0100\ 1101 \\ 0000\ 0110 \\ \hline 0101\ 0011^3 \end{array}$$

0 --- 15 Hexadecimal
0 --- 9 Decimal

⑤

$$\begin{array}{r} 28 \\ 29 \\ + \\ \hline 57 \end{array}$$

$$\begin{array}{r} 0010\ 1000 \\ 0010\ 1001 \\ \hline 0101\ 0001 \end{array}$$

$$\begin{array}{r} 0101\ 0001 \\ 0000\ 0110 \\ \hline 0101\ 0111 \end{array}$$

Rule: If $LN < 9$ & $AC=1$ then Add 6

$$\begin{array}{r} 88 \\ 89 \\ \hline 177 \end{array} = \begin{array}{r} 1000 \quad 1000 \\ 1000 \quad 1001 \\ \hline 10001 \quad 0001 \end{array}$$

AC=1
CY=1

Rule: AC=1 & LN < 9 add 6 to LN
CY=1 & UN < 9 add 6 to UN

⇒

$$\begin{array}{r} 10001 \quad 0001 \\ 0110 \quad 0110 \\ \hline 10111 \quad 0111 \end{array}$$

177

④ Zero-Flag: Acc=0 ZF=1

2000: LDA 2080

2003: JZ 4000

2004:

4000:

JZ: Jump if zero

PC: 2004
↓
4000

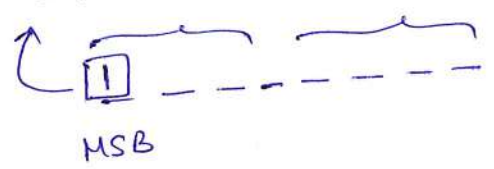
```
if (x==0)
{
}
else
{
}
```

```
for (i=10; i>0; i--)
{
}
=
```

JNZ: Jump if Non-zero

while
switch

⑤ Sign-Flag:



Signed-Integer

(30)

⑥ overflow: Signed - Arithmetic

unsigned $0 - (2^n - 1)$
Ph: 844-844-0102

2's Complement - Binary

0010
0111
0000
1000
1111
1101

Equivalent Decimal

+2
+7
+0
-8
-1
-3

Signed -8 ... -1 +0 ... +7

1000
1's Complement 0111
↓
2's Complement 1011
1000 (-8)

-1 ⇒
0001
↓
1's 1110
2's 1111

-3 ⇒
0011
1's 1100
+1
2's 1101

n-bit
unsigned Integers

$0 - 2^n - 1$

Signed Integers

$-(2^{n-1})$ to $+(2^{n-1}-1)$

②

+7 0111
+3 0011
+10777 1111
1010 ⇒ overflow = 1

(b)
$$\begin{array}{r} +7 \\ -13 \\ \hline +4 < 7 \end{array}$$

$$\begin{array}{r} 0111 \\ 1101 \\ \hline 0100 \end{array}$$

(c)
$$\begin{array}{r} -7 \\ +3 \\ \hline -4 \end{array}$$

$$\begin{array}{r} 11001 \\ 0011 \\ \hline 1100 \end{array}$$

$$\Rightarrow \begin{array}{r} 1100 \\ -4 \end{array}$$

No In carry
No OUT carry

OV=0

	Incarry	out-carry	overflow
(c)	0	0	0
(d)	0	1	1
(a)	1	0	1
(b)	1	1	0

(d)
$$\begin{array}{r} -7 \\ -3 \\ \hline -10 \end{array}$$

$$\begin{array}{r} 11001 \\ 1101 \\ \hline 0110 \end{array}$$

-10 -8 -10 -7
OV=1

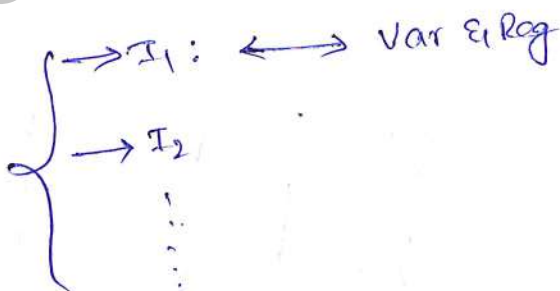
MSB

Incarry XOR out carry

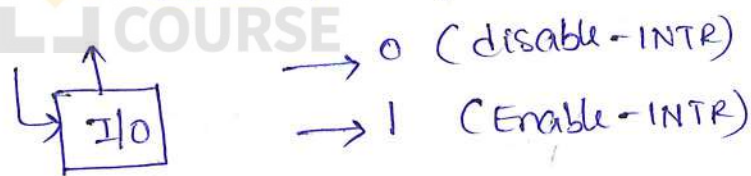
Control Flags:

(1) TRAP $\rightarrow$ 0 (G10)
 $\rightarrow$ 1 (TRCE Single break step)

debugging: gdb

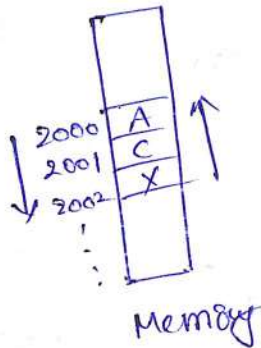


② Interrupt-Flag:

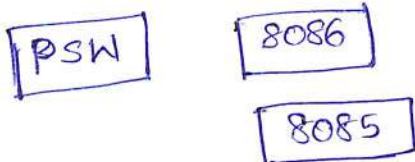


Ph: 844-844-0102

③ Direction Flag: String-instructions → Seq of char



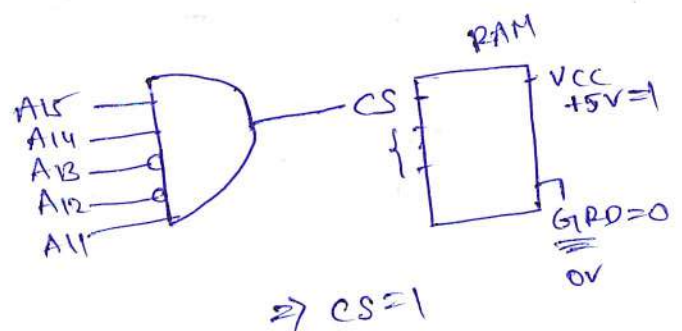
0: L → H ACX
1: H → L XCA



More Solved problems:

① The Chip Select logic for a certain DRAM chip in a single memory system design is shown below. Assume that the memory system has 16 address lines denoted by A15 to A0. What is the range of addresses (in hexa decimal) of the memory system that can get enabled by the chip select (CS) signal?

- Ⓐ 8000 to CFFF
- Ⓑ CA00 to CAFF
- Ⓒ 8000 to C8FF
- Ⓓ DA00 to DFFF



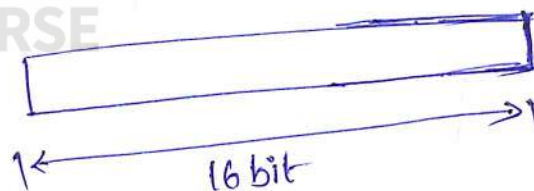
$A_{15} A_{14} A_{13} A_{12} A_{11} A_{10} \dots A_0$
1 1 0 0 1

1100 | 1 000 | 0-F | 0-F
C
1111

C 8-F 0-F 0-F

$\Rightarrow$ C800
 $\downarrow$
CFFF

- ② A processor has 16 integer registers ($R_0, R_1, \dots, R_{15}$) and 64 floating point registers ($F_0, F_1, \dots, F_{63}$). It uses a "2-byte" instruction format. There are 4 categories of instructions:
- Type-1, Type-2, Type-3 and Type-4. Type-1 category consists of 4 instructions, each with 3 integer register operands (3R).
 - Type-2 Category consists of eight instructions, each with 2 floating point register operands (2FS). Type-3 category consists of fourteen instructions, each with one integer register operand and one floating point register operand (1R+1F). Type-4 category consists of N instructions, each with a floating point register operand (1F).
- The maximum value of N is _____


 4 bits
int reg.

float 6 bits

Type 1: $\boxed{4}$ $\frac{4}{4} \frac{4}{4} \frac{4}{4} \Rightarrow 2^{12} \times 4 = 2^{14}$ $\frac{2^{16}}{2^{14}}$

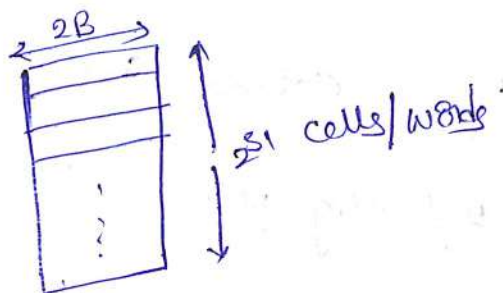
Type 2: $\boxed{8}$ $\frac{4}{4} \frac{6}{6} \frac{6}{6} \Rightarrow 2^{12} \times 8 = 2^{15}$ $\frac{2^{16}}{2^{15}}$

Type 3: $\boxed{14}$ $\frac{6}{6} \frac{4}{4} \frac{6}{6} \Rightarrow 2^{10} \times 14$ $\frac{2^{10} \times 14}{2048} = 2^{11}$

Type 4: $\boxed{N}$ $\frac{10}{6} \frac{6}{6} \Rightarrow N \times 2^6 = 2048$
 $\Rightarrow N = \frac{2^{11}}{2^6} \Rightarrow 2^5 = \boxed{32}$

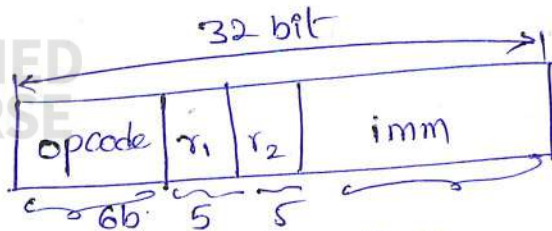
Q3) A processor can support a maximum memory of 4GB, where memory is word-addressable (a word consists of two bytes). The size of the address bus of the processor is at

31 Least bits.



$$\Rightarrow \frac{2^{32}}{2} \Rightarrow 2^{31}$$

Q4) A processor has 40 distinct instructions and 24 general purpose registers. A 32-bit instruction word has an opcode, two register operands and an immediate operand. The number of bits available for the immediate operand field is



40 distinct instructions $\Rightarrow 32 - (6+5+5)$

$\Downarrow$

$$2^5 = 32$$

$$2^6 = 64$$

16 bits

$$\begin{matrix} 2^4 = 16 \\ 2^5 = 32 \end{matrix} \} \text{reg}$$

Q5) Consider a processor with 64 registers and an instruction set of size twelve. Each instruction has five distinct fields, namely opcode, two source register identifiers, one destination register identifier, and a twelve-bit immediate value. Each instruction must be stored in a memory in a byte-aligned fashion. If a program has 100 instructions the amount of memory (in Bytes) consumed by the program text is _____

opcode, $\downarrow$ 4b
 $sr_1, \downarrow$ 6b
 $sr_2, \downarrow$ 6b
 $d_1, \downarrow$ 6b
 12bit imm $\downarrow$ 12b
 twelve $\downarrow$ 4 bits

$$\frac{34b}{8} = 4.25B \approx 5B$$

$$5 \times 100 = 500B$$



(3)

(Q6)

For computers based on 3-address instruction format, each address field can be used to specify which of the following

S1: A memory operand

S2: A processor register

~~X~~ S3: An implied accumulator register.

~~(A)~~ Either S1 or S2

(B) Either S2 or S3

(C) Only S2 and S3

(D) All of S1, S2 and S3.

3-address

opcode addr1, addr2

(Q7)

Consider two processors P1 and P2 executing the same instruction set. Assume that under identical conditions for the same input, a program running on P2. takes 25% less time but incurs 20% more CPI (clock cycles per instruction)

a) Compared to the program running on P1. If the clock frequency of P1 is 1GHz, then the clock frequency of P1 is 1GHz, then the clock frequency of P2 (in GHz) is _____

~~(a)~~ 1.6

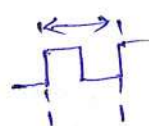
(b) 3.2

(c) 1.2

(d) 0.8

P1 P2

$f \geq 1 \text{ GHz}$ $f = ?$



$$\Rightarrow t = \frac{1}{f}$$

$$= \frac{1}{1 \times 10^9}$$

$$= 10^{-9} \text{ s} = 1 \text{ ns}$$

P₁

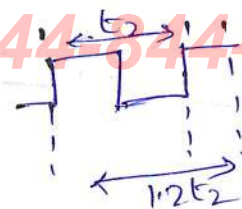
1 CPI

$$t_1 = 1 \text{ ns}$$

P₂

1.2 CPI

$$t_2 = 0.75 \text{ ns}$$



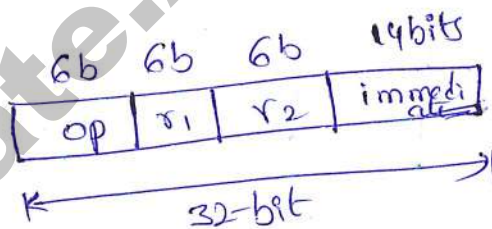
$$\Rightarrow 0.75 \text{ ns} = 1.2t_2$$

$$\Rightarrow t_2 = \frac{0.75 \text{ ns}}{1.2}$$

$$f_2 = \frac{1.2}{0.75} \text{ GHz}$$

$$= \boxed{1.6 \text{ GHz}}$$

Q8) A machine has 32-bit architecture, with two long instructions. It has 64 registers, each of which is 32 bits long. It needs to support 45 instructions, which have an immediate operand in addition to two register operands. Assuming that the immediate operand is an unsigned integer, the maximum value of immediate operand is _____

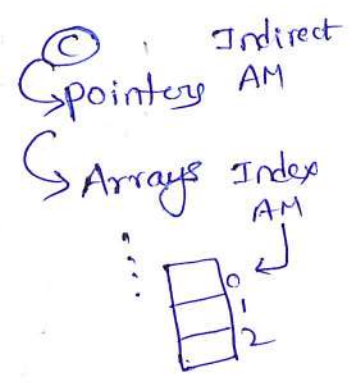
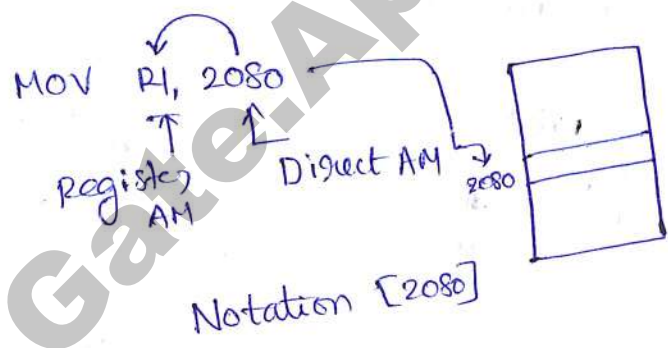
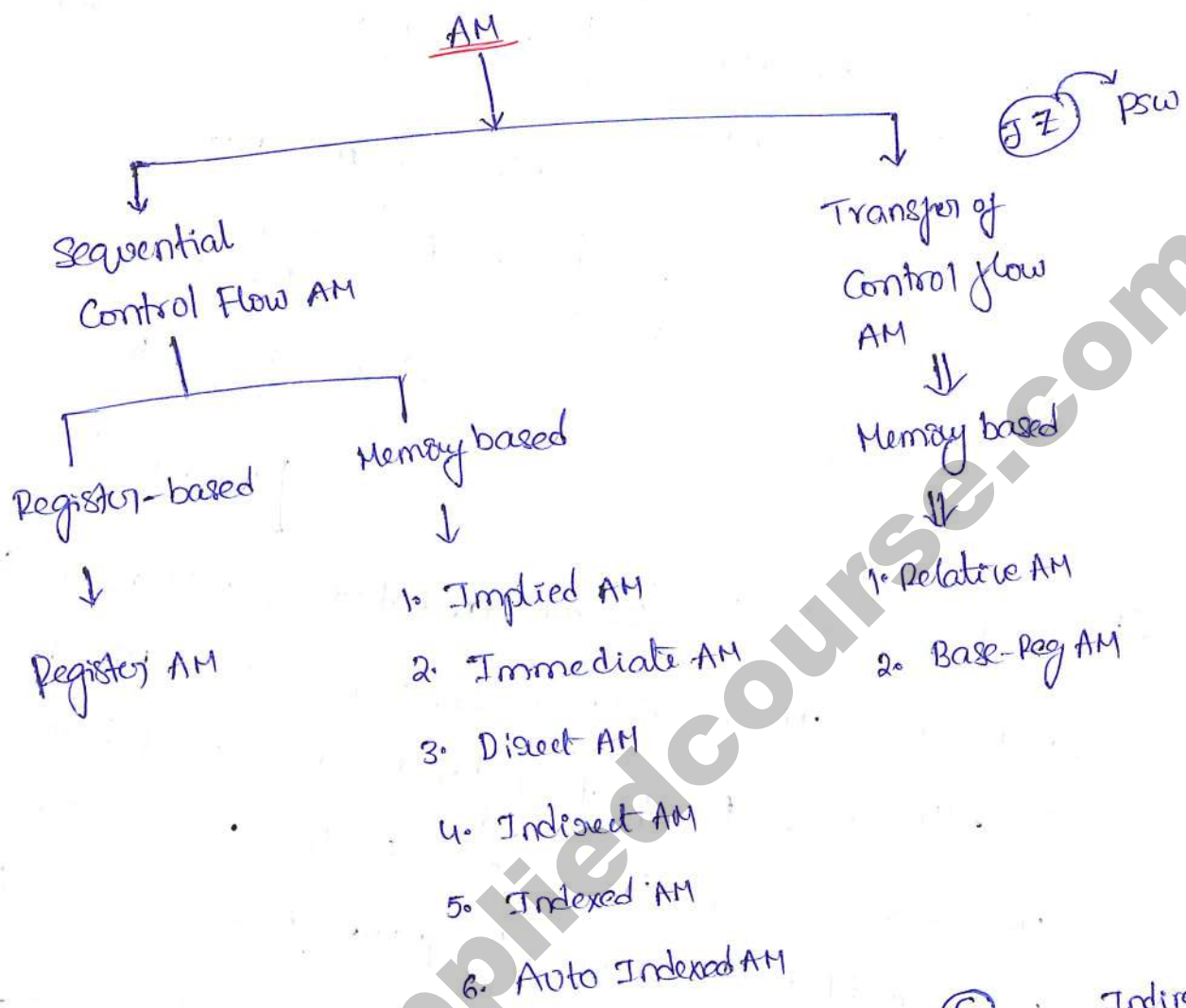


14bits

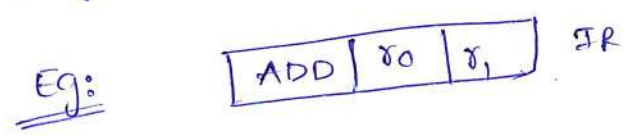
$$0 \text{ to } (2^{14} - 1)$$

$$\Rightarrow 16384 - 1$$

$$\Rightarrow \boxed{16383}$$

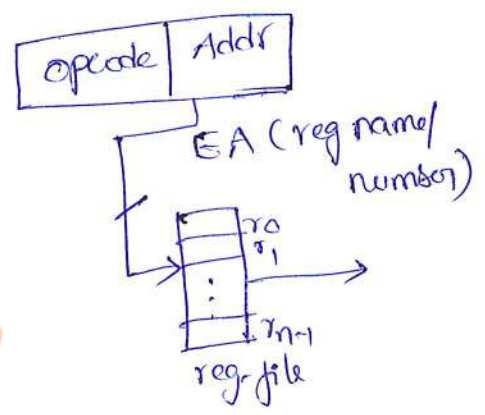


Register-based AM:



$r0 \leftarrow r0 + r1$

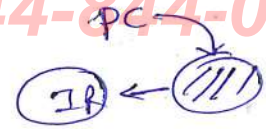
Effective Address



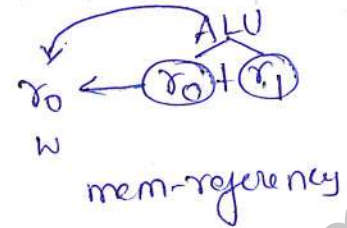
Instruction decoding stage (ID)

Ph: 844-844-0102

Register 2 memory Reference (Reg)



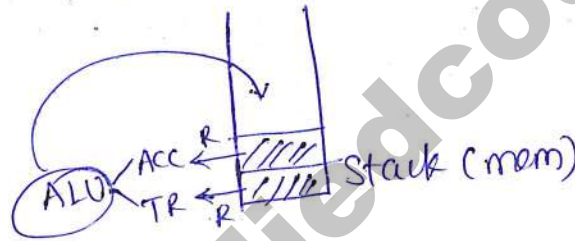
Execute stage: 3-reg mem-ref



Implied/ Implicit AM:

eg: STC 1 reg mem ref Set carry $cy \leftarrow 1$ PSN/flag-Reg
CLC Clear carry $cy \leftarrow 0$
ADD (stack opn)

0-addr-instr
opcode



Zero-addressing instructions

Immediate AM:

(Eg) MOV r_0 $(23)_{16}$ $r_0 \leftarrow 23$
Source
Constant

Range is limited
↑
data (limited bits)

int a = 56;

(Eg) MOV $(23)_{16}$ r_0
dest Src
Meaningless

$(23)_{16} \leftarrow r_0$ X

int 56 = a;

⑥ up with multiplexed $AD_0 - AD_{23}$

Ph: 844-844-0102

Address - 24 bits

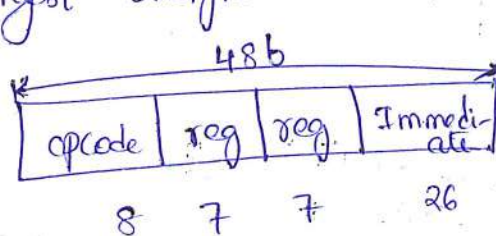
Data - 24 bits

⑧ 256 2-word instructions

⑨ 128 processor-registers

Inst format: opcode, reg, reg, immediate

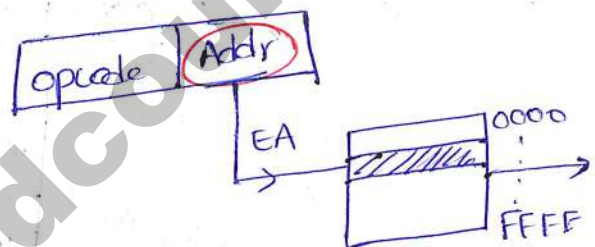
Largest unsigned number in immediate field?



$$= 2^{26} - 1$$

Direct / Absolute AM:

Eg: $\uparrow \text{reg} \uparrow \text{EA}$
Add $r_0, [2080]$
 $r_0 \leftarrow r_0 + m[2080]$



Usage: static-variables

int main() {
static int i = 0;
}
 $\rightarrow$ Dir AM
 $\rightarrow$ Imm AM

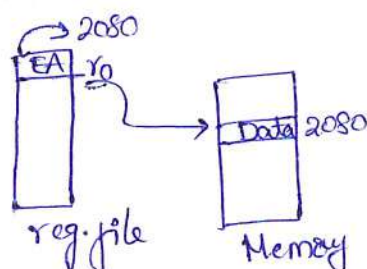
MOV $[2080] (00)_{16}$

Compiler-Design
Symbol Table

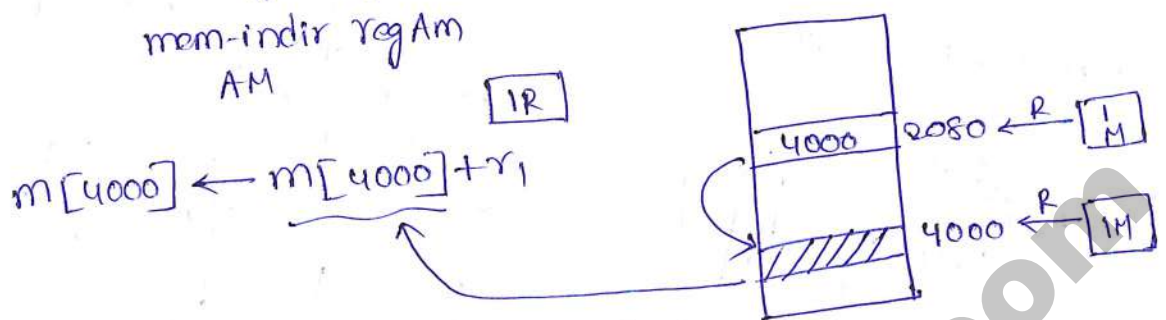
Life-time of static variable: program throughout the

Indirect Addressing Mode:- pointers

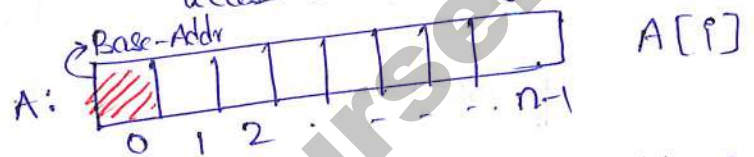
Eg: Add $@r_0, r_1$ (memory Register Indirect)
 $\downarrow$ $\downarrow$
 reg.indir AM reg.AM
 [r_0]



Eg: Add @2080, r₁ (Memory-Indirect AM)



Indexed AM: Arrays → Randomly access data @ any index



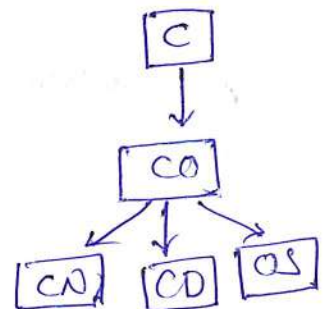
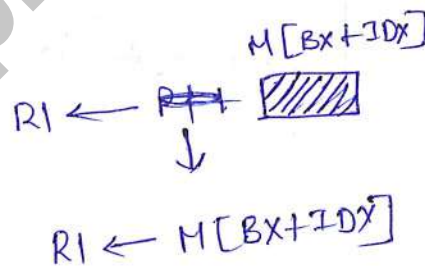
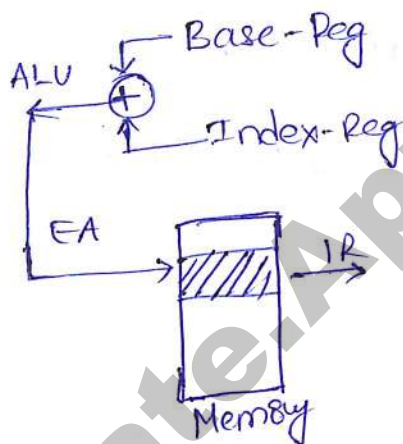
Eg: `MOV RI, BX, IDX`

↓

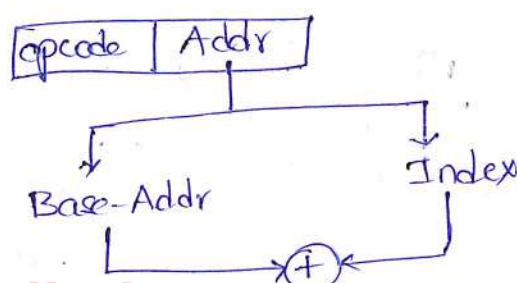
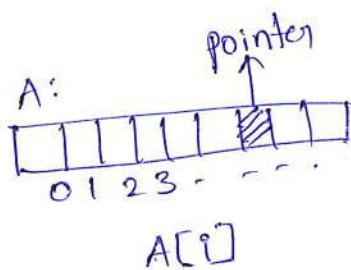
Reg AM

Base: Addr → Base-addr-Reg

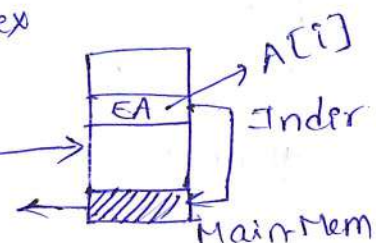
IDX: Index-Reg



Indirect-Indexed AM:



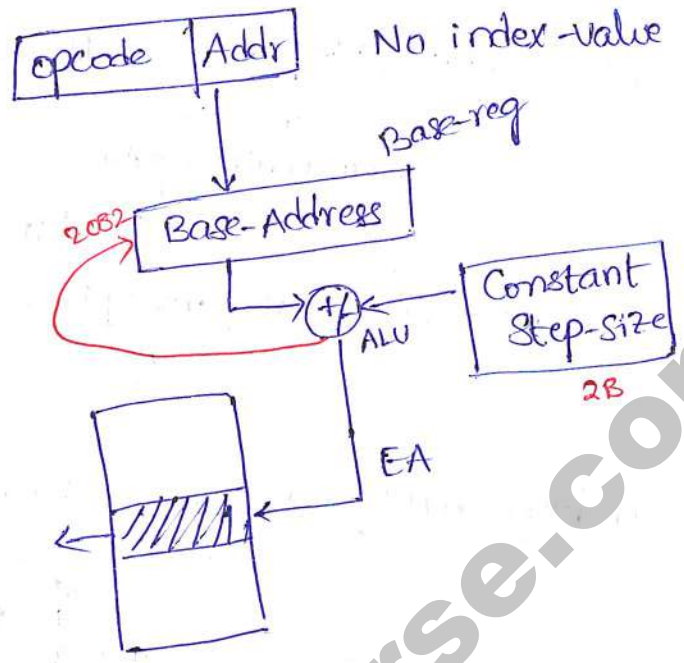
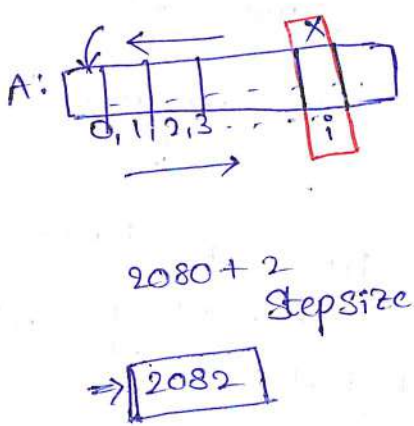
Indexed part



Auto Indexed AM:

Linear-array-elements + linear-access Ph: 844-844-0102

- Auto-increment AM
- Auto-decrement AM



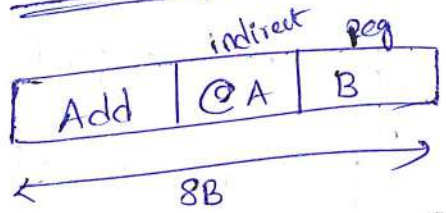
- Auto-increment AM
 - Pre-auto incr $MOV R_0, +(R_1)$ C: $x++$ $y++$
 - Post auto incr $MOV R_0, (R_1)++$
- Auto-decrement AM
 - Pre-auto decr $MOV R_0, -(R_1)$
 - Post auto decr $MOV R_0, (R_1)--$

$R_1 : 2080$
 $S-S : 2B$

More Solved problems:

Q

32 bit up

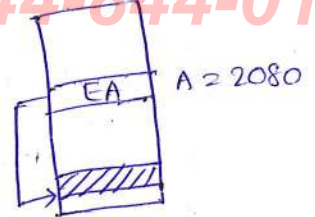


A: Mem-addr
B: Reg-addr

memory-cycle required during execution-cycle.

Fetch	Execute
2 Mem-R	① 2 mem-R ② 1 reg-R ③ Arithmetic ④ 1 mem-R + 1 mem-W

ID: 0 1-Reg R

① 3.2 GHz μ P

AM	$\#$ reg	
Reg	20% $\rightarrow 0 \Rightarrow 0$	
Immediate	18% $\rightarrow 0 \Rightarrow 0$	
Direct	22% $\rightarrow 8 \Rightarrow 0.22 \times 8$	
Mem-Indirect	17% $\rightarrow 8+8 \Rightarrow 0.17 \times 16$	
Indexed	11% $\rightarrow 4+8 \Rightarrow 0.11 \times 12$	
Auto-Indexed	12% $\rightarrow 4+8 \Rightarrow 0.12 \times 12$	
		<u>7.24 Cycles</u>
		7.24×0.3125
		$\Rightarrow 2.26 \text{ ns}$
		$\Rightarrow 1W$

8-clock cycles: Mem-Reg

4: Arithmetic of
0: Register-operandAvg-fetch-rate = ?
(in W/s)

$$f = 3.2 \text{ GHz}$$

$$\Rightarrow t = \frac{1}{3.2} \times 10^9$$

$$= 0.3125 \text{ ns/cycle}$$

Avg-fetch rate = ?

$$= \frac{1}{2.26 \times 10^9}$$

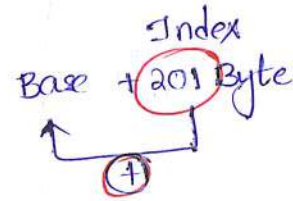
$$= 442.4 \text{ MW/s}$$

Q3) Consider the C statement defining below:

Ph: 844-844-0102

Statement data {

Let
int marks[100]; $\rightarrow 2B$
char grade; $\rightarrow 1B$
int number; $\rightarrow 2B$
};



Statement data Student;

The base address of the student is available in register R1.

The field student.grade can be accessed efficiently using

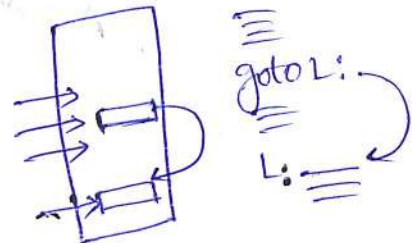
- ☒ (A) post-increment addressing mode $(R1)^+$
- ☒ (B) post-decrement addressing mode $-(R1)$
- ☒ (C) Register direct addressing mode $R1$
- ☒ (D) Index addressing mode, $X(R1)$, where X is an offset represented in 2's Complement 16-bit representation.

Transfer of Control AM: Based & Relative

$\rightarrow$ PC-relative AM

$\rightarrow$ Based/ Base-reg AM

$\rightarrow$ Jump/Branch: unconditional



$\rightarrow$ Conditional Jump/Branch

if ZF in psw=1

for (. ; ;)
 {
 ...
 }

while ()
 {
 ...
 }

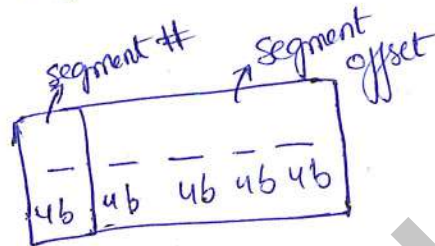
Ph: 844-844-0102

Segmented Memory :

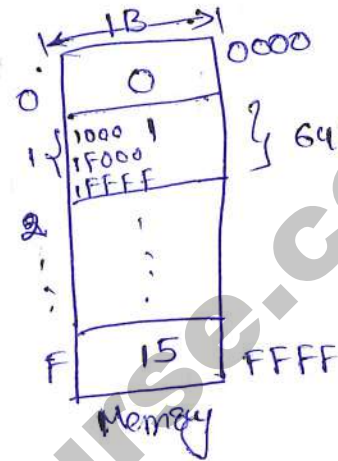
20 bit addr

2^{20} cells = 1MB mem

$$\frac{2^{20}}{2^4} = 2^{16} = 64KB$$

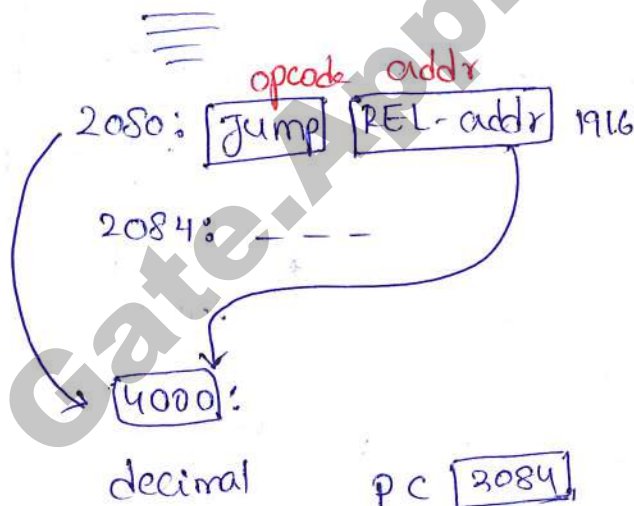


OS: paging, seg, VM



16 - equal sized Segments

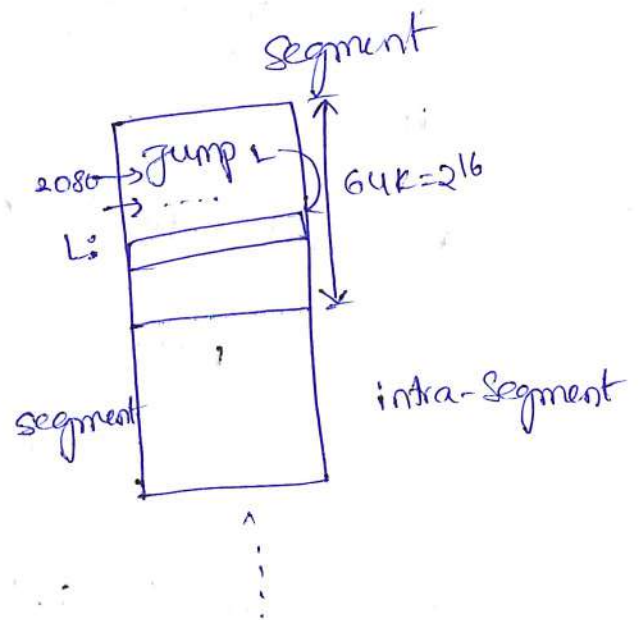
PC-relative AH:



PC 2084

$$4000 - 2084$$

$$= 1916$$



(46)

Base-Addr Am: / Based-Am

$$EA = \underset{1000}{Base-Addr} + \underset{2345}{addr-offset}$$

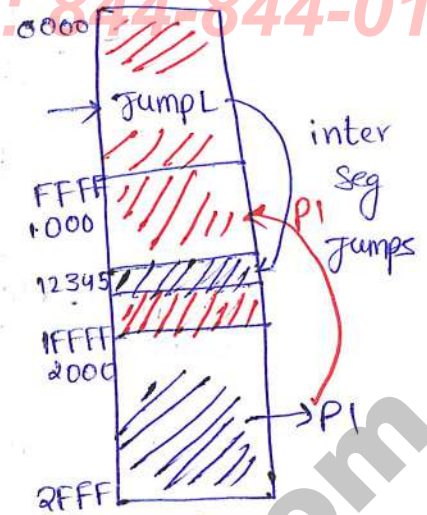
Dynamic program-relocation



jump Bx offset

jump R1 2345

Ph: 844-844-0102



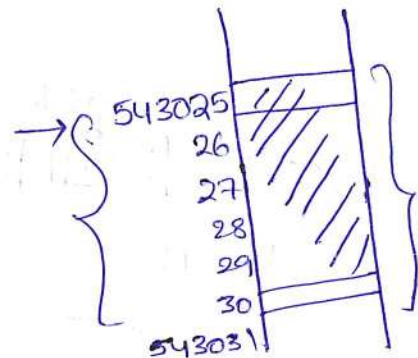
Solved problems:

- ① 6B long pc-relative jump-instr started at addr (543025)₁₀ onwards. JUMP Rel-addr (-48) Signed displacement is present in the addr-field of the instruction.

- ② what is the branch-addr:

$$PC: 543031 + (-48)$$

$$= \boxed{542983}$$



- ③ If base-reg contains 683823; what's the branch address if base-reg-addr is used.

jump R1 -48

$$\begin{array}{r} \text{Base reg: } 683823 \\ + \quad -48 \\ \hline \end{array}$$

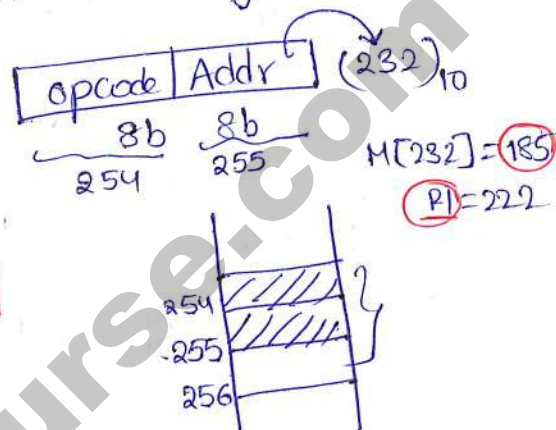
(C) based Indexed AM is used with index-value = 32

branch-addr = ?

$$683823 + 32 = 48$$

(Q2)

A 2B-instruction is present in the memory location starting at $(254)_{10}$.



(a) calc EA if immediate AM

→ 255

(b) Reg AM: Reg Address of R1

(c) Direct AM: 232

(d) Reg-indirect AM: 222

(e) Mem-indirect AM: 185

(f) Indexed-AM (R1: index-reg): $BA + PA$
 $232 + 222 = 454$

(g) PC-relative AM

→ PC: $256 + 232$
 = 488

(Q3)

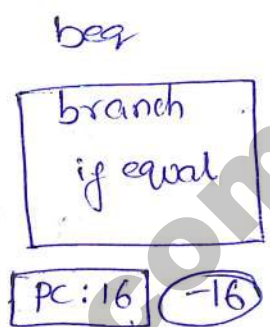
Consider a RISC machine where each machine is exactly 4 bytes long. Conditional and unconditional branch instructions use pc-relative addressing mode with offset specified in bytes to the target location of the branch instruction. Further the offset is always with respect to

the address of the next instruction, in the program sequence.

Ph: 844-844-0102

consider the following instruction sequence.

Instr. No	Instruction
0-3 → i	add R2, R3, R4
4-7 → i+1	sub R5, R6, R7
8-11 → i+2	cmp R1, R9, R10
12-15 → i+3	beq R1, offset
16 →	



If the target of the branch instruction is i, then the decimal value of the offset is -16.

* Instruction Set: Introduction: *

- ① Data-Transfer : MOV, LOAD, STORE, PUSH, POP, IN, OUT etc.
- ② Data Manipulation
- ③ Transfer of Control / program control
 - ↳ jump
 - JZ
 - JNZ...

Data-Manipulation-instr:

Data sheet

- ① Arithmetic instruction: ADD, MUL, SUB, DIV, INC, DEC etc

① INC R0

R0: 1111 1111

0 ← CY: 0000 0000

② DEC R0

R0: 0000 0000

CY: 1111 1111

cy bit does not change.

```
for (i=0; i<10; i++)
{
    ...
}
```

Ph: 844-844-0102

```

1 0 0 0 0 0 0 0 0
  ↓
  0 1 1 1 1 1 1 1
  
```

② Logical Instructions : AND, OR, XOR, CMP etc.

① CMP r_0, r_1 $r_0 - r_1 \rightarrow \text{SUB (ALU)}$

$r_0 = r_1 \Rightarrow \text{CY: 0} \quad \text{Z: 1}$

$r_0 > r_1 \Rightarrow \text{CY: 0} \quad \text{Z: 0}$

$r_0 < r_1 \Rightarrow \text{CY: 1} \quad \text{Z: 0}$

PSW

```

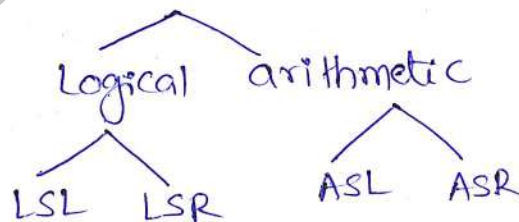
      CY
      ↓
  1 0 1 1 1 0 1 1  r0
  1 0 1 0 1 0 1 0  r1
  -----

```

```

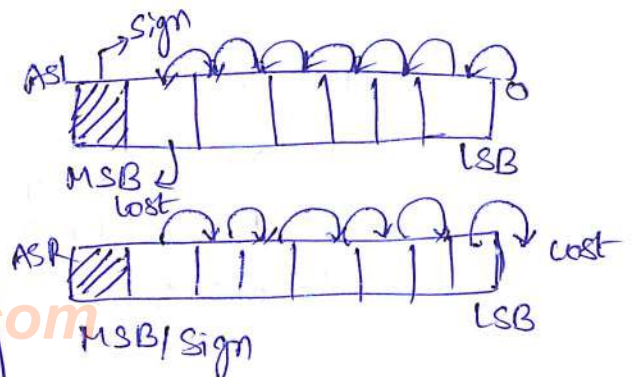
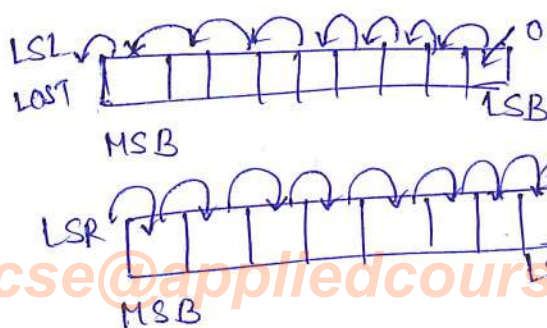
      CY
      ↓
  1 0 1 0  ← r0
  - 1 0 1 1 ← r1
  -----
  1 1 1 1
  
```

③ Shift operations :-



$a \ll 2; \quad a \gg 2;$

Signed: Numerical



4

Rotate Instruction:

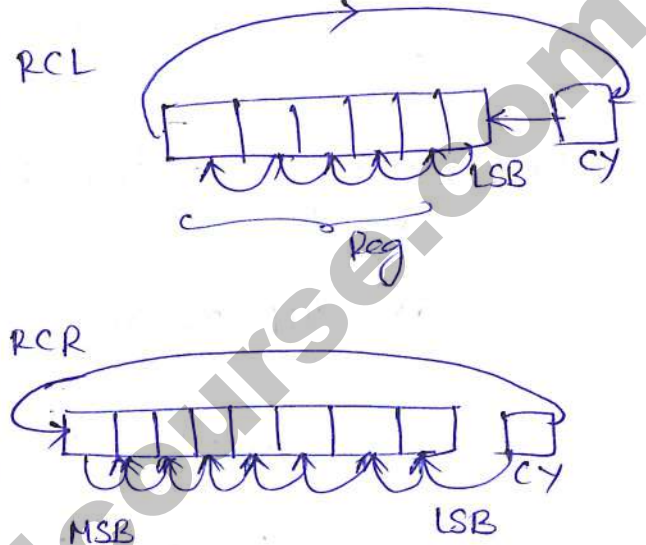
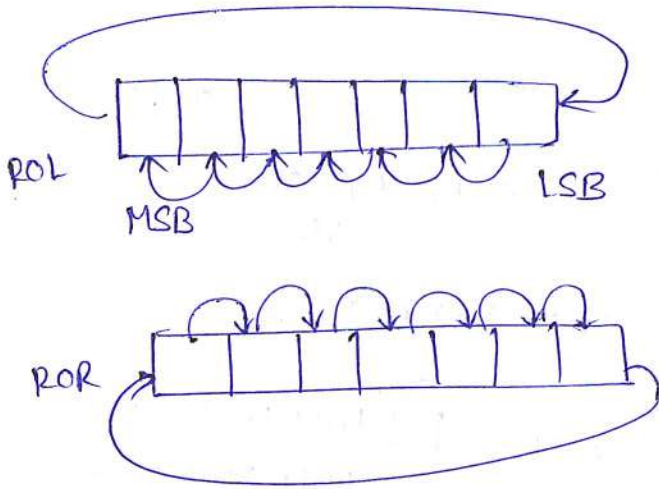
without carry

with carry

ROL ROR

RCL RCR

Rotate with Carry Right



Transfer of Control / program control Instructions:

unconditional

conditional

JMP

JMP

JMP

JMP

HALT

JZ

sub-routines /
sub-programs /
junctions

① Jump / BRANCH

1000: J1

1002: J2

→ 1004: J3 (HALT) JUMP [2000] PC: 1003 PC: 2000

1006: J3

1006 : I₃

2000 : I₄

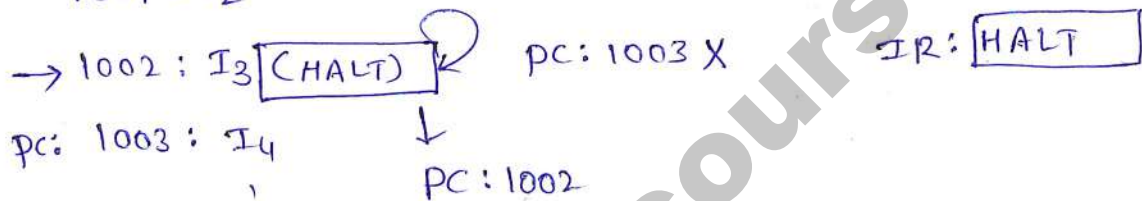
2002 : I₅

- ② HALT → User view : Computer is Halted/Stopped RESTART
 → CPU-view → HALT ⇒ (PC = 1002)

HLT

1000 : I₁

1001 : I₂



- ③ JZ & JNZ (Conditional Transfer of Control)

✓ 1000 : MOV r0, (03)₁₆

✓ 1001 : I₂

✓ 1002 : DEC r0

1004 : JNZ [1001]

→ 1006 : I₃

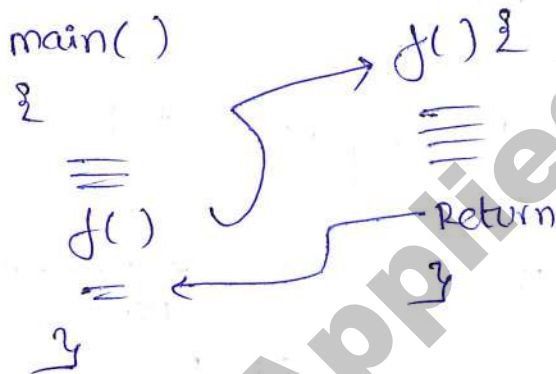
1007 : I₄

r0: ~~03~~ → ACC & ALU

ZF: 1

ZF: 1 if ACC = 0

ZF: 1 if result of prev-instr is zero



Direct vs Indirect Transfer of Control

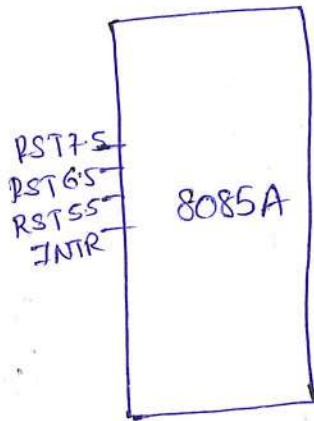
EA & Addressing Mode

RET → unconditional

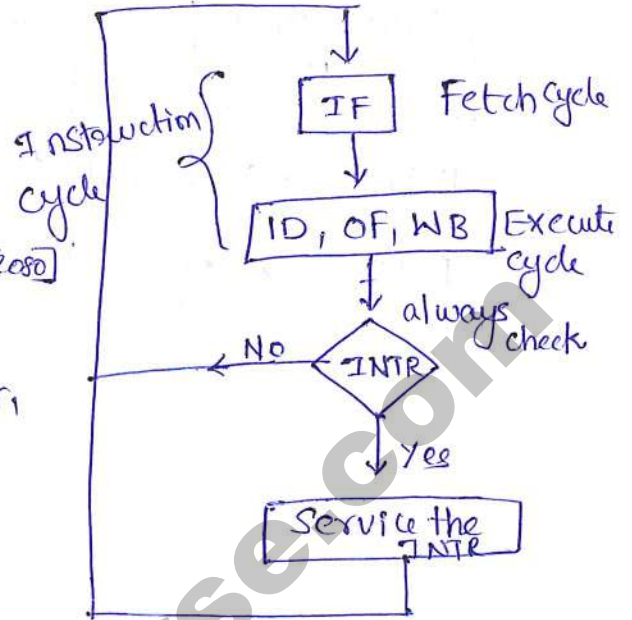
RET Z, RET NZ
RET C, RET NC } Conditional
RET

Instruction Cycle with interrupts + problems:

PI: 844-844-0102



→ MOV R0, [2080]
↕
→ ADD R0, R1



- RST 7.5
- RST 6.5
- RST 5.5
- INTR

H/W interrupt pins

IOT

INTA - Interrupt Acknowledgement

Instruction Fetch

Instruction Decode

Operand Fetch

Write back

Interrupt Cycle

① Check the INTR pins

② Context-switch/Transfer of Control

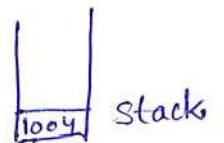
using Interrupt vector table (IVT)

③ Interrupt sub-routine (ISR)

→ I₃

④ IRET/RETI

Interrupt Return/Return Interrupt



- 1000: I₁
- 1002: I₂ ← INTR I₃
- 1004: I₃ ← check INTR pins
- 1005: I₄

I ₁	2080
I ₂	3080
I ₃	4080
I ₄	5080

4080:
IRET

IVT

(94)

Single-INTR VS Nested-INTR processing

Ph: 844-844-0102

Disable other INTR.

INTR $\rightarrow$ Interrupt Cycle
RST 5.5 $\rightarrow$ ignore this

priority based

RST 7.5 ✓
6.5 ✓
5.5 ✓
INTR ✓

IoT

Need to place the highest priority interrupt first

Oven

INTR $\rightarrow$ IC INTR
 $\downarrow$
RST 5.5
 $\downarrow$
I.C 5.5

PC INTR
PC original

(Q1)

Instruction

Meaning

Size (W)

Addr IFG ID WD

I₁: MOV R0, @3000

$R_0 \leftarrow M[3000]$

2

5230 16 16

I₂: MOV R1, @2000

$R_1 \leftarrow M[2000]$

2

5238 16 16

I₃: Add R0, R1

$R_0 \leftarrow R_0 + R_1$

1

5246 8 2

I₄: MUL R0, R1

$R_0 \leftarrow R_0 \times R_1$

1

5250 08 2

I₅: SUB R0, [4000]

$R_0 \leftarrow R_0 - M[4000]$

2

5254 16 2+8

I₆: MOV 3(R0), R1

$M[30 + (R_0)] \leftarrow R_1$

2

5262 16 2+8

INT $\rightarrow$ I₇: Halt

Stop

5270 8 -

5274 88 56

a) Byte-address-memory 32 bit μP

Starting-addr = $(5230)_{10}$

INTR occurs during execution of Halt

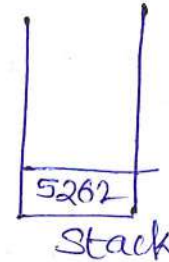
what's the return address pushed to the stack?

Gatecse@appliedcourse.com



- (b) INT occurs during execution of SUB, Return-addr pushed into Stack=?

while executing IS
PC 5262
contains



- (c) $\mu P : 2.5 \text{ GHz}$

IF&ID: 8 cycles/word

$$\text{Cycle time} = \frac{1}{2.5 \text{ GHz}}$$

$$= 0.4 \text{ ns}$$

Arithmetic op: 2 cycles

Mem-ref: 8 cycles

Reg-ref: 0 cycles

execution time: = ?

$$\text{Total cycles} = 144$$

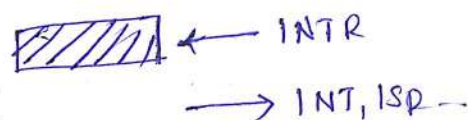
$$\Rightarrow 144 \times 0.4 = 57.6 \text{ ns}$$

② up that supports 4 INTR : I_1, I_2, I_3, I_4

Ph: 844-844-0102

ISP service time: 20ns, 40ns, 16ns & 10ns

Response time of INTR : 5ns



priority: $I_1 > I_2 > I_3 > I_4$

Max & Min time required to service I_4 given nested - INT - processing.

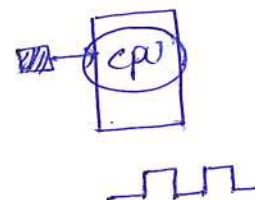
If only I_4 Interrupt occurs then time to resolve I_4 : 5 + 10

Min Time = 15ns

Max time to resolve I_4 = 25 + 45 + 21 + 15 = 106ns

If high priority Interrupts are coming frequently then serving of I_4 may lead to starvation.

Types of Interrupts:



① Hardware: Pine

② External: I/O devices & sensors

③ Internal: clock-error, cpu-temp, invalid-opcode, div-by-zero, stack-overflow.

② Software Interrupts!

8085 RST 0
RST 1
RST 2
!
RST 7

INT

INTR	Addr
RST 0	00H
RST 1	08H
RST 2	10H
RST 3	18H
RST 4	20H
RST 5	28H
RST 6	30H
RST 7	38H

8085

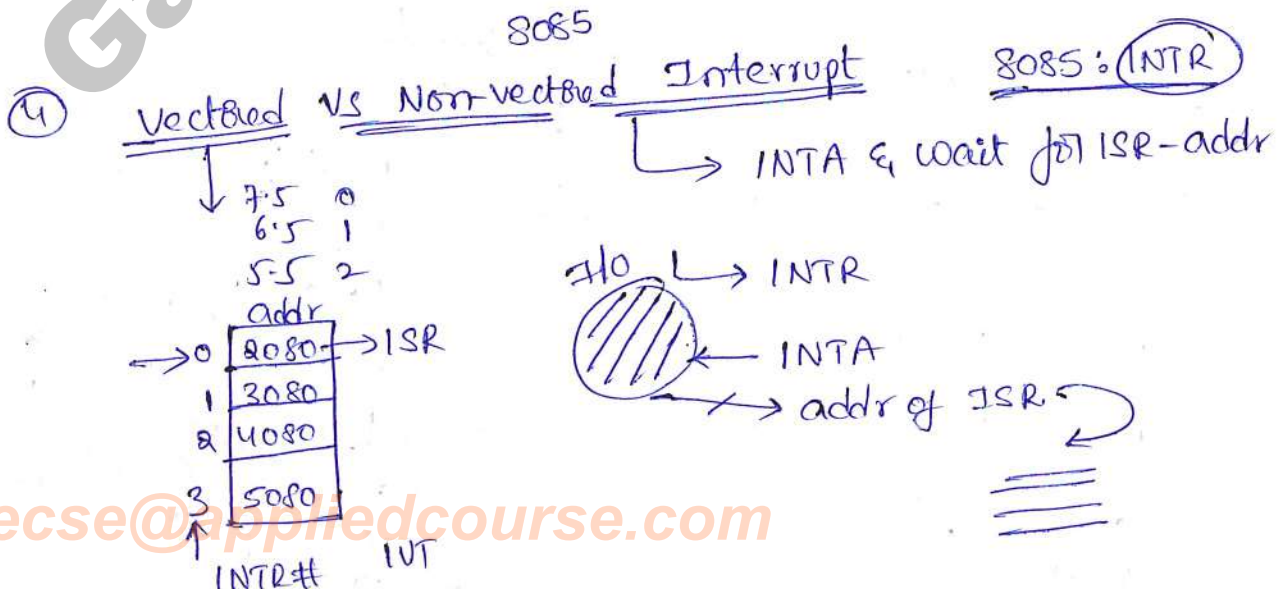
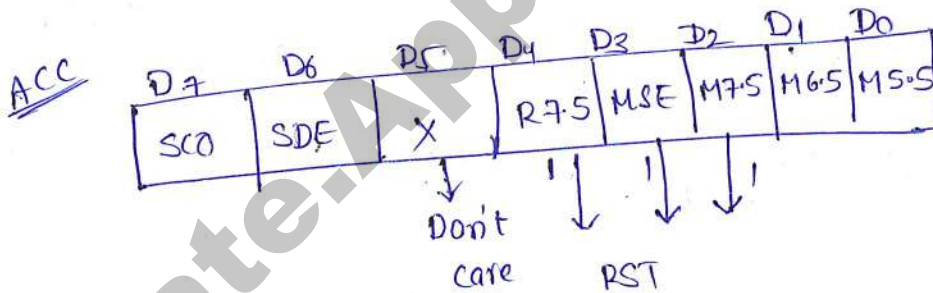
Non-Maskable

Maskable

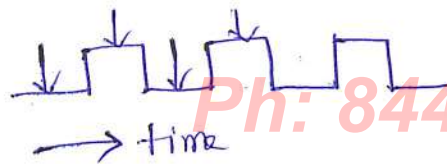
③ Maskable & Non-Maskable INTR

Ignore (RST 7.5 6.5 5.5) (TRAP) cannot be ignored by the Micro processor

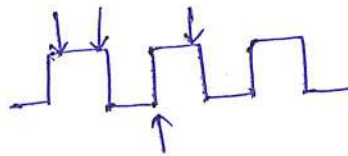
Eg: SIM : Set Instruction Mask
RIM : Read Instruction Mask.
Zero - addr - instruction.



⑤ Level-Triggered
Elec. Engg



⑥ Edge-Triggered



RISC (Reduced Instruction Set Computer) (VCB, ...)

CISC (Complex Instruction Set Computer)

CISC MUL [2080] [3080] 4 cycles
 $\left\{ \begin{array}{l} \text{MUL } R_0, R_1 \\ \text{MUL } R_0, [2080] \end{array} \right.$

- More # of instr ⇒ More Circuitry
- fewer registers
- Variable # cycle Instr
- Pentium, i3, i5, i7, ... (Win)

RISC 1 LOAD R0, [2080]
 1 LOAD R1, [2080]
 1 MUL R0, R1
 1 STORE [2080], R0

- fewer # of instr ⇒ Simpler Circuitry
- More # of reg ⇒ Pipelining
- 1 cycle / instr
- ARM mobile, ipads, ...

cpu → Laptops → pipelining

Register Organization in RISC:

$$\text{Total \# of reg} = g + wxl + \frac{w(2xc)}{2}$$

g: # of Global-reg

L: # of local reg / window

c: Common-reg / window

w: # windows



Global registers file



reg window

C: common
L: local



over lapped reg windows

Q1.

Consider the following processor design characteristics.

Ph: 844-844-0102

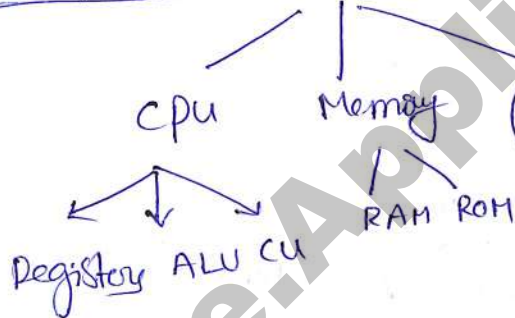
- I. Register-to-Register arithmetic operations only.
- II. Fixed-length instruction format 1 cycle/Instr
- III. Hard wired control unit

→ DLC

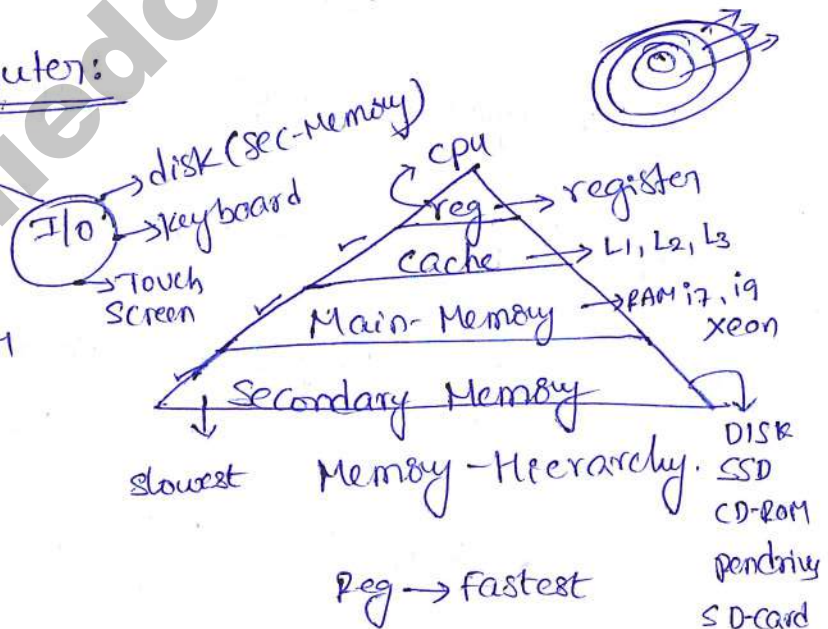
IV. which of the characteristics above are used in the design of a RISC processor?

- (A) I and II only
- (B) II and III only
- (C) I and III only
- (D) I, II and III

Components of a Computer:



for $i = 0$ to 1000



cpu - 2.4 GHz
RAM: 144 MHz

① ↑ performance $\propto \frac{1}{\text{execution Time}}$ ↓

② ↓ execution time $\propto$ access-time ↓

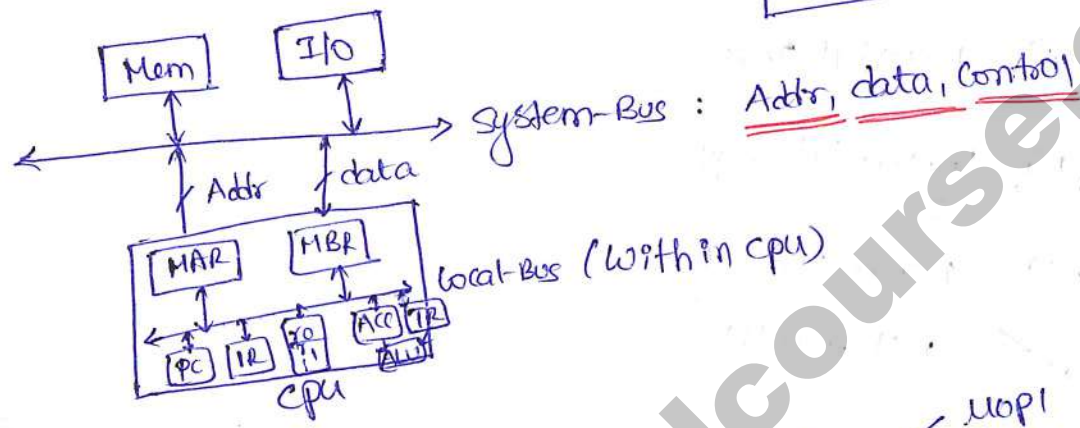
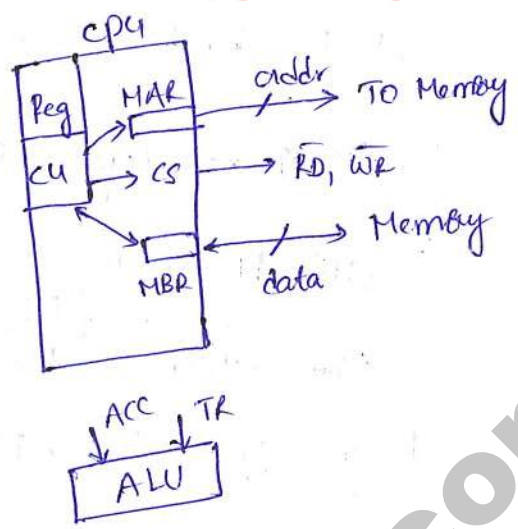
⇒ ↑ performance $\propto \frac{1}{\text{access-time}}$ ↓

LDA @2080 | MOV r0, r1

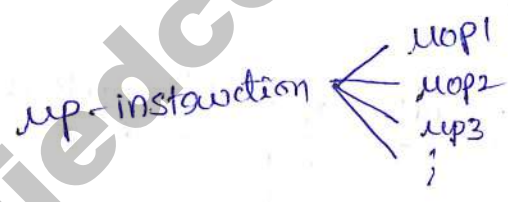
Registers: PC, IR, ACC, TR, MAR, MBR/MDR, PSW, GPR

Memory-Addr-Reg : Stores the address temporarily (MAR)

Memory Buffer/data Reg (MBR) ↓ Stores the data temporarily.



Micro-operation (uop)



Eg: MOV r0, r1 → single uop

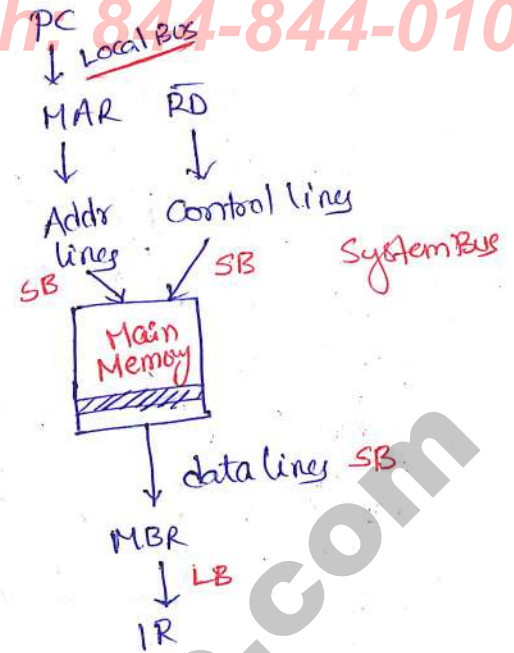
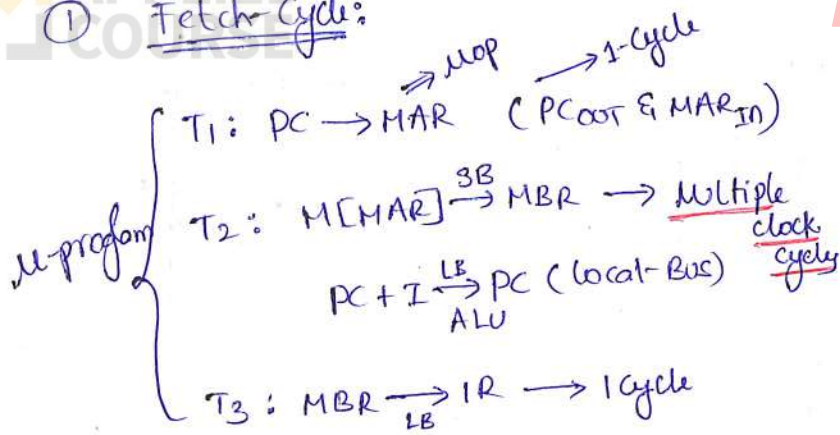
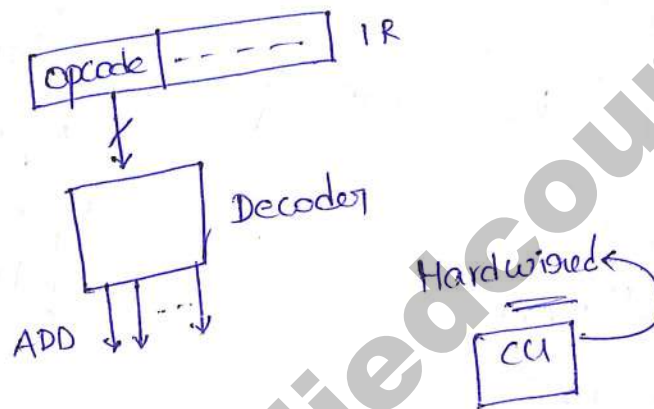
- cycles for uop → 1 clock cycle
- Memory uop (multiple-cycles)

→ Control Signals : seq of cs



Micro-program (u-program) : sequence of u-operations



① Fetch Cycle:② Execution-Cycle: Instruction-Decode③ Execution-cycle: operand Fetch (OF)④ Direct Mode:

e.g) LDA [2080]

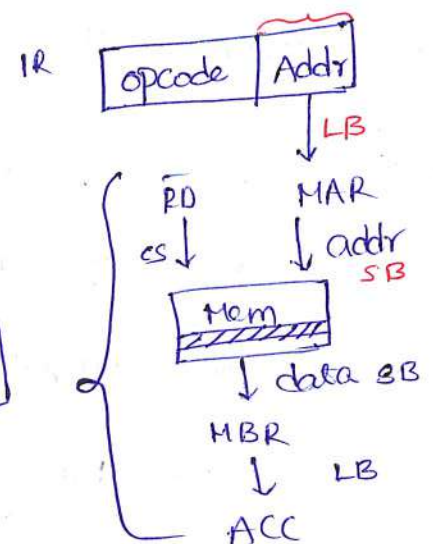
T₁: IR[addr] $\xrightarrow{LB}$ MAR

T₂: M[MAR] $\xrightarrow{SB}$ MBR

$PC + I \xrightarrow{LB} PC$

T₃: MBR $\xrightarrow{LB}$ ACC

Multiple clock cycles



④ Indirect-Mode: LDA @2080

T1: $IR[addr] \xrightarrow{LB} MAR$

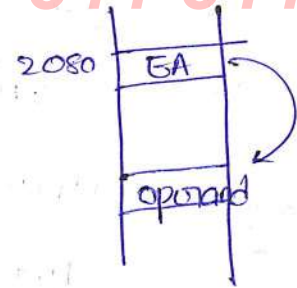
T2: $M[MAR] \xrightarrow{SB} MBR$ [2080]

$PC + 1 \xrightarrow{LB} PC$

T3: $MBR \xrightarrow{LB} MAR$

T4: $M[MAR] \xrightarrow{SB} MBR$ operand

T5: $MBR \xrightarrow{LB} ACC$



④ Execution-stage: Write-back

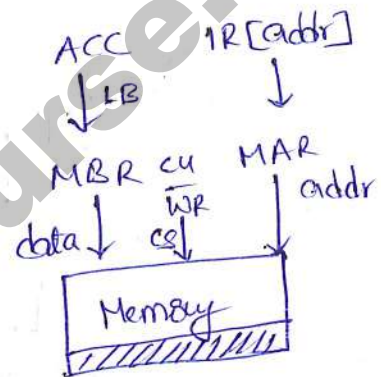
Eg: $STA [2000]$ direct-Addressing

T1: $ACC \xrightarrow{LB} MBR$

T2: $IR[addr] \xrightarrow{LB} MAR$

T3: $MBR \xrightarrow{SB} M[MAR]$

$PC + 1 \xrightarrow{LB} PC$



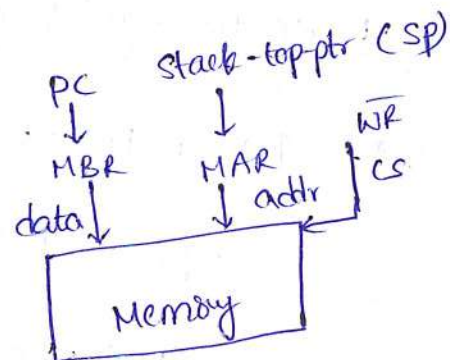
⑤ INTR-Cycle:

T1: $PC \rightarrow MBR$

T2: $SP \rightarrow MAR$

T3: $MBR \rightarrow M[MAR]$

T4: $ISR-addr \rightarrow PC$



$ISR-addr \rightarrow PC$

ADD r0 r1

$\left. \begin{array}{l} r0 \xrightarrow{LB} ACC \\ r1 \xrightarrow{LB} TR \\ ACC + TR \rightarrow ACC \\ ACC \xrightarrow{LB} R0 \end{array} \right\}$

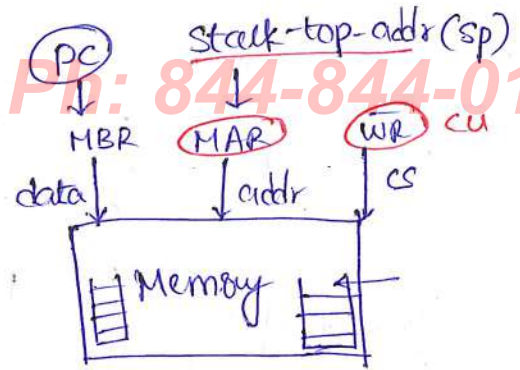
⑤ INTR - Cycle:

✓ T₁: PC $\xrightarrow{LB}$ MBR

✓ T₂: SP $\xrightarrow{LB}$ MAR

T₃: [MBR $\rightarrow$ MCMAR]

✓ T₄: ISR-addr $\rightarrow$ PC $\rightarrow$ IVT
 $\parallel$
 $\rightarrow$ MBR $\rightarrow$ PC



⑥ which of the following operations is satisfied by the μ -prog

T₁: PC $\rightarrow$ MBR

T₂: X $\rightarrow$ MAR

T₃: MBR $\rightarrow$ memory

T₄: Y $\rightarrow$ PC

PC $\rightarrow$ Memory (stack)

① IF ② OF ③ Conditional Jump

X

X

X

[CALL...ret]

④ INTR Subprogram initialization.

Control unit: (CU) $\leftarrow$ Hardwired / Microprogrammed

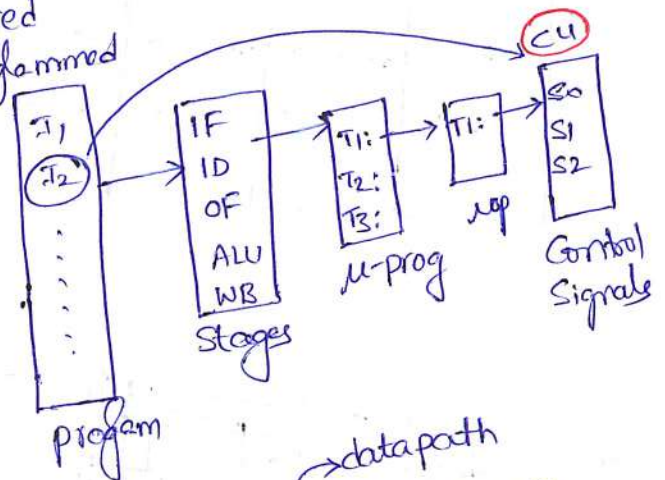
$\rightarrow$ Control-signal-sequence

$\rightarrow$ # Control signals

instructions

Inst $\rightarrow$ μ ops

μ op $\rightarrow$ control signals



(eg) PC $\xrightarrow{LB}$ MAR (μ op)

S₀: copy value from PC to LB

S₁: copy from LB to MAR

(Eg)

$$CS = \{S_0, S_1, \dots, S_7\}$$

$$\text{Inst-set} = \left\{ \begin{array}{ccc} I_1 & I_2 & I_3 \\ \text{ADD} & \text{SUB} & \text{MUL} \\ 00 & 01 & 10 \end{array} \right\}$$

Ph: 844-844-0102

CU

Mop	I_1	I_2	I_3
T_1	S_0, S_3	S_1, S_7	S_6
T_2	S_1, S_4	S_0, S_5	S_1, S_2
T_3	S_0, S_6	S_1, S_2, S_3	S_6, S_7

Control Signal-DB

$$I_1 \rightarrow T_1: \text{[Hatched Box]}$$

 $T_2:$ $T_3:$

Designing CU

Hardwired Approach:

- DLX using sum-of-products expressions
- fast & real-time applications
- Inflexible
- RISC

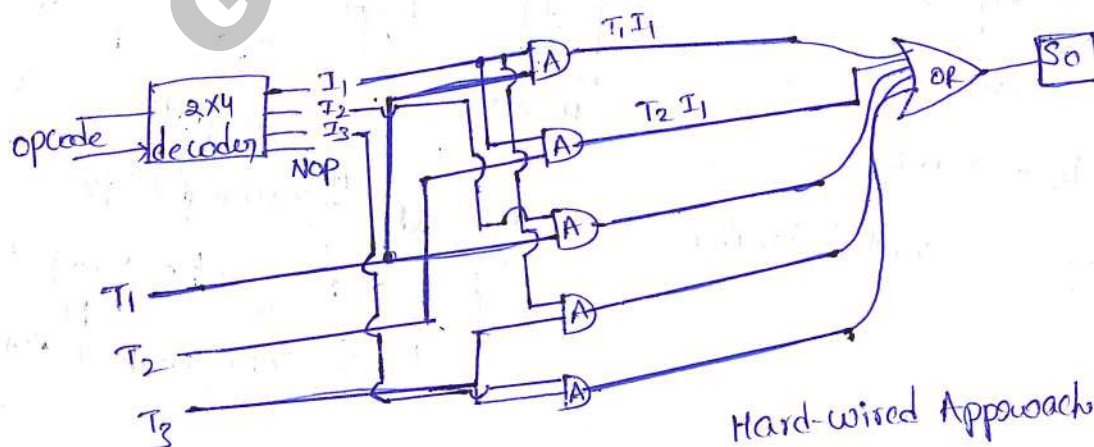
OR AND

Mop	I_1	I_2	I_3
T_1	S_0, S_2, S_3	S_1, S_0, S_3	S_2, S_3, S_1
T_2	S_1, S_0, S_2	S_2, S_1, S_3	S_2, S_1
T_3	S_0, S_2, S_3	S_2, S_1	S_0, S_2, S_3

$$S_0 = T_1 (I_1 + I_2) + T_2 (I_1) + T_3 (I_1 + I_3)$$

$$S_1 = \dots = T_1 I_1 + T_1 I_2 + T_2 I_1 + T_3 I_1 + T_3 I_3$$

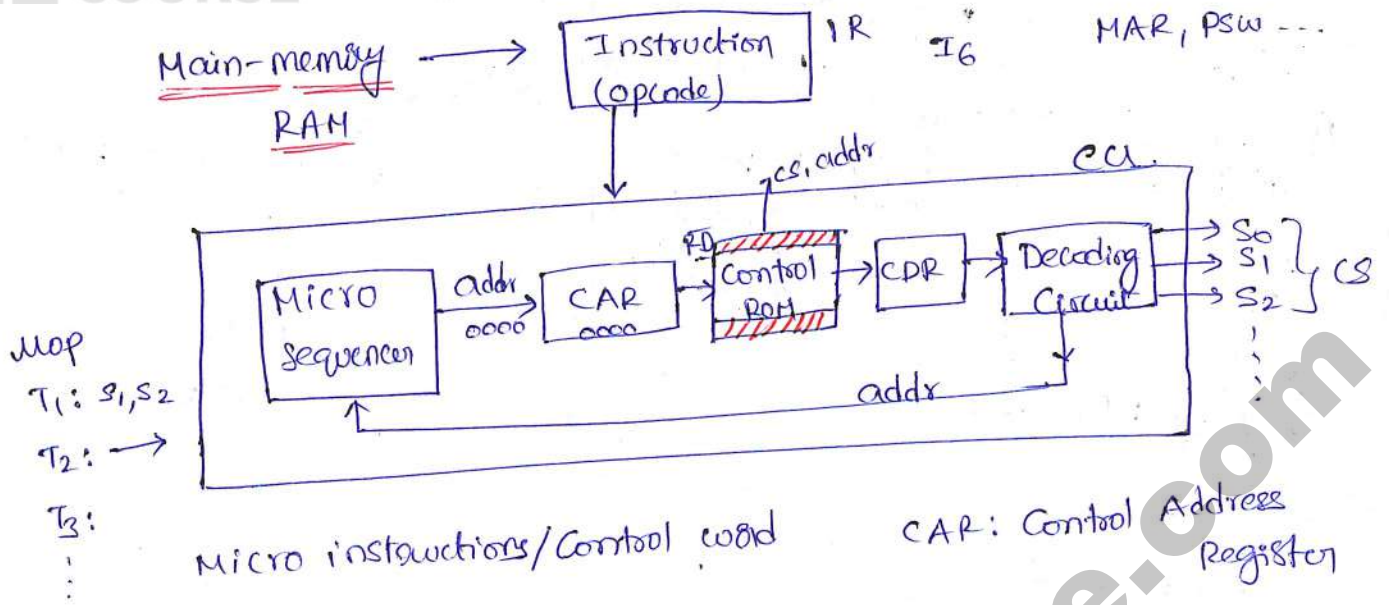
SOP



Hard-wired Approach

Micro-programmed Control unit:

u-op, u-instruction
Ph: 844-844-0102

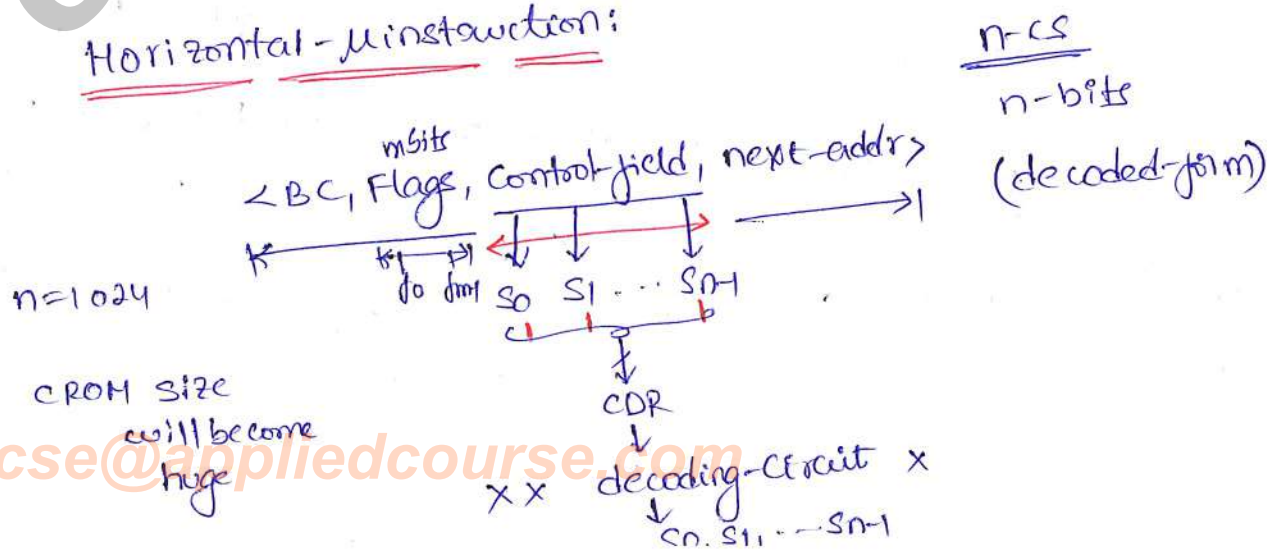


Control word / μ -instruction format:

(U, Z, N) Branch Condition	carry, sign, Z Flags	Control field $S_0, S_1, S_2, \dots, S_{n-1}$	next-control mem-addr	Control ROM
0000 U	101	1 0 1 ... 1	0006	
0001 N	010	0 1 0 0 ... 1	0002	
0002 Z	101	1 0 1 1 ... 1	0010	

Control-word / Instruction

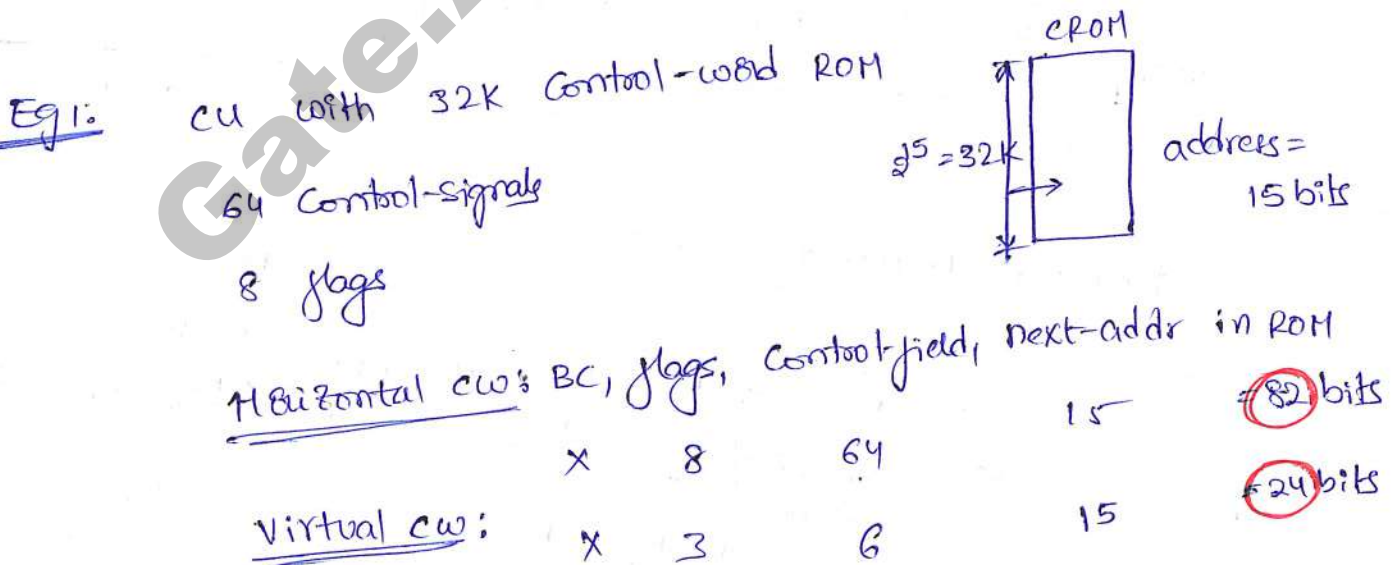
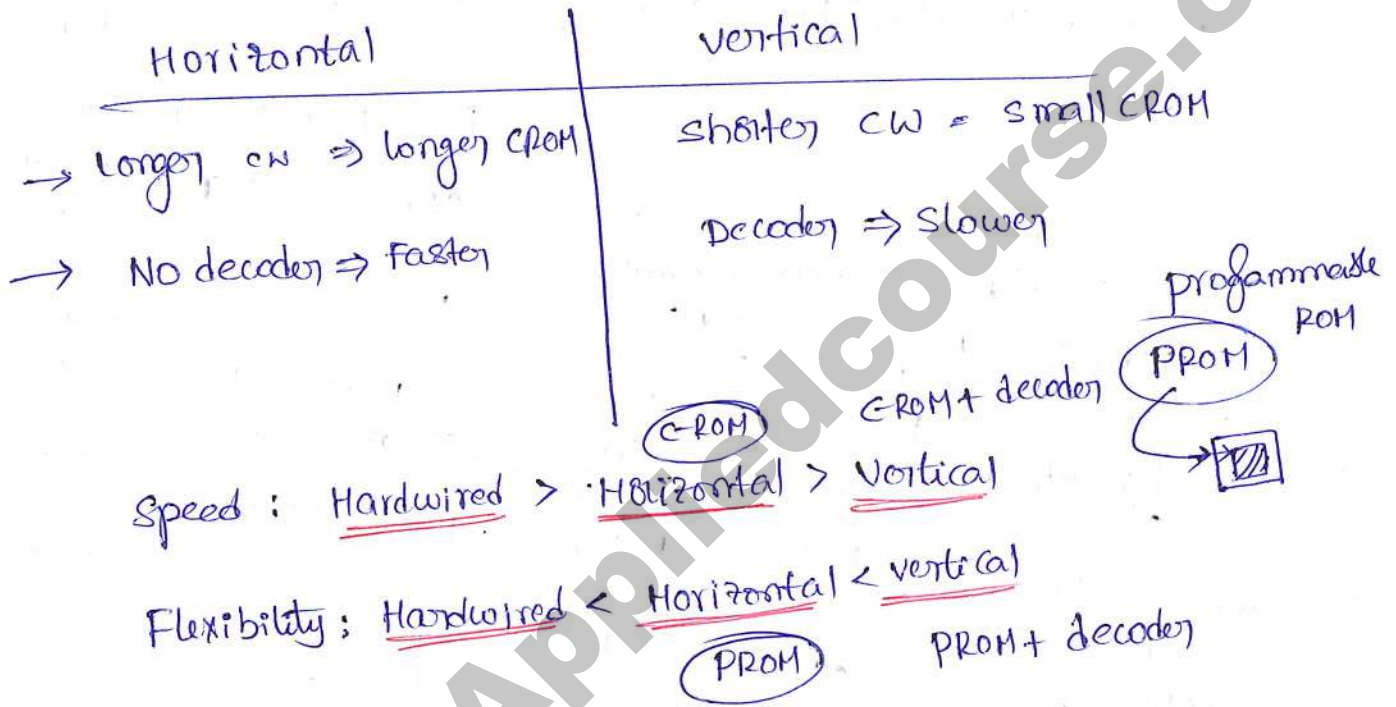
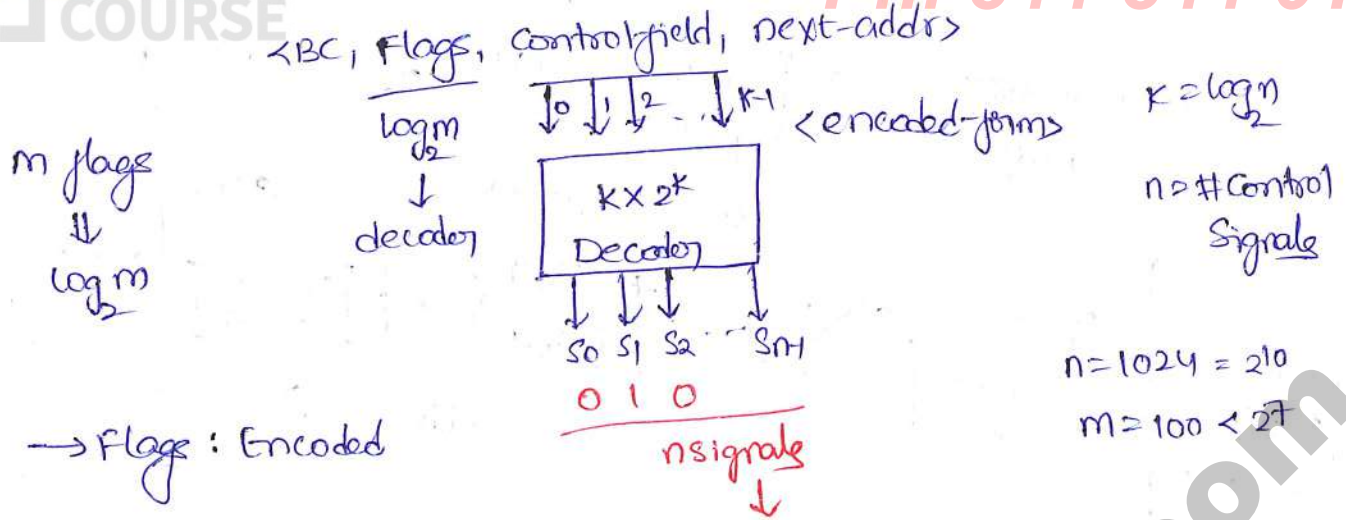
Horizontal- μ instruction:



⑥

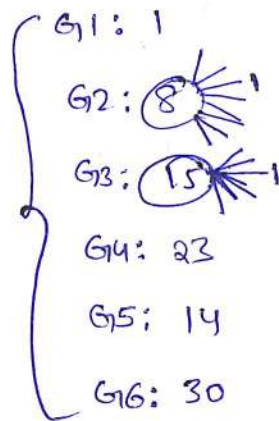
Vertical - Instruction:

Ph: 844-844-0102



More Solved problems:

Ph: 844-844-0102

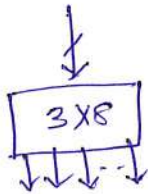
Q1) 6-groups of mutually exclusive CS.

CF
 a) # bits in Control field (Horizontal)

$$1 + 8 + 15 + 23 + 14 + 30 = 91 \text{ bits.}$$

b) vertical

c) bits-saved $91 - 22 = 69$



Q Two groups of CS: G1 & G2

256
 (one @ a time)
 ↓ vertical
 # bits in Control = 8 + 6 = 146
 Field

⑥ (Multiple-Signals)
 ↓ @ time
 horizontal

Q 256 instructions $\Rightarrow 256 \times 16 \Rightarrow 2^8 \times 2^4 = 2^{12}$

$T_1, T_2, \dots, T_{16}$ 16 Cycles/Instruction $\rightarrow \mu\text{op}$

Horizontal - w
 240 - CS
 16 - flags
 16 - BC
 Size of CAR & CDR = ?
 12 bits 284 bits.

BC 16
 Flags 16
 CS 240
 addr 12

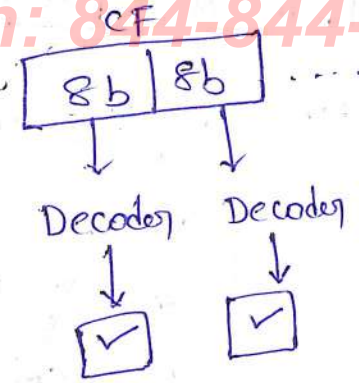
$2^{12} W$

$16 + 16 + 240 + 12$
 $= 284$

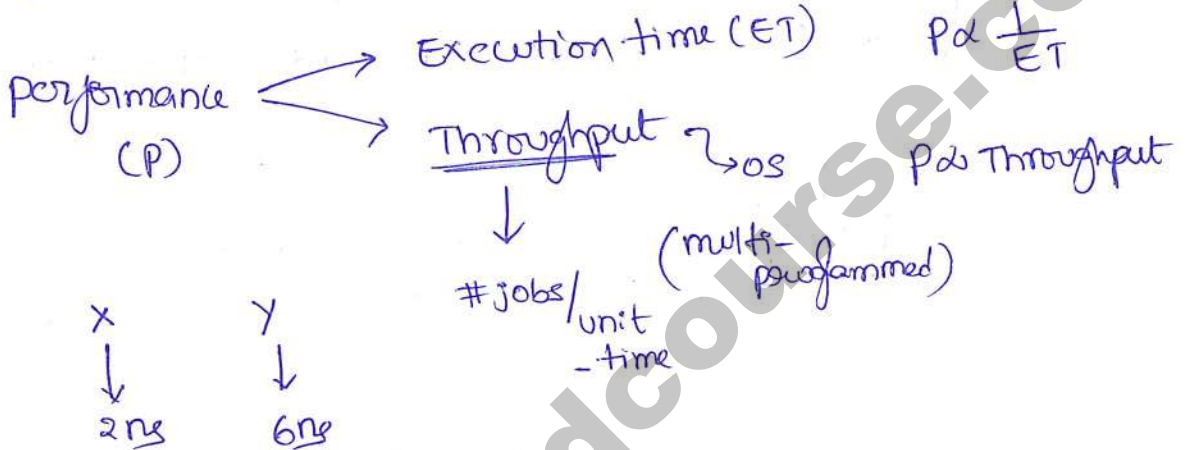
$$\begin{array}{r} 32 \\ 12 \\ \hline 240 \\ 284 \end{array}$$

default: Vertical

256-CS
Each - map has 2 CS-active
CF 8 bits #bits needed for CF: ?
16 bit



Performance Evaluation: Pipelining



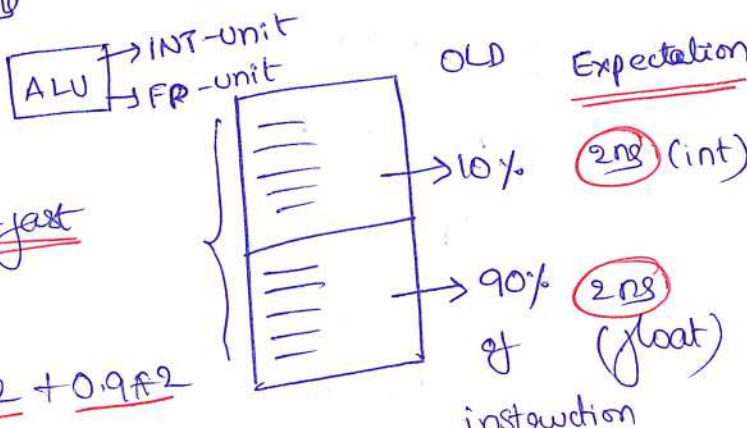
Speed-up-factor (S):

$$S = \frac{P_x}{P_y} = \frac{1/ET_x}{1/ET_y} = \frac{ET_y}{ET_x} = \frac{6}{2} = 3$$

How fast is X as Compared to Y?
2ns 6ns

Arndt's Law:

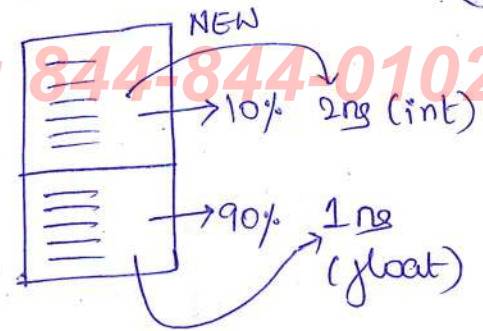
↳ Common-case-fast



$$E(ET_{old}) = 0.1 \times 2 + 0.9 \times 2 = 2ns$$

$$E(ET_{new}) = 0.1 * 2 + 0.9 * 1$$

$$= 1.0 \text{ ns}$$



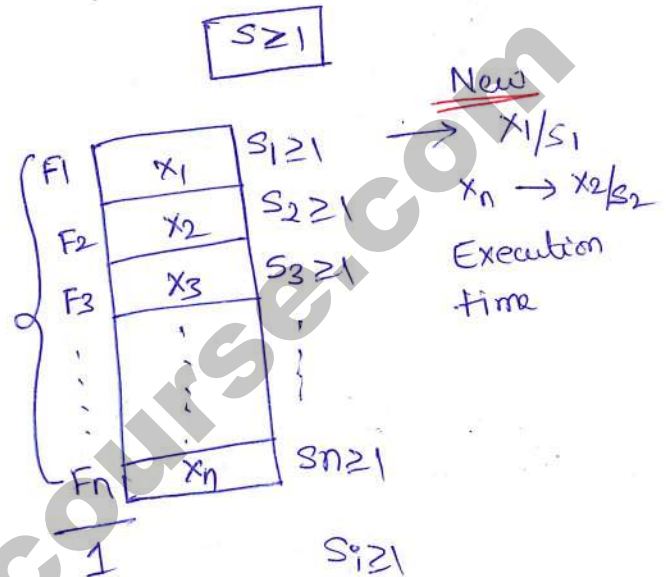
$$S_{overall} = \frac{2}{1.1}$$

Generalization:

$$E(ET_{old}) = \sum_{i=1}^n X_i F_i$$

$$E(ET_{new}) = \sum_{i=1}^n F_i (X_i / S_i)$$

$$S_{overall} = \frac{ET_{old}}{ET_{new}}$$

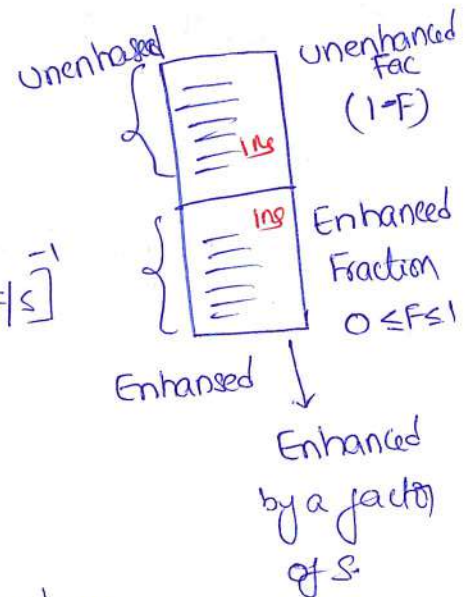


Special-case:

$$S_{overall} = \frac{E(ET_{old})}{E(ET_{new})} = \frac{1}{(1-F) + F/S}$$

$$= [(1-F) + F/S]^{-1}$$

$$(1-F) * 1 + \frac{1}{S} * F$$



Assumption:
Each instruction takes same time

Q up for Scientific applications

Ph: 844-844-0102

Integer and floating point instructions.

FP-unit is enhanced & It runs 2x faster

10% of instructions in program are FP-operations

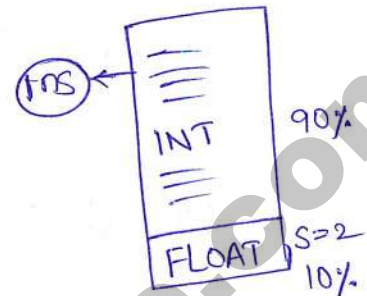
performance-gain = Overall = ?

$$ET_{old} = 1ns$$

$$ET_{new} = 0.9 + \frac{0.1}{2}$$

$$Overall = \frac{1}{0.9 + 0.05}$$

$$= 1.05$$



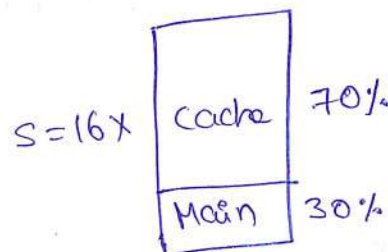
Q Cache is 16x faster than main-memory

cpu-refers to cache 70% of times

performance-gain = Overall = ?

$$= \frac{1}{0.3 \times 1 + 0.7 \times \frac{1}{16}}$$

$$= 2.9$$



Q

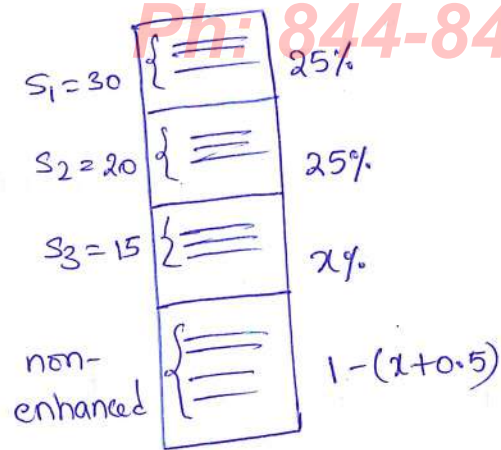
3-enhancements:

Soverall = 10

Find $x\%$ $E_{old} = 1ns$

$$E_{new} = 0.25 \times \frac{1}{30} +$$

$$\frac{0.25}{20} + \frac{x}{15} + (0.5-x)$$



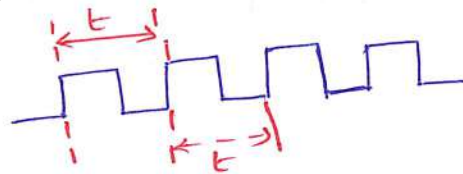
Soverall = 10

$$\Rightarrow \frac{0.25}{30} + \frac{0.25}{20} + \frac{x}{15} + (0.5-x) = 10$$

$$\Rightarrow \boxed{x = 0.45}$$

Calculation of cpu-time:

→ processor-clock

→ Cycle-Time (t) = $\frac{1}{f}$ → Constant frequency (f) = $\frac{1}{t}$ 

cpu

 3.26 Hz
1 cycle → t sec $\frac{1}{t}$ Hz ← 1 seccpu-time: $IC \times CPI \times t$

$$\# \text{ instr} \times \frac{\text{cycles}}{\text{instr}} \times \frac{\text{Time}}{\text{cycles}}$$

IC: Instruction-Count

CPI: Cycles-per-instruction

cpu-time = $\sum_{i=1}^k I C_i * CPI_i * t$

Q $f = 3.2 \text{ GHz}$ μp $t = \frac{1}{3.2 \times 10^9} = 0.31 \text{ ns}$

Instr-Type	IC	CPI
Load	200	12
Store	300	10
ARITH	200	8
Logical	100	6
Shift	150	4
Branch	50	2
	1000	

a Avg-instr-ET = $8.3 \times 0.31 \text{ ns} = 2.57 \text{ ns}$
 $0.2 \times 12 + 0.3 \times 10 + 0.2 \times 8 + 0.1 \times 6 + 0.15 \times 4 + 0.05 \times 2$
 avg cycles per instruction = 8.3 cycle/instr

b MIPS-rate: avg 1 instr $\rightarrow 2.57 \text{ ns}$
 $\leftarrow 15$

MIPS
 Millions of Instructions per Second
 $0.3861 \times 10^9 \leq \frac{1}{2.57 \times 10^{-9}}$
 $\Rightarrow 386.1 \times 10^6$
 $\Rightarrow 386 \text{ MIPS}$

$$\Rightarrow 1000 \times 2.57 \text{ ns}$$

$$\Rightarrow \boxed{2.57 \times 10^6 \text{ s}}$$

Q

cpu-1

$$t = 4 \text{ ns (Cycle Time)}$$

Load - 8 Cycles }
STORE - 8 } - 40%

ALU - 6 $\rightarrow$ 40%

Branch - 4 $\rightarrow$ 20%

$$\begin{aligned} \text{avg. instr ETold} &= (0.4 \times 8 + 0.4 \times 6 + 0.2 \times 4) \times 4 \\ \text{Speed up} &=? \\ &= \underline{25.6 \text{ ns}} \end{aligned}$$

$$\frac{25.6}{4.8} = \boxed{5.33}$$

cpu-2

$$t = 4.8 \text{ ns}$$

$$\boxed{CPI = 1}$$

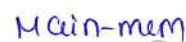
$$\begin{aligned} \text{avg - Instr ET}_{\text{new}} &= [0.4 \times 1 + 0.4 \times 1 + 0.2 \times 1] \times 4.8 \\ &= \underline{4.8 \text{ ns}} \end{aligned}$$

Flynn's cpu classification :- Performance $\xrightarrow{\text{ET}}$ RISC VS CISC
Throughput

Concurrency: 2 or more events @ same-time.

Event: program / sub-program / instruction / Stage of an instruction.

SISD, SIMD, MISD, MIMD

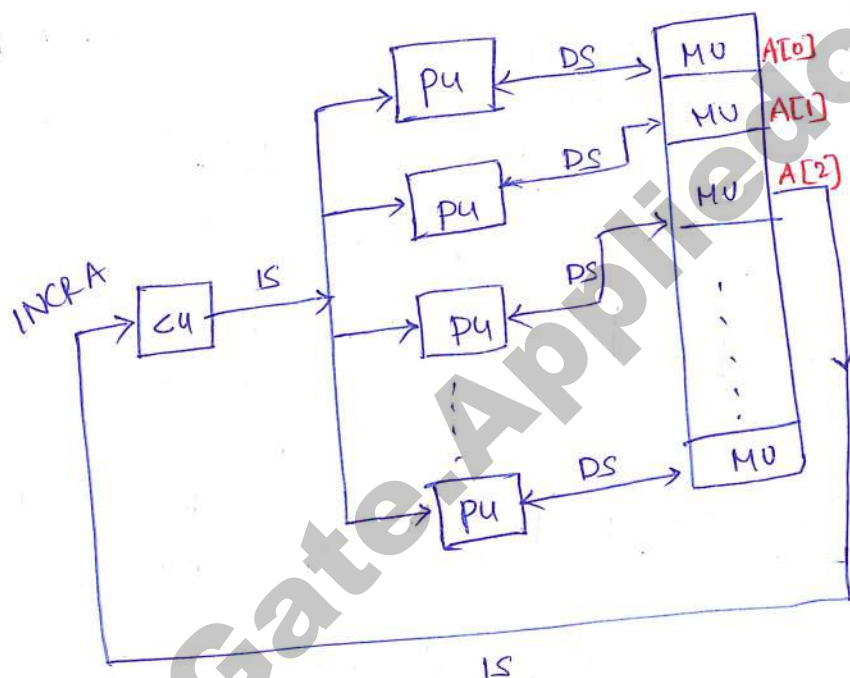

$$x = x + 5;$$

Uni-processor system



2. MU: Memory unit

② SIMD: Single Instruction multiple data.



Eg: Add 1 to all elements
of an array



→ vector-processing

→ GPU (Graphical processing)

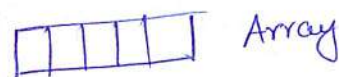
AT&ML
(DL)

Ardeno 90's

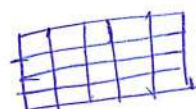
Nvidia

AMD

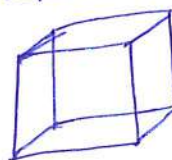
→ Scientific Applications



Array



Matrix



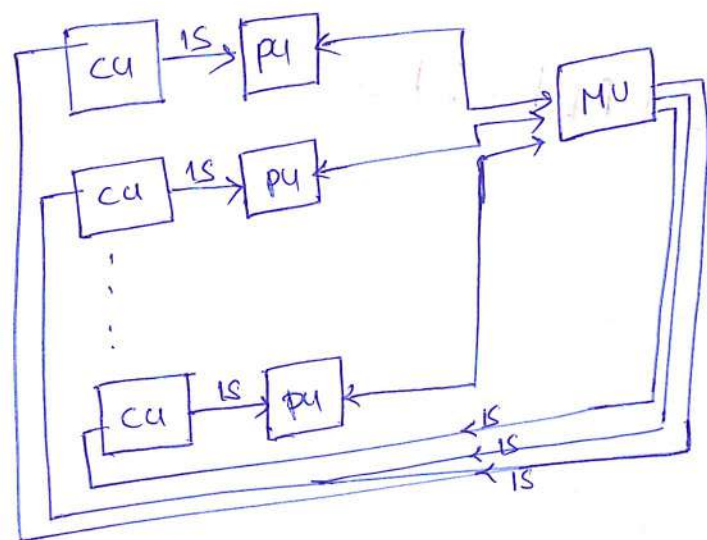
Tension
(Tension flow)

③ MISD: Multiple Instructions Single data.

Ph: 844-844-0102

Not very popular.

uni processor
↑
SISD
SIMD
MISD
MIMD
↓
Multi processor



x : Variable

$$y = x^2$$

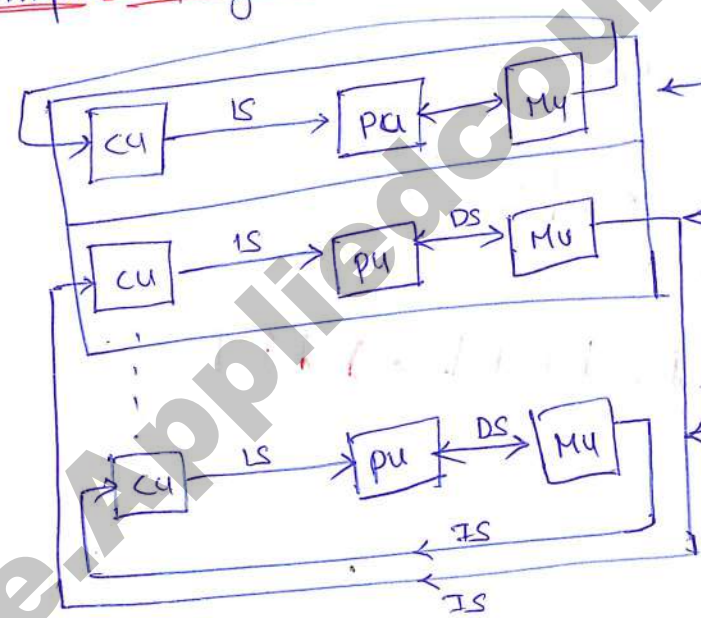
$$z = \sqrt{x}$$

$$k = x^{1/2}$$

$$m = x/\sqrt{2}$$

④ MIMD: Multi-processor - System

General purpose

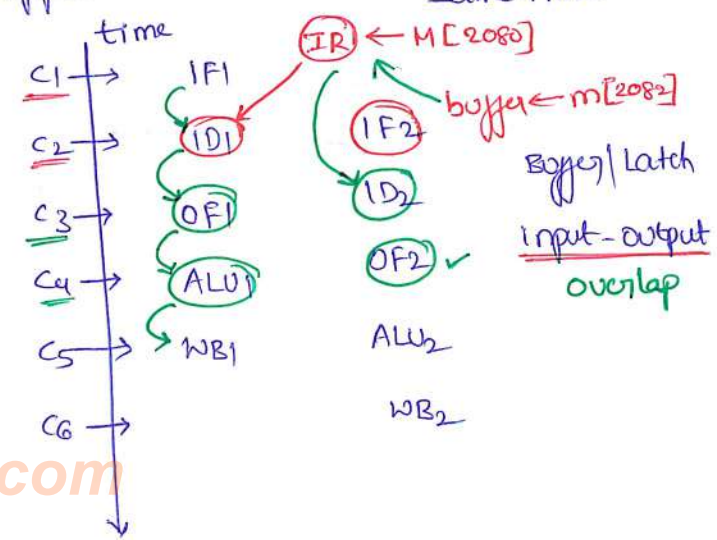


i7, i5,
Qualcomm
Snapdragon

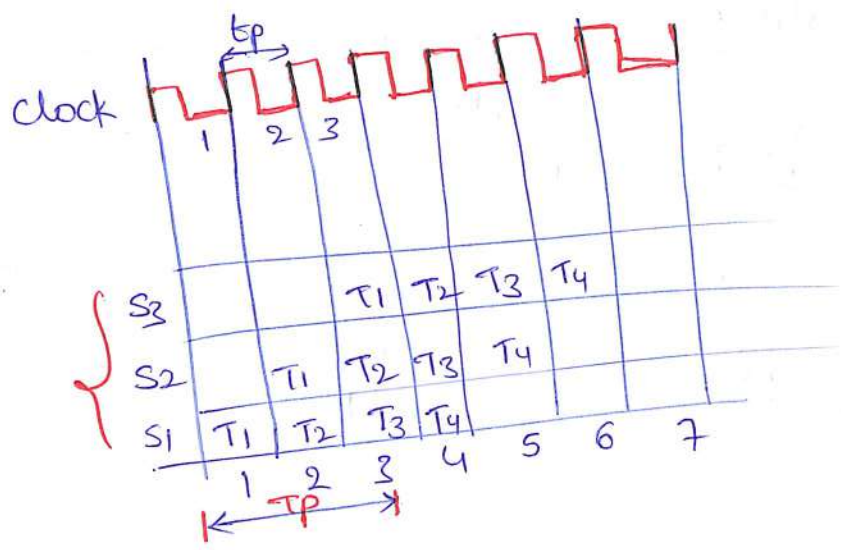
Pipelining:

A Concurrency Approach → 2 or more Events @ same time

I_1 : IF₁ S₁ ID₁ S₂ OF₁ S₃ ALU₁ S₄ WB₁ S₅
[2080]
 I_2 : IF₂ ID₂ [2082] OF₂ ALU₂ WB₂



Pipelined - Overlapped Execution:

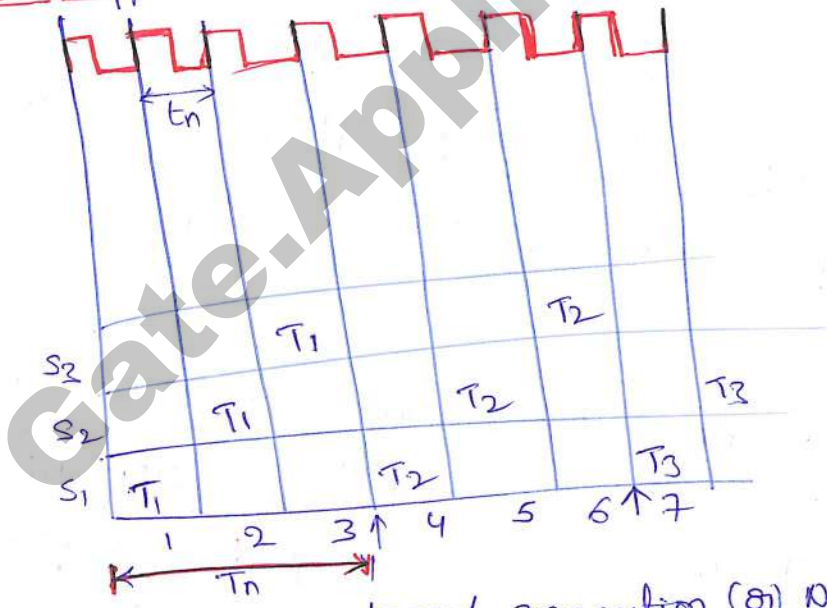


T_i : Task_i / instruction_i

pipeline-depth = 3 = #stages.

$T_p = 3 \times t_p \Rightarrow K t_p$

Non-overlapped Execution:



Non-overlapped execution (or) non-pipelined

Performance of a pipeline:

$$\# \text{ segments} = k$$

$$\text{clock-cycle-time} = t_p, t_n$$

$$\# \text{ Tasks } \text{or} \text{ instructions} = n$$

→ Each segment takes one-cycle

Ph: 844-844-0102

$$ET_{\text{pipe}} = \underline{(k + (n-1))t_p}$$

$$ET_{\text{non-pipe}} = \underline{n(t_n \times k)}$$

$$S = \frac{n \cdot t_n \cdot k}{(k + (n-1))t_p}$$

$$S = \boxed{\frac{n \cdot T_n}{T_p + (n-1)t_p}}$$

Case: $n \rightarrow \infty, S = \frac{n T_n}{n t_p - t_p + k t_p}$

$$\approx \frac{n T_n}{n t_p + (k-1)t_p}$$

k -small $\leq n$ & very large

$$\Rightarrow \boxed{S = T_n / t_p = \frac{k t_n}{t_p}} \approx \underline{k, \text{ if } (t_n \approx t_p)}$$

Throughput of a pipeline

$$= \frac{\# \text{ tasks - completed}}{\text{time - taken}}$$

$$n \text{ tasks} \rightarrow (k + n-1) t_p \text{ sec}$$

$$\boxed{\frac{n}{(k+n-1)t_p} \leftarrow 1 \text{ sec}}$$

Efficiency of a pipeline: $\frac{S}{k} = \eta$

Q Instruction-pipeline with $S=12$ and operating at 70% efficiency # stages = ?

$$\eta = 70\%$$

$$= \frac{S}{K}$$

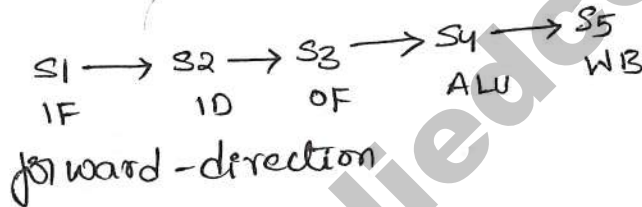
$$= \frac{12}{K}$$

$$= 0.7K = 12$$

$$\Rightarrow K = \frac{12}{0.7} = 17.1 \Rightarrow \boxed{18}$$

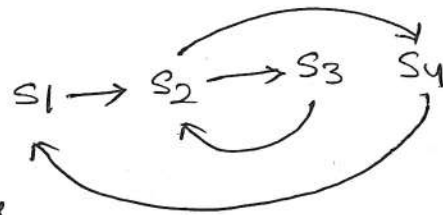
Types of Pipelines:

① Instruction/Linear pipeline



② Non-Linear pipeline

Tasks: Segments/ Stages



③ Synchronous:

pipe-line operation is Controlled by the clock-Signal

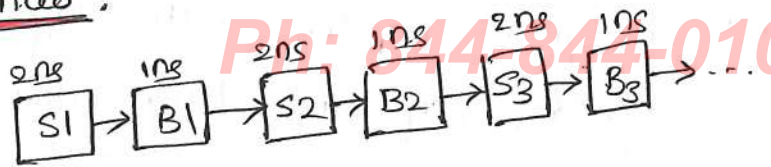
↳ All linear pipelines are Synchronous

④ Asynchronous pipeline:

operations are controlled based on Handshaking Signals.

↳ All Non-linear pipelines are asynchronous.

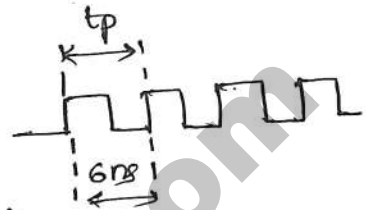
⑤ Uniform-delay / balanced:



$$t_p = (\text{Stage-delay} + \text{Buffer-delay} + \text{overhead})$$

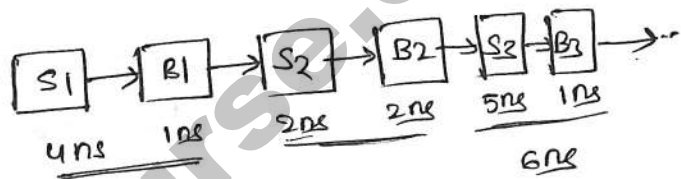
$$t_{1s}$$

⑥ Non-uniform delay / Imbalanced:



$$t_p = \max (SD_i + BD_i + O_i)$$

clock
time



⑦ Pipeline-A

$$K_1 = 8$$

$$\text{uniform-delay} = 2ns$$

Time saved when 100-tasks are
instead of B.

$$ETA = (8 + 99)2 = 214ns$$

$$= (K + n - 1)t_p$$

Pipeline-B

$$K_2 = 6$$

Stage-wise
delays } : 2, 6, 4, 1, 3,
pipelined through A

$$(6 + 99) * 6 = 630ns$$

$$\Rightarrow 630 - 214 = \boxed{416ns}$$

(38)

Q2) 4-segment pipeline with delays: 20 ns, 30 ns, 40 ns & 15 ns
 speedup & η when large-number of tasks are executed.

$$S = \frac{T_n}{t_p} = \frac{\text{Total time for non-pipelined execution}}{\text{clocktime for pipelined execution}}$$

$$= \frac{105}{40} = 2.625$$

$$\eta = \frac{S}{K} = \frac{2.625}{4} \times 100$$

$$= \boxed{65.6\%}$$

Q3) 4-stage pipeline with stage-delays: 30 ns, 40 ns, 20 ns & 10 ns. Interface-register between the stages has a delay of 5 ns. performance-gain of the pipeline?

let $n = \text{Large}$.

$$S = \frac{T_n}{t_p} = \frac{100}{45}$$

$$= 2.22$$

$$\eta = \frac{S}{K} = \frac{2.22}{K=4} \times 100$$

$$= \boxed{55.5\%}$$

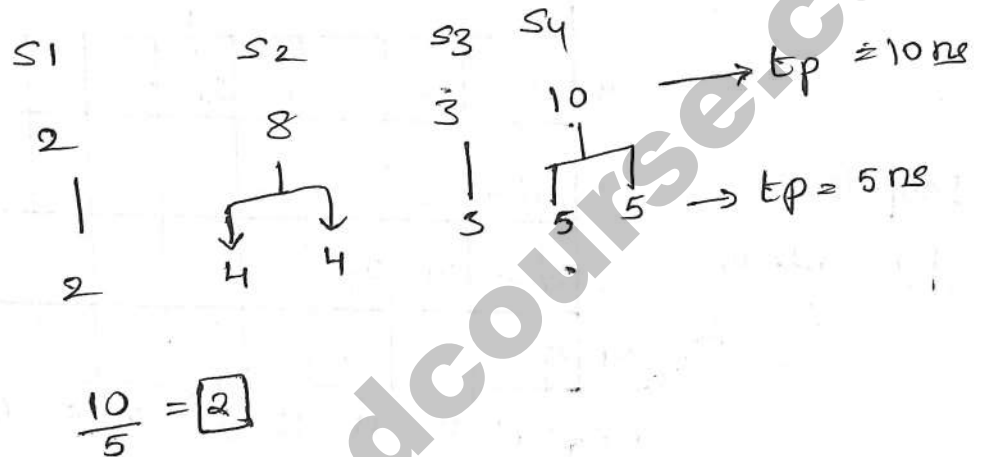
$$\text{MAX}(30, 40, 20, 10)$$

$$\text{MAX}(35, 45, 25, 15)$$

Let $B = \{0, 1, 2, \dots, 5\}$

old:

Alt | New:

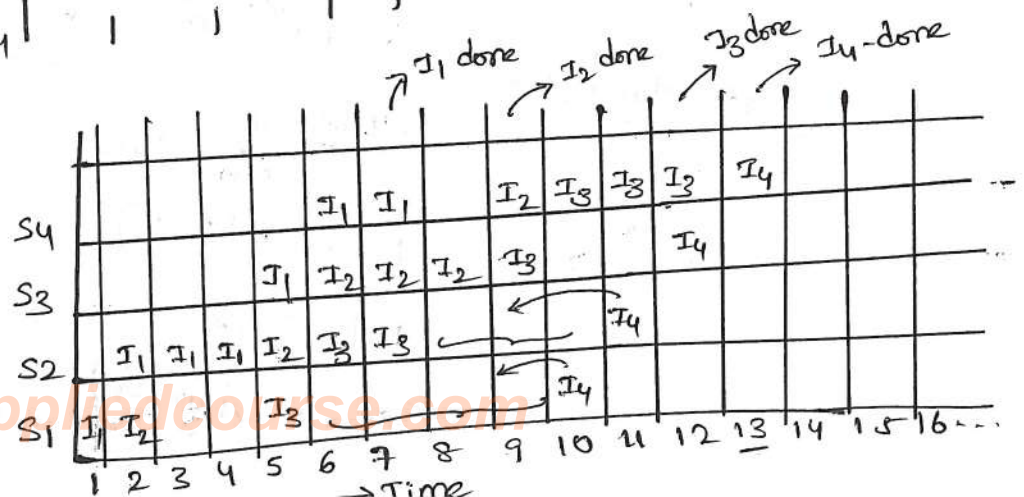


Q5) { 4-stage pipeline where different instructions spent different amount of time @ each-stage

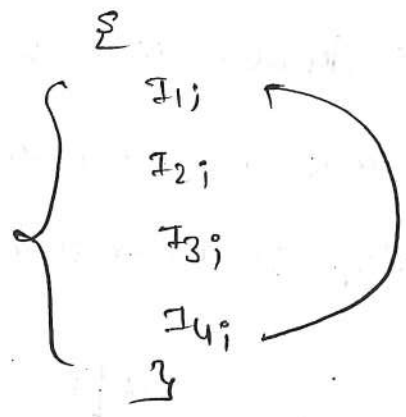
real world		S_1	S_2	S_3	S_4
7	I_1	1	3	1	2
6	I_2	1	1	3	1
7	I_3	1	2	1	3
4	I_4	1	1		1
<u>24</u>					

① # cycles to complete the above program.

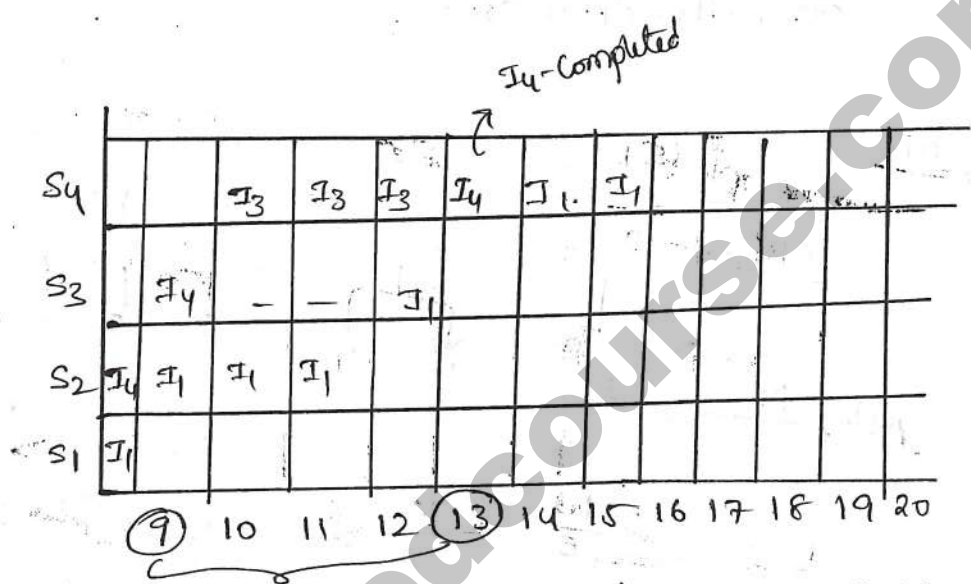
⇒ 13 Cycles



b) $\text{for } (i=1; i \leq 1000; i++)$



Loop-level parallelism

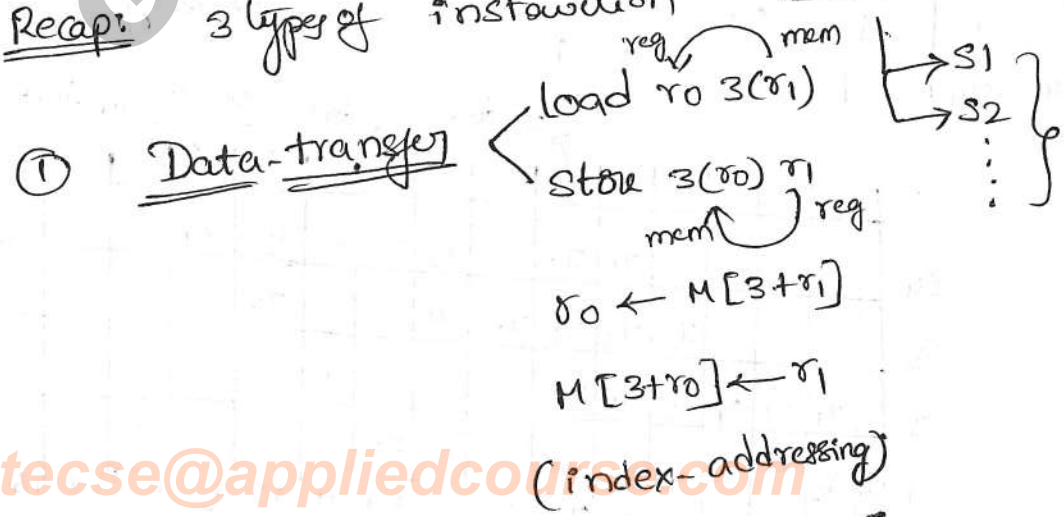


Cycles - overlapped between 2-iterations = $13 - 9 + 1 = 5$

$i=1$ I₄
 $i=2$ I₁

RISC Pipelining: 5-Stage Instruction-pipeline

Recap: 3 types of instruction smaller Instruction set



② Data Manipulation:

Ph: 844-844-0102

Add r_0, r_1, r_2

$$r_0 \leftarrow \overset{ALU}{r_1 + r_2}$$

all operands in registers
(RISC)

③ Transfer of Control:

Unconditional
Jump 2080

$$PC \leftarrow 2080$$

Conditional

JNZ $r_0, 2080$

→ Dec r_0

Compare $r_0 == 0$

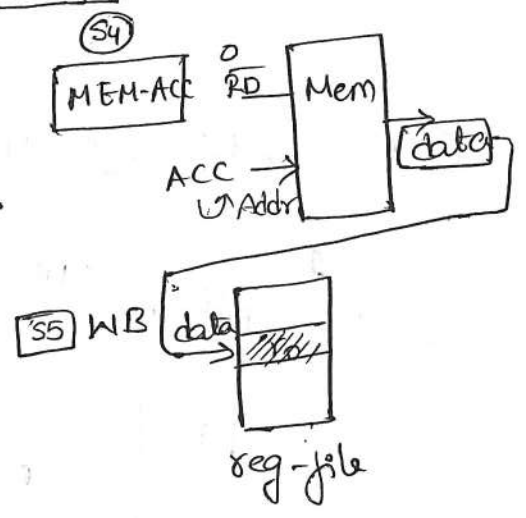
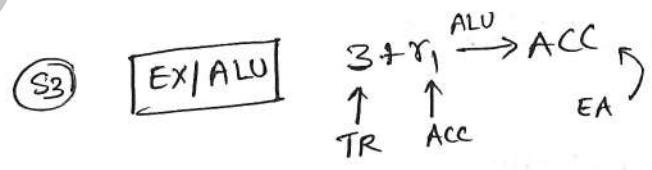
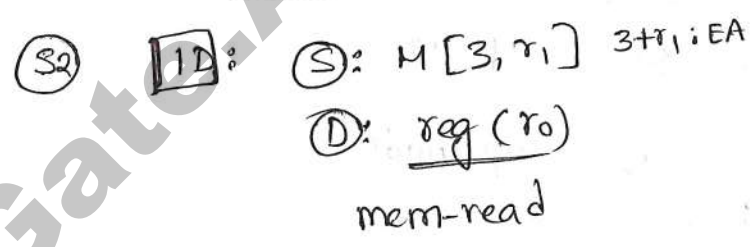
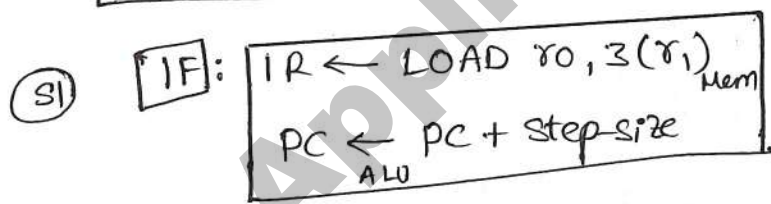
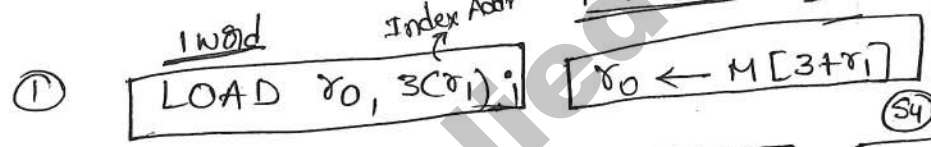
T F

$$PC \leftarrow 2080$$

Complex Conditional
Transfer of Control

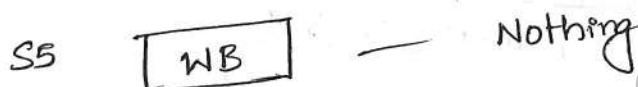
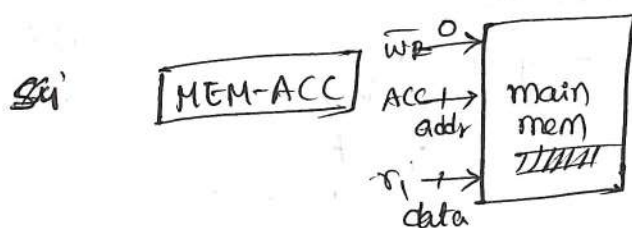
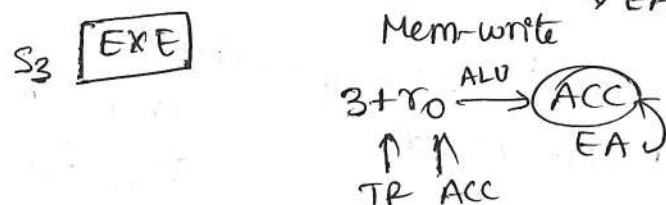
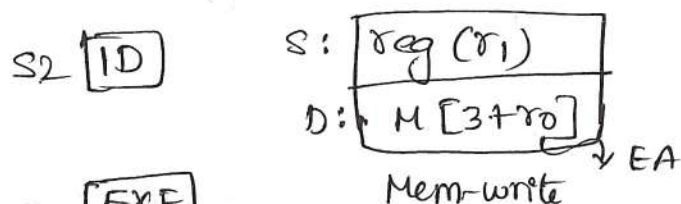
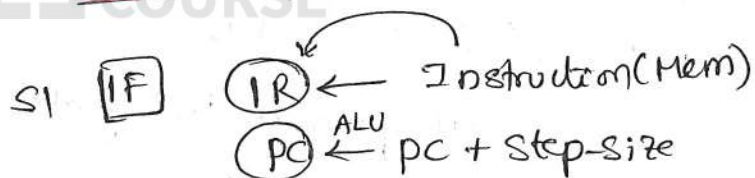
JZ
JNZ

5 stages → IF, ID, ALU-Execute, Mem-Access, WB

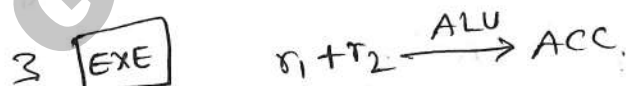
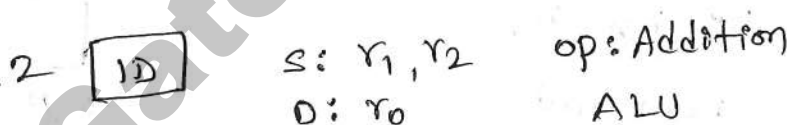


② STORE $3(r_0), r_1$; $M[3+r_0] \leftarrow r_1$

Ph: 844-844-0102



③ ADD r_0, r_1, r_2



④ Jump 2080

① IF $IR \leftarrow \text{instruction}$
 $PC \leftarrow PC + \text{step-size}$

② ID $\boxed{\text{opcode}}$
 $PC \leftarrow 2080$

③ Exe - X

④ Mem-Acc - X

⑤ $\boxed{WB} \leftarrow PC \xrightarrow{\overline{WR}} 2080$

⑤ DJNZ R0, 2080

① IF $IR \leftarrow \text{instruction}$
 $PC \leftarrow PC + \text{step-size}$

② ID ALU-operation on R0

③ Exe $R0 \leftarrow R0 - 1$
 if $R0 \neq 0$
 $ACC = 1$
 else $ACC = 0$

④ Mem-Acc

⑤ $\boxed{WB}$ $PC \leftarrow 2080$ if $ACC = 1$
 — if $ACC = 0$

iphone & ipad (A13)
 (ARM)

$ACC \xrightarrow{\overline{WR}} \boxed{PC}$
 2080

Pipeline Dependencies:

→ Structural → Fix / Solution
→ Data
→ Control

→ Major problems / issues

Structural Dependency:



→ Resource Conflict

→ Waiting / Stall

	C1	C2	C3	C4	C5
I ₁	IF	ID	ALU	WB	
I ₂		IF	ID	ALU	
I ₃			IF	ID	
I ₄				—	IF

wait / stall

	S ₁	S ₂	S ₃	S ₄
	C1	C2	C3	C4
I ₁	IF	ID	ALU	<u>WB</u>
I ₂		IF	ID	ALU
I ₃			IF	ID
I ₄				<u>IF</u>

→ Mem-WB

→ Mem-RD

WB RD

IF: Memory-RD

WB: Memory-WR



Eg 2:

	C1	C2	C3	C4	C5	C6	C7
I ₁	Mem	ID	ALU	Mem	WB _R		
I ₂		Mem	ID	ALU	Mem	WB _R	
I ₃			Mem	ID	ALU	Mem	WB _R
I ₄				X	X	X	Mem

3-cycle-stall

5-Stage RISC

WB: reg-write

Solution to structural-dep: Renaming

Ph: 844-844-0102

→ Code-Memory (CM)

→ Data-Memory (DM)

2-independent module

RD → CM Instruction

WR → DM Data/op

	C ₁	C ₂	C ₃	C ₄	C ₅	C ₆	C ₇	C ₈
I ₁	CM	ID	ALU	DM	WB			
I ₂		CM	ID	ALU	DM	WB		
I ₃			CM	ID	ALU	DM	WB	
I ₄				CP	ID	ALU	DM	WB

Zero-Stall

② Data Dependency:

I₁: Add r₀, r₁, r₂
 I₂: Sub r₃, r₀, r₄

(r₀)old
 $r_{0new} = r_1 + r_2$

RISC

	C ₁	C ₂	C ₃	C ₄
I ₁	IF	ID	EXE	WB
I ₂		IF	ID	

src₁: r₀
 src₂: r₄
 dest: r₃

ID-Stage:

	op	DEST	Indep src ₁	Indep src ₂	dep src ₁	dep src ₂
I ₁	ADD	r ₀	r ₁	r ₂	—	—
I ₂	SUB	r ₃	—	r ₄	r ₀ (I ₁)	—

TOMASULO
- Algorithm

	C ₁	C ₂	C ₃	C ₄	C ₅
I ₁	IF	ID	EXE	WB	
I ₂		IF	—	—	ID

2-Stall

Eg: 5-stage RISC

I₁: Add r₀, r₁, r₂
 I₂: Sub r₃, r₀, r₄

	C1	C2	C3	C4	C5	C6
I ₁	IF	ID	EX	MA	WB	
I ₂		IF	ID	-	-	EX

2-stall

Buffer/Latch

Solution:
(operand forwarding)
 ↑ default in RISC

	C1	C2	C3	C4	C5	C6
I ₁	IF	ID	EX	MA	WB	
I ₂			ID	EX	MA	WB

$I_1(r_0) = r_1 + r_2$

No stall

Eg:

I₁: Load r₀, 3(r₁)

I₂: Add r₃, r₀, r₂

I₃: Load r₄, 4(r₂)

I₄: Sub r₃, r₅, r₄

→ Load & store spend 3-cycles in MA-stage.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
I ₁	IF	ID	EX	MA	MA	MA	WB								
I ₂		IF	ID	-	-	-	EX	MA	WB						
I ₃			IF	-	-	-	ID	EX	MA	MA	MA	WB			
I ₄							IF	ID	-	-	-	EX	MA	WB	

	IF	ID	EX	MA	WB
I ₁	1	1	1	3	1
I ₂	1	1	1	1	1
I ₃	1	1	1	3	1
I ₄	1	1	1	1	1

Cycles to Complete
 I₁-I₄

14

Eg: APPLIED COURSE

I_1 : Add r_0, r_1, r_2
 I_2 : MUL r_3, r_0, r_4

I_3 : DIV r_5, r_6, r_7
 I_4 : sub r_8, r_5, r_7

→ EX takes 3 cycles for MUL & 6 cycles for DIV

→ RISC ⇒ operand forwarding

	c1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
I_1	IF	ID	EX	WB											
I_2		IF	ID	EX	EX	EX	WB	WB							
I_3			IF	ID	-	-	EX	EX	EX	EX	EX	WB			
I_4				IF	ID	-	-	-	-	EX	WB	-	EX	WB	

clock cycles to complete I_1-I_4

14

③ Control - Dependency:

✓ 1000: I_1

✓ 1001: I_2

✓ 1002: I_3 (Jump 2080)

✗ 1003: I_4

✓ 2080: BI_1

✓ 2081: BI_2

RISC

Jump 2080

IF: $IR \leftarrow$ instruction
 $PC \leftarrow PC + SS$

ID: $PC \leftarrow 2080$

	C1	C2	C3	C4	C5	C6	C7
I_1	IF	ID	EX	MA	WB		
I_2		IF	ID	EX	MA	WB	
I_3			IF	ID			
I_4				IF	ID	EX	
BI_1					IF	ID	EX...
BI_2						IF	ID...

Unwanted instruction
 ↓
 {FLUSH/ DELAY}

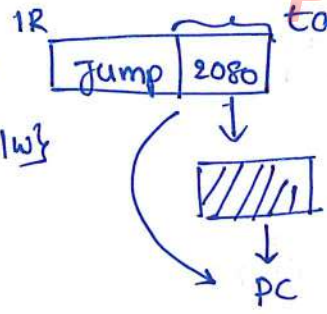
$PC \leftarrow PC + SS$

Hardware - solution:

Branch - prediction → CISC

→ Branch prediction:

→ { additional HW }
No stalls



IF Stage
RISC
fast Branch target Buffer

→ jump @ 2080

↳ Not the target-addr ⇒ Branch prediction is not possible.

Branch-penalty = # stalls due to branch operation.

Software - solutions: Delayed-Branch (Compiler)

→ Rearrange (1) add NOP (2) instruction

↓
user-program:

1000: I₁
→ 1001: I₂
→ 1002: I₃ (jump 2000)
1003: I₄
:
2000: BI₁
2001: BI₂

	C ₁	C ₂	C ₃	C ₄	C ₅	C ₆	C ₇
I ₁	IF	ID	EX	MA	WB		
I ₂		IF	ID				
I ₃			IF	ID	EX	MA	WB
BI ₁				IF	ID	EX	MA
BI ₂					IF	ID	EX...

I₂ & I₃ are independent

→ NOP-instruction

user program

1000: I₁
1001: I₂
1002: I₃ (jump 2000)
1003: NOP

X 1004: I₄
:
2000: BI₁
2001: BI₂

Instruction Scheduling (to avoid stalls)

DBMS

Ph: 844-844-0102

I₁: Add r₀, r₁, r₂

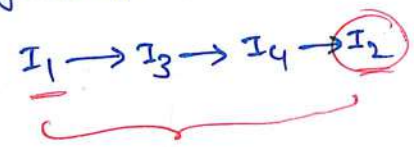
I₂: Sub r₃, r₀, r₄

I₃: MUL r₄, r₅, r₆

I₄: DIV r₃, r₇, r₈

data dep

out-of-order:



order: I₁ → I₂ → I₃ → I₄
(Stalls)

Error Anti-data dependency:

I₁: Add r₀, r₁, r₂

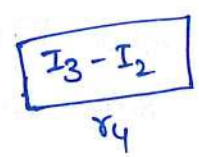
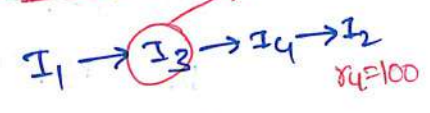
I₂: Sub r₃, r₀, r₄

I₃: MUL r₄, r₅, r₆

I₄: DIV r₃, r₇, r₈

r₄ = 10
r₅ = 10
r₆ = 10

out of order



write-before-read

Order: I₁ → I₂ → I₃ → I₄



Error: o/p Dependency:

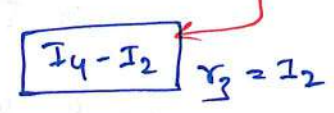
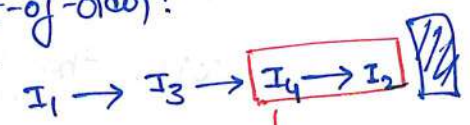
I₁: Add r₀, r₁, r₂

I₂: Sub r₃, r₀, r₄

I₃: MUL r₄, r₅, r₆

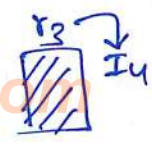
I₄: DIV r₃, r₇, r₈

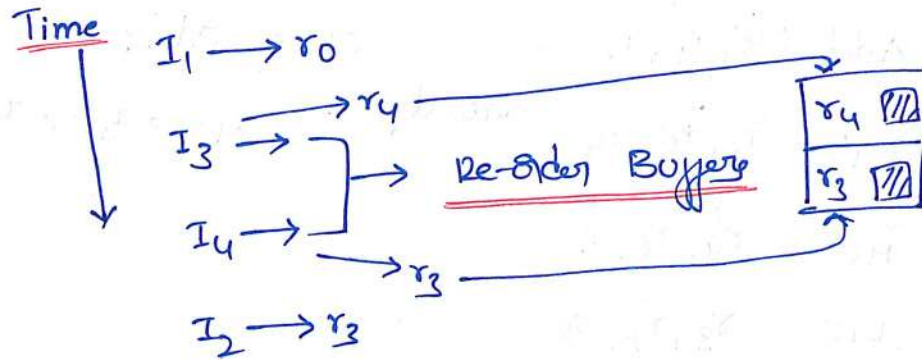
out-of-order:



write-before-write

Order: I₁ → I₂ → I₃ → I₄





Re-Order Buffers to r_4 & r_3 from I_3 & I_4 resp

More Solved problems -1

① Consider the Sequence of machine instructions

- ✓ MUL $R5, R0, R1$ 3
- ✓ DIV $R6, R2, R3$ 5
- ✓ ADD $R7, R5, R6$ 1
- ✓ SUB $R8, R7, R4$ 1

In the above Sequence $R0$ to $R8$ are general purpose Registers. In the instructions shown, the first register stores the result of the operation performed, on the Second and third Registers. This Sequence of instructions is to be executed in a pipelined instruction processor

with the following 4 stages: (1) Instruction fetch and decode (IF) (2) operand Fetch (OF) (3) perform operation (PO) and (4) write back the Result (WB). The IF, OF and WB

stages take 1 clock cycle each for any instruction.

The PO stage takes 1 Clock cycle for ADD or SUB

instruction, 3 clock cycles for MUL instruction and 5 clock cycles for DIV instruction. The pipelined processor uses "operand forwarding" from the po stage to the of stage. The number of clock cycles taken for the execution of the above sequence of instructions is _____

- (A) 11
- (B) 12
- (C) 13
- (D) 14

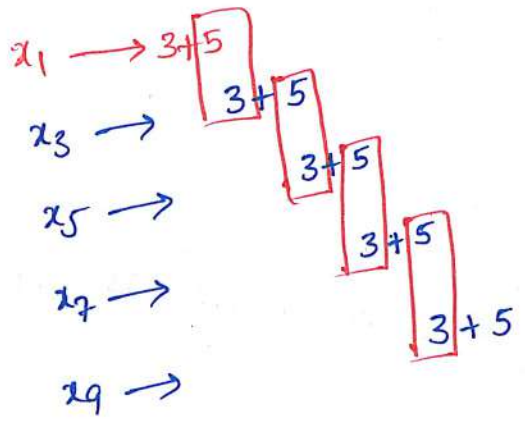
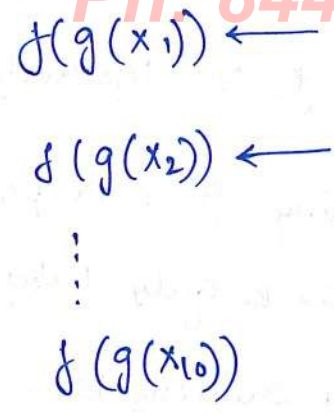
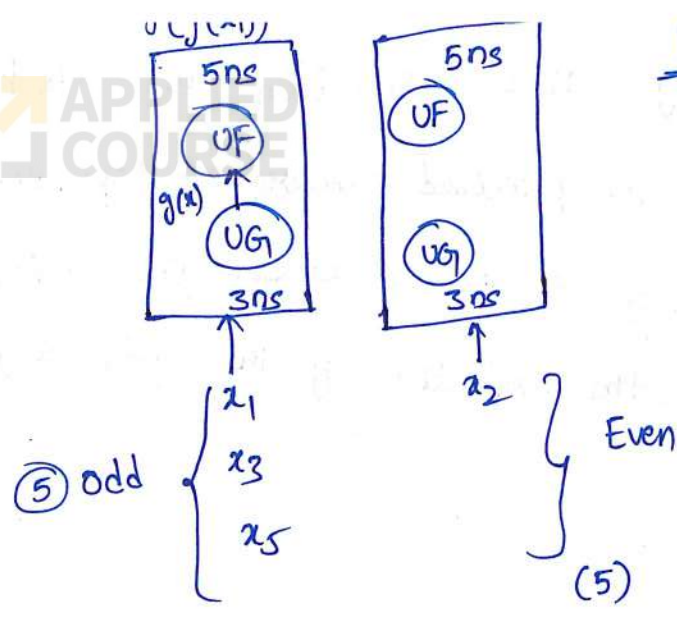
	1	2	3	4	5	6	7	8	9	10	11	12	13	14
I ₁	IF	OF	PO	PO	PO	WB								
I ₂		IF	OF			PO	PO	PO	PO	PO	WB			
I ₃			IF											
I ₄				IF				OF						

② Suppose the functions F and G can be computed in 5 and 3 nano seconds by functional units UF and UG, respectively. Given two instances of UF and two instances of UG, it is required to implement the computation $F(G(x_i))$ for $1 \leq i \leq 10$. Ignoring all other delays, the minimum time required to complete this computation is _____ nano seconds.

(14)

Pipelining

Ph: 844-844-0102



Both the Even & odd cases run parallelly.

$3+5 \times 5 = 28$

③ The instruction pipeline of a RISC processor has the following stages. Instruction Fetch (IF), Instruction Decode (ID), operand Fetch (OF), perform operation (PO) and write back (WB). The IF, ID, OF and WB stages take 1 clock cycle each for every instruction. Consider a sequence of 100 instructions. In the PO stage, 40 instructions take 3 clock cycle each, 35 instructions take 2 clock cycle each, and the remaining 25 instructions take 1 clock cycle each. Assume that there are no data Hazards and no Control Hazards. The number of clock cycle

required for Completion of execution of the Sequence of instructions is 219

Ph: 844-844-0102

IF, ID, OF, WB $\rightarrow$ 1 Cycle

$n = 100$ instructions $\rightarrow 40 \rightarrow 3$ cycles (Po)

$35 \rightarrow 2$ cycles (Po)

$25 \rightarrow 1$ cycle (Po)

$K=5$

$$(K+n-1) + (40 \times 2) + (35 \times 1) + (25 \times 0)$$

$$= 104 + 80 + 35$$

$$= \boxed{219}$$

④ Instruction Execution in a processor is divided into 5 Stages. Instruction Fetch (IF), Instruction Decode (ID), Operand Fetch (OF), Execute (EX) and Write Back (WB). These stages take 5, 4, 20, 10 and 3 nano seconds respectively. A pipelined implementation of the processor requires buffering between each pair of consecutive stages with a delay of 2ns. Two pipelined implementations of the processor are contemplated.

(i) a Naive pipeline implementation (NP) with 5 stages and

(ii) An Efficient pipeline (EP) where the OF stage is divided into stages OF1 and OF2 with execution times of 12ns and 8ns respectively.

The Speedup (Correct to two decimal places) achieved by EP over NP in Executing 20 independent instructions with no hazard is _____

Ph: 844-844-0102

NP $\downarrow$
K
5

$$t_p = 22ns$$

$$n = 20$$

$$K = 5$$

$$\Rightarrow (5 + 20 - 1) \times 22ns$$

$$\Rightarrow 24 \times 22ns$$

$$\Rightarrow \boxed{528ns}$$

EP $\downarrow$
K
6

$$t_p' = 14ns$$

$$n = 20$$

$$K = 6$$

$$(20 + 6 - 1) \times 14ns$$

$$25 \times 14$$

$$\boxed{350ns}$$

$K+n-1$
 $5, 4, 20, 10 \text{ and } 3$

5, 4, 12, 8, 3, 10, 3

$$\Rightarrow \text{Speedup} = \frac{528}{350} = 1.508 = \boxed{1.51}$$

- ⑤ The stage delays in a 4-stage pipeline are 800, 500, 400 and 300 pico seconds. The first stage (with delay 800 pico seconds) is replaced with a functionally equivalent design involving two stages with respective delays 600 and 350 pico seconds. The Throughput increase of the pipeline is _____ percent.

$4 = K$	$K = 5$
800, 500, 400, 300	600, 350, 500, 400, 300

Nano sec - 10^{-9} sec

Pico sec - 10^{-12} sec

$t_p = 800 \text{ ns}$
If n is very large

$$\frac{n-K+1}{n} \approx 1 \text{ instruction} \rightarrow \text{cycle}$$

$$\text{Thr}_1 = \frac{1}{800}$$

$$t_p = 600$$

$$\text{Thr}_2 = \frac{1}{600}$$

$$\Rightarrow \frac{\text{Thr}_2 - \text{Thr}_1}{\text{Thr}_1} \times 100$$

$$= 33.33\%$$

More Solved problems-11

- ① Consider a 3 GHz processor with a three stage pipeline and stage latencies v_1, v_2 and v_3 such that $v_1 = 3v_2, v_2 = 2v_3$. If the longest pipeline stage is split into two pipeline stages of equal latency, the new frequency is (4) GHz, ignoring delays in the pipeline registers.

- (A) 2
(B) 4
(C) 8
(D) 16

$$\left\{ \begin{array}{l} K=3 \\ v_1 = \frac{3}{4} v_2 = 2v_3 \end{array} \right.$$

$$\text{let } v_1 = 6x \text{ ns}$$

$$v_2 = \frac{4}{3} \times 6x = 8x \text{ ns}$$

$$v_3 = 3x \text{ ns}$$

$$t_p = 8x \text{ ns}$$

$$f = \frac{1}{8x} = 3 \text{ GHz} \quad \text{Given}$$

$$\frac{1}{x} = 24 \text{ GHz}$$

$$\begin{array}{cccc} 6x & 8x & 3x & \\ 6x & 4x & 4x & 3x \end{array}$$

$$t_p = 6x$$

$$f = \frac{1}{6x} = 4 \text{ GHz}$$

② Consider a non-pipelined processor with a clock rate of 2.5 GHz and average cycles per instruction of four.

The same processor is upgraded to a pipelined processor with five stages, but due to the internal pipeline delay, the clock speed is reduced to 2 GHz. Assume that there are no stalls in the pipeline. The speedup achieved in this pipelined processor is _____

✓ (A) 3.2

(B) 3.0

(C) 2.2

(D) 2.0

$$f = 2.5 \text{ GHz} \Rightarrow T = \frac{1}{2.5}$$

$$CPI = 4 \quad \Sigma T_{old} \Rightarrow 4 \times \frac{1}{2.5} \text{ ns} = \boxed{1.6 \text{ ns}}$$

pipeline processor

$$K = 5$$

$$f_p = 2 \text{ GHz}$$

$$t = \frac{1}{2 \text{ GHz}}$$

$$= \boxed{0.5 \text{ ns}}$$

$$CPI = 1$$

$$ET_{new} = 1 \times 0.5 \text{ ns}$$

$$= 0.5 \text{ ns}$$

$$\text{Speedup} = \frac{1.6}{0.5}$$

$$= \boxed{3.2}$$

③ Consider the following code sequence having five instructions I_1 to I_5 . Each of these instructions has the following format.

OP R_i, R_j, R_k

where operation OP is performed on the contents of

I1: ADD R1, R2, R3
 I2: MUL R7, R1, R3
 I3: SUB R4, R1, R5
 I4: ADD R3, R2, R4
 I5: MUL R7, R8, R9

Consider the following three statements.

- X S1: There is an anti-dependence between instructions I2 and I5
 → write-before-read
 ✓ S2: There is an anti-dependence between instructions I2 and I4.
 X S3: Within an instruction pipeline an anti-dependence always creates one or more stalls.

which one of the above statements is/are correct?

- (A) Only S1 is true
 (B) Only S2 is true
 (C) Only S1 and S2 are true.
 (D) Only S2 and S3 are true.

4. Consider the following processors (ns stands for nano-seconds).

Assume that the pipeline registers have zero latency.

P1: Four-stage pipeline with stage latencies 1ns, 2ns, 2ns, 1ns $\Rightarrow 2$

P2: Four-stage pipeline with stage latencies 1ns, 1.5ns, 1.5ns, 1.5ns $\Rightarrow 1.5$

P3: Five-stage pipeline with stage latencies 1ns, 1ns, 0.6ns, 1ns, 1ns $\Rightarrow 1$

P4: Five-stage pipeline with stage latencies 0.5ns, 0.5ns, 1ns, 1ns, 1.1ns $\Rightarrow 1.1$

which processor has the highest peak clock frequency?

(A) P1

(B) P2

(C) P3

(D) P4

$$\uparrow f = \frac{1}{t} \downarrow$$

5.

An instruction pipeline has five stages, namely instruction fetch (IF), instruction decode and register fetch (ID/RF), instruction Execution (EX), memory access (MEM), and register write back (WB) with stage latencies 1ns, 2.2ns, 2ns, 1ns, and 0.75 ns respectively. (ns stands for nano seconds).

To gain intensity of frequency, the designers have decided to split the ID/RF stage into three stages (ID, RF1, RF2)

each of latency $2.2/3$ ns. Also EX stage is split into two

stages (EX1, EX2) each of latency 1 ns. The new design has a

total of 8 pipeline stages. A program has 20% branch

instructions which execute in the EX stage and produce

the next instruction pointer at the end of the EX stage

in the old design and at the end of the

EX2 stage in the new design. The IF stage stalls after

fetching a branch instruction until the next instruction

pointer is computed. All instructions other than the

branch instructions have an average CPI of one in both

the designs. The execution times of this program on the

old and the new design are p and q nanoseconds, respec-

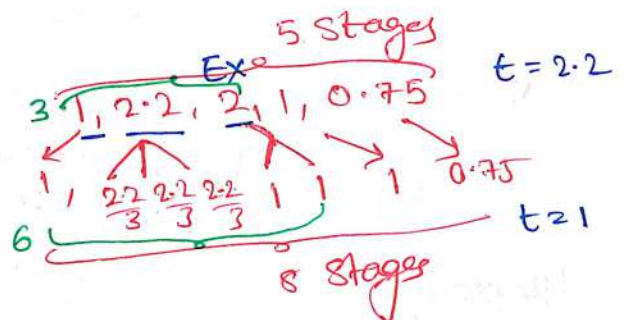
tively. The value of P/Q is 1.54

(A) 1.5

(B) 1.4

(C) 1.8

(D) 2.5

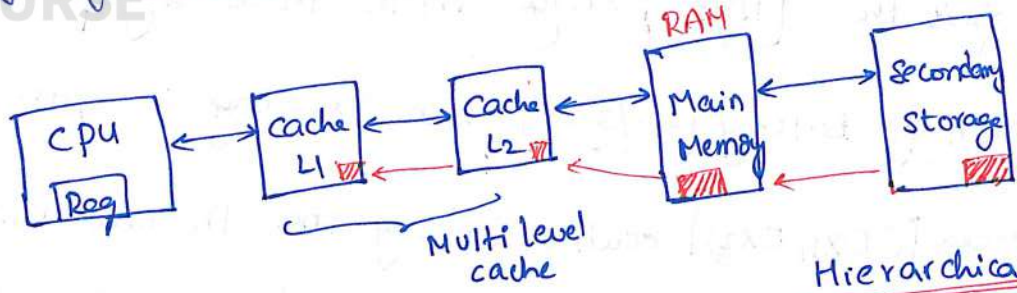


Ist case	II nd Case
$= (0.80 \times 1 + 0.2 \times 3) 2.2$ $P = 3.08$	$= (0.8 \times 1 + 0.2 \times 6) 1 \text{ ns}$ $Q = 2 \text{ ns}$

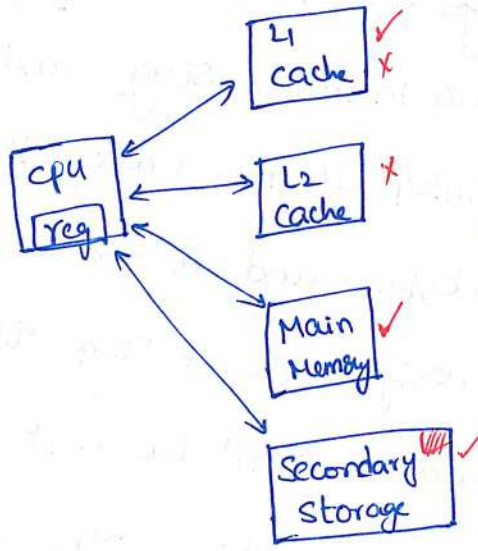
$$P/Q = 3.08/2 = 1.54$$

Memory Organization:

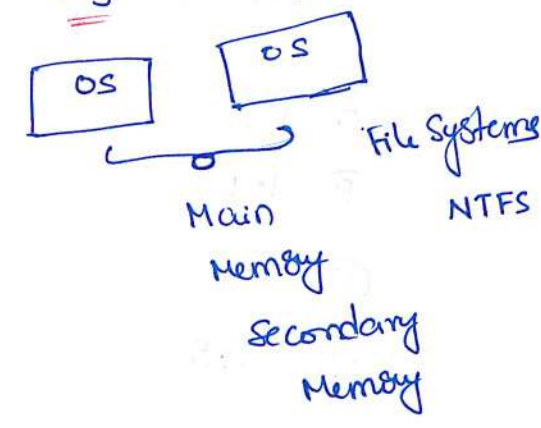
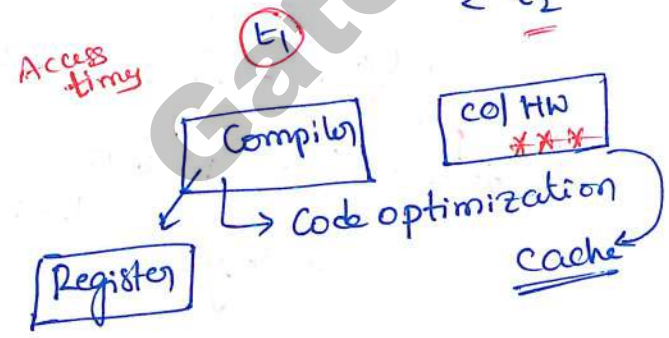
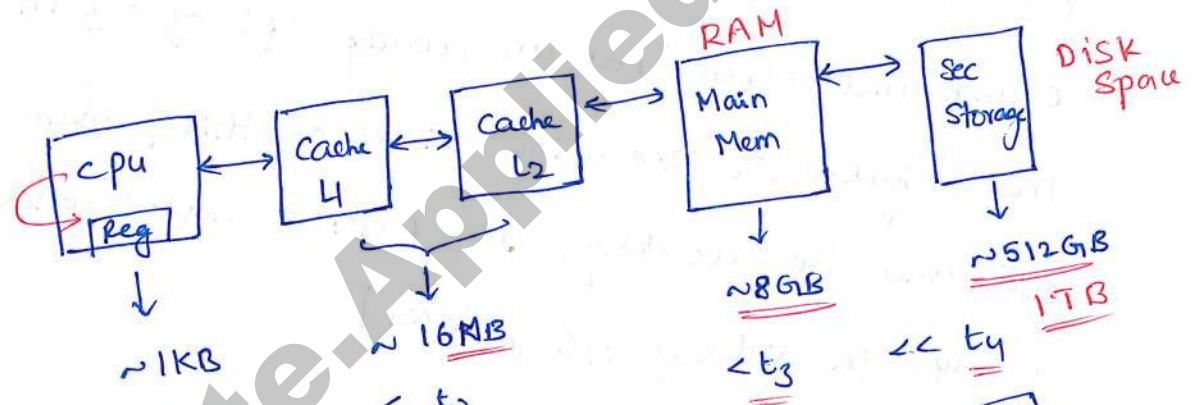
Reg, Cache, RAM, Disks
Ph: 844-844-0102

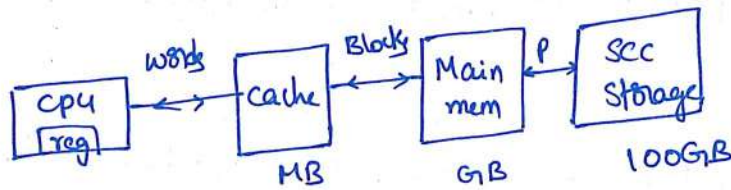


Hierarchical-access-memory-hierarchy
(default)

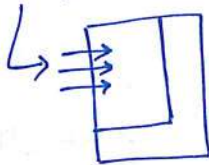
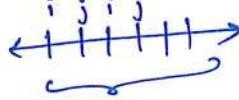


Simultaneous-access-Memory-Hierarchy.



Locality of Reference:SpatialTemporal

10X

SpatialTemporal $i, j, \dots$ variables

Repeatedly accessing the variables kept in cache known as Temporal LOR.

Keep Instructions & data in Cache is known as Spatial LOR.

Simultaneous - Access:

Expected time to access

 T_{avg}

Expected Time to Access

$$H_1 T_1 + (1-H_1) H_2 T_2$$

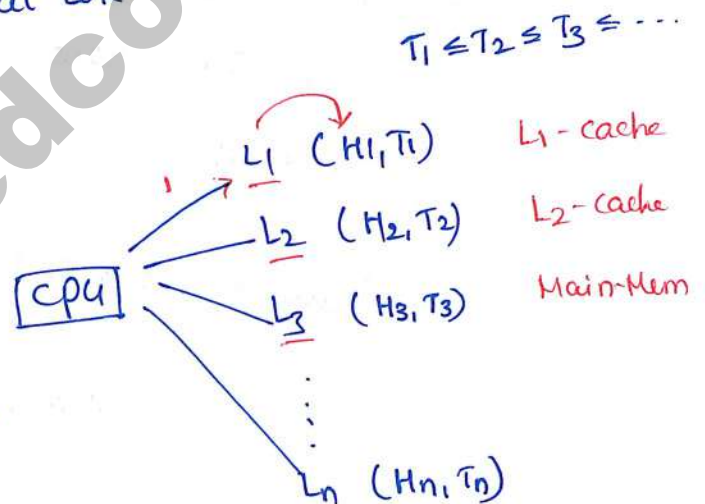
$$+ (1-H_1)(1-H_2) H_3 T_3 +$$

$$\dots + (1-H_1)(1-H_2) \dots (1-H_{n-1}) H_n T_n$$

$$+ (1-H_1)(1-H_2) \dots (1-H_{n-1}) H_n T_n$$

$$\eta_{mem} = \frac{1}{T_{avg}} \text{ words/sec}$$

Avg Access Speed.



$$T_1 \leq T_2 \leq T_3 \leq \dots$$

L1 - cache

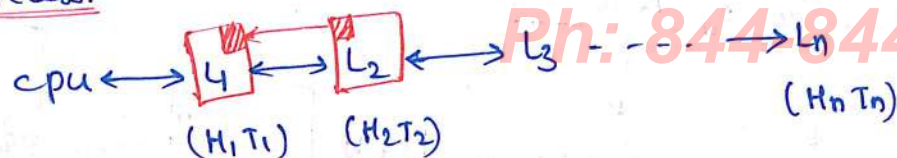
L2 - cache

Main-Mem

$$\text{Hit-ratio/rate} = \frac{\# \text{ hits}}{\# \text{ accesses}}$$

$$= H_i \text{ Miss}$$

T_i : time to access stage i

Hierarchical - Access:

$$T_{avg} = \underbrace{H_1 T_1}_{\substack{\text{Hit in } L_1 \\ \downarrow}} + \underbrace{(1-H_1) H_2 (T_2 + T_1)}_{\substack{\text{Miss in } L_1 \\ \uparrow}} + (1-H_1)(1-H_2) H_3 (T_1 + T_2 + T_3) + \dots$$

$$+ (1-H_1)(1-H_2) \dots (1-H_{n-1}) H_n (T_1 + T_2 + \dots + T_n)$$

L_1 Miss L_2 Miss L_{n-1} Miss $\downarrow$ Hit in L_n

$\Rightarrow H_n = 1$

10. 2-level memory-hierarchy (sequential)

L_1 is 8 times faster than L_2

$$T_1 = 20 \text{ ns}$$

$$T_1 = T_{avg} - 40 \text{ ns}$$

Find H_1

$$8t_1 = t_2$$

$$8 \times 20 = t_2$$

$$\Rightarrow t_2 = 160$$

$$T_{avg} = H_1 \times T_1 + (1-H_1) H_2 T_2 = T_1 + 40 \text{ ns}$$

$$20 \times H_1 + (1-H_1) \times 160 = T_1 + 40$$

$$= 20 + 40$$

$$20H_1 + 160 - 160H_1$$

$\Rightarrow$

$$H_1 = 0.714 \quad (71.4\%)$$

Cache - Memory organization:

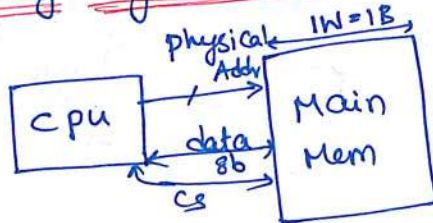
- Memory-organization
- Mapping-techniques
- Replacement - Methods
- Updates
- Multi-level-Cache → L₁ L₂ L₃

Hierarchical Access



(Locality of Reference)

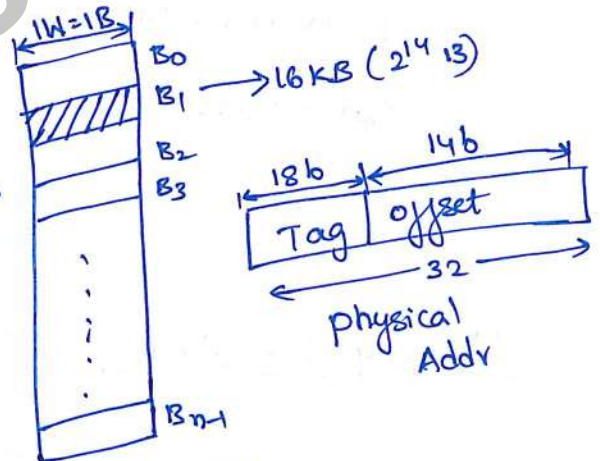
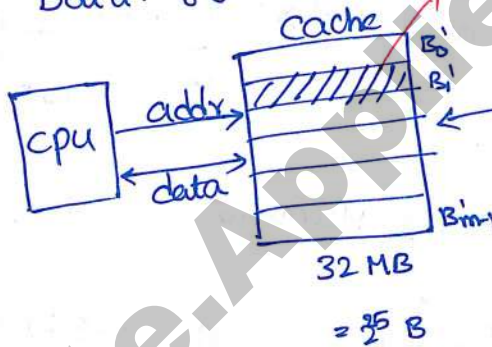
Memory Organization:



$$4 \text{ GB} = 2^{32} \text{ B} = 2^{32} \text{ W}$$

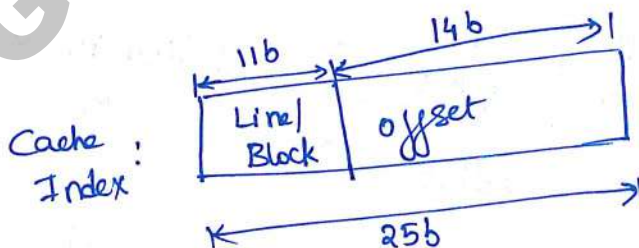
Addr: 32b

Data: 8b



Main Memory

$$4 \text{ GB} = 2^{32} \text{ B} = 2^{32} \text{ W}$$

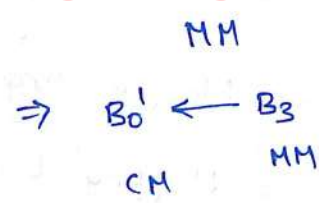
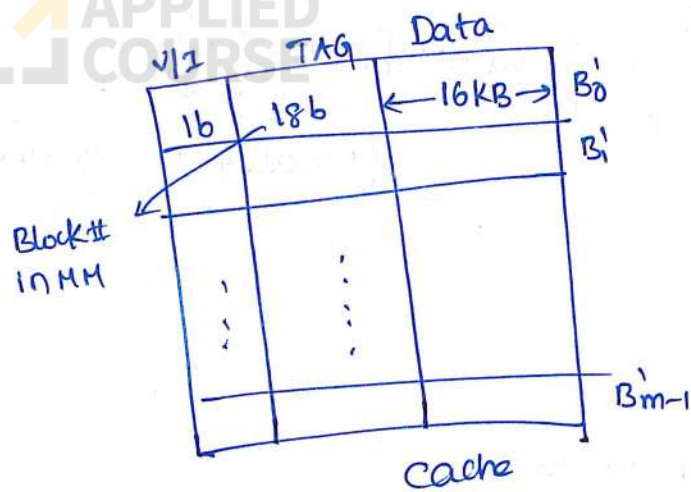


$$\# \text{ Blocks in MM} = \frac{2^{32}}{2^{14}} = 2^{18} = n$$

$$\# \text{ Blocks in CM} = \frac{2^{25}}{2^{14}} = 2^{11} = m$$

Total size of CM = $m * (1 + 18 + 16k * 8)$

Ph: 844-844-0102



Q1) 32KB cache = 2^{15} B
 32B - blocks = 2^5 B $\Rightarrow 1B = 1W$
 CPU: 32b physical Addr

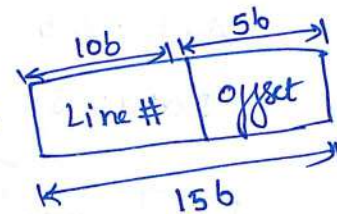
lines in CM = $\frac{2^{15}}{2^5} = 2^{10} = m$

blocks in MM = $\frac{2^{32}}{2^5} \Rightarrow 2^{32-5} \Rightarrow 2^{27} n$

Addr-Format =



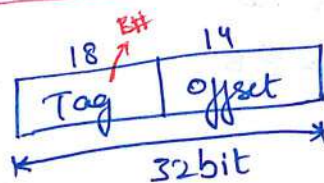
Cache Index:



Mapping Techniques: Direct-Address-Mapping

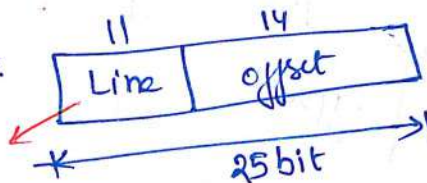
$f() : \text{mod}$

physical-addr:



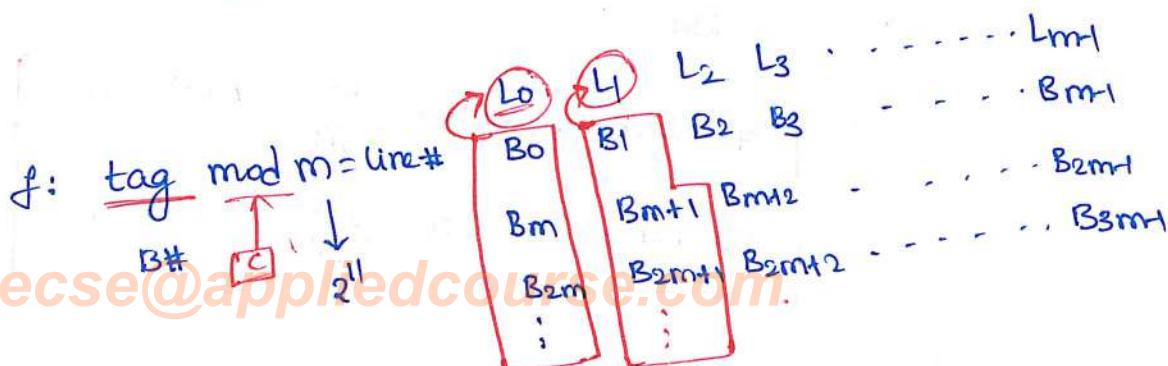
blocks in MM = $2^{18} = n$

cache-index:



lines in CM = $2^{11} = m$

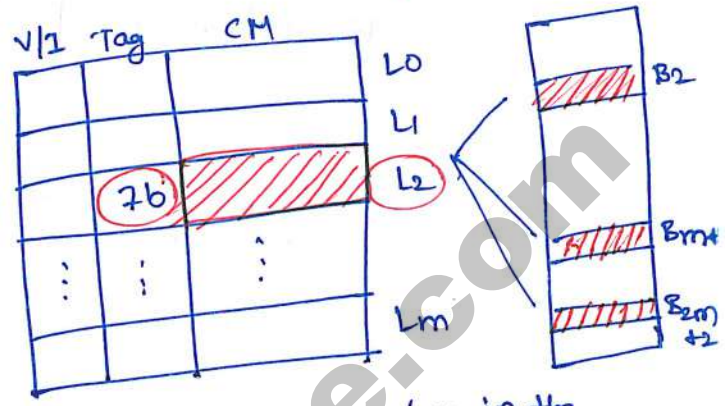
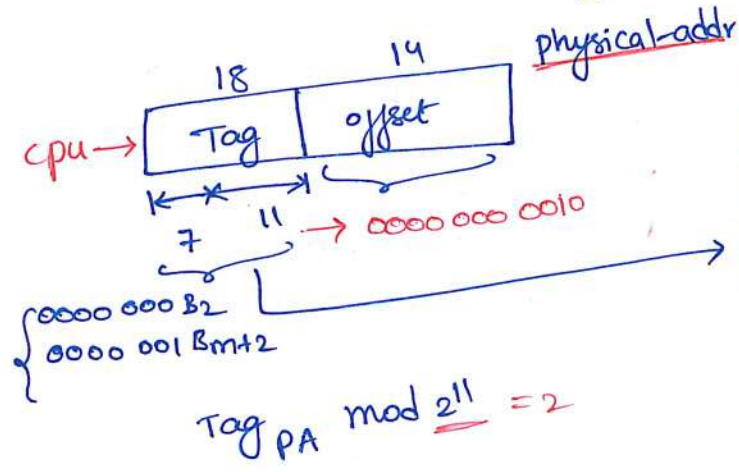
$pa_1 = B_2$
 $pa_2 = B_{m+2}$



Computing - Mod:

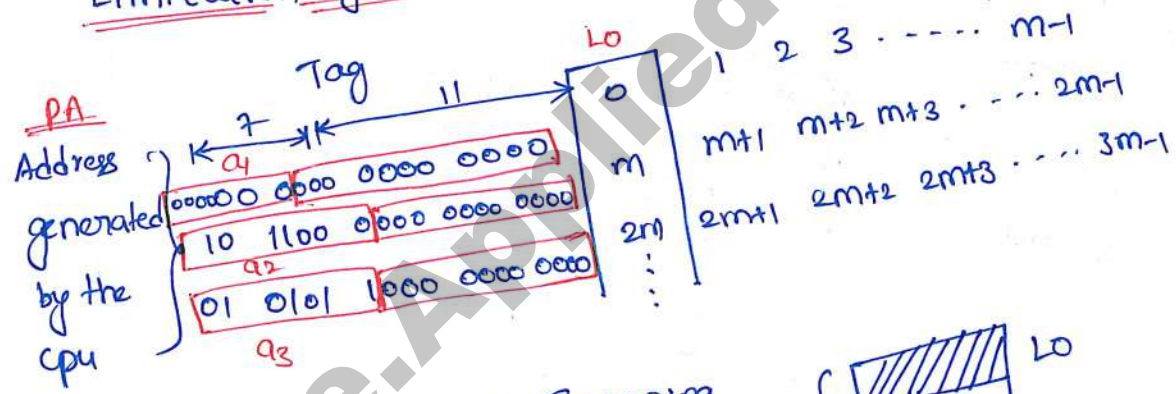
Base 10: $123 \bmod 10 = 3$
 $1678 \bmod 100 = 78$

Base 2:
 $1000 \bmod 100 = 00$
 $1001 \bmod 100 = 01$
 $1010 \bmod 100 = 10$

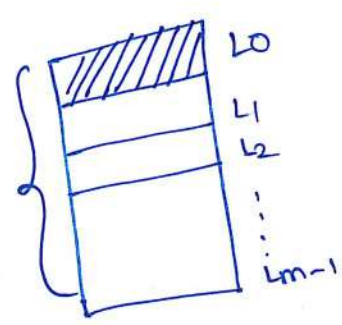


In case of cache MISS, we need to replace in the cache.

Limitation of Direct Mapping:



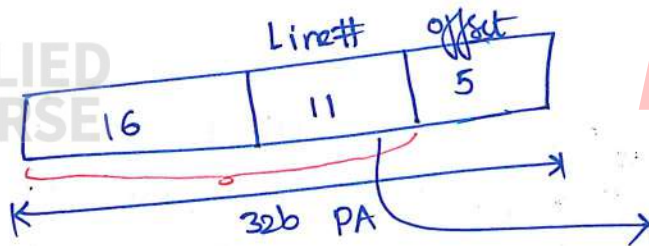
→ Trashing: Too much swapping



Eg: 64KB direct-mapped cache
32B-blocks/line $\Rightarrow 2^5$
cpu: 32b-addr

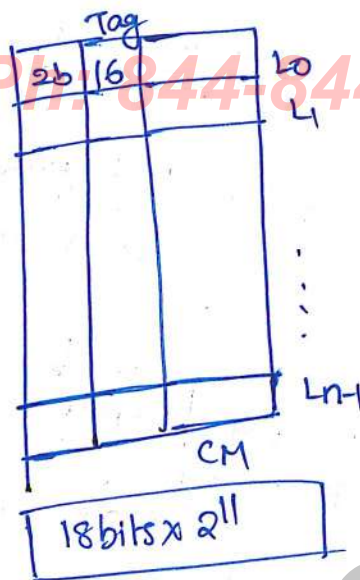
cache-controller: one valid bit, one modified bit, tag-bits.

← Tag →
tag-bits # size of tag-memory!

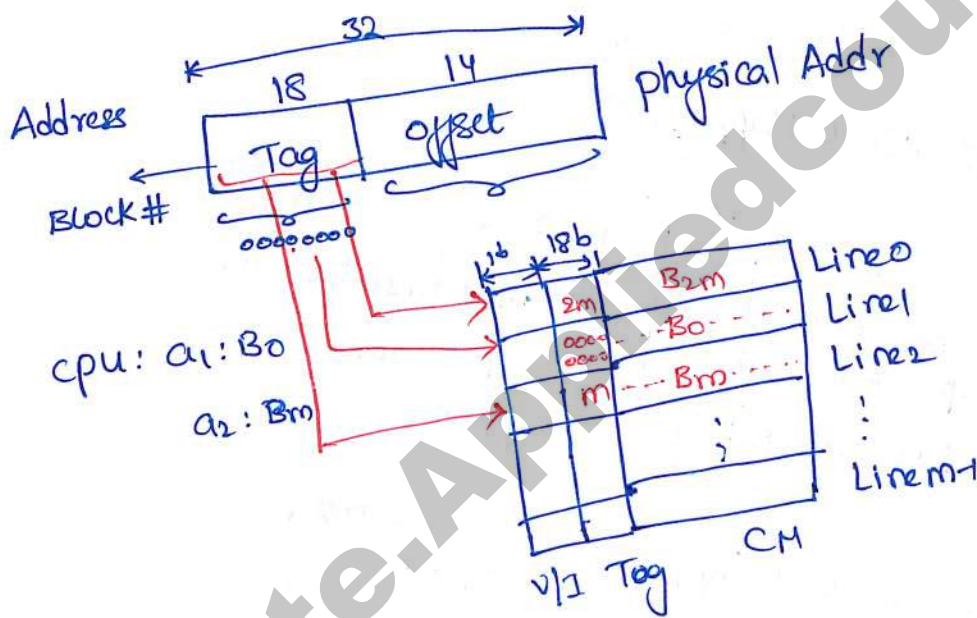


tag bits = 16

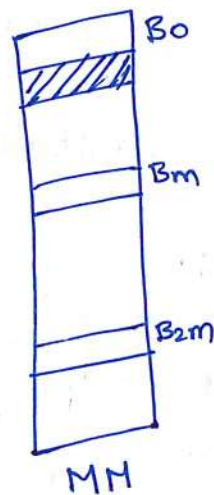
size of TAG Memory = $18 \text{ bits} \times 2^{11}$



Associative - addr - Mapping: (Fully)

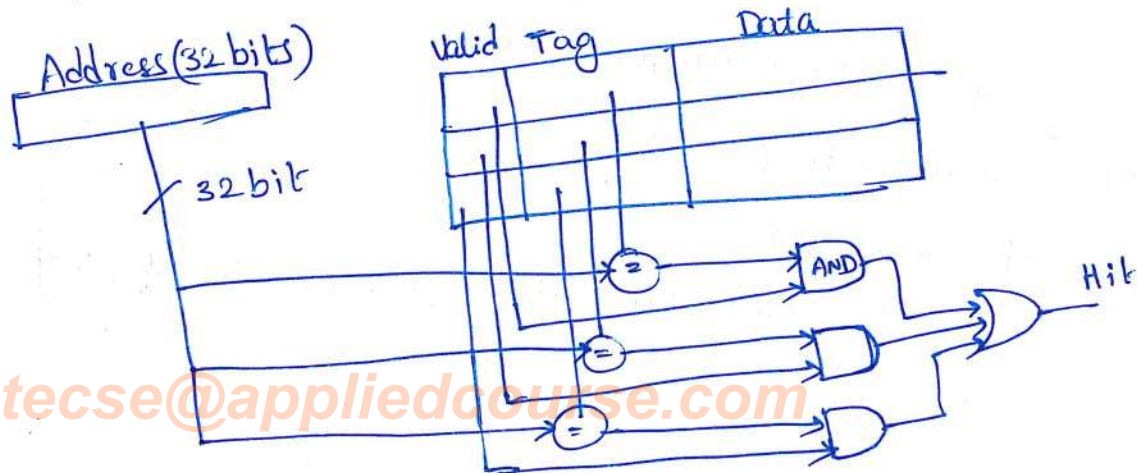


n-blocks



m-parallel-Comparisons:

$m - 2^{11}$
Circuit Complex



Q. 32-KB fully-Associative - Cache Memory

Ph: 844-844-0102

2¹⁵

Line size = 32B $\Rightarrow 2^5$ B

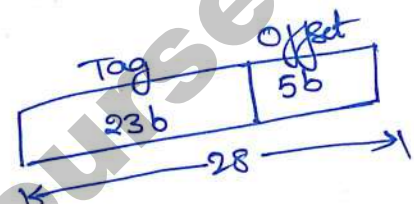
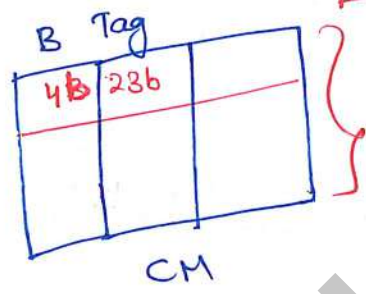
Main-mem-size = 2^{28} B

1 valid bit, 1 modified bit, 2 replacement bits.

(a) # tag-bits : 23

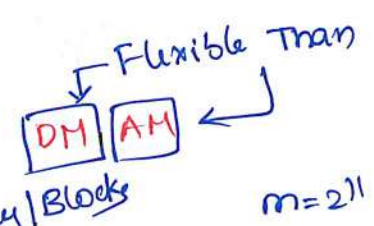
(b) Tag-Memory size: $(4 + 23) \times 2^{10}$ # blocks $= n = \frac{2^{28}}{2^5}$

$\downarrow$
 27×2^{10}

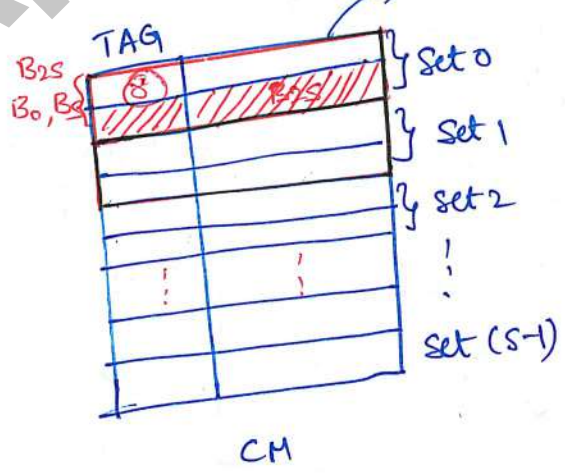
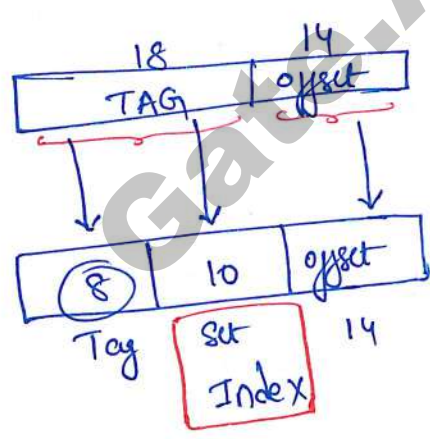


$\Rightarrow n = \frac{2^{15}}{2^5} = 2^{10}$

Set-Associative Mapping / Cache Memory:



$m = 2^{11}$



2 way
 $\downarrow$
2 Blocks/set

each set : 2 lines

Set-Index = TagPA mod 2¹⁰

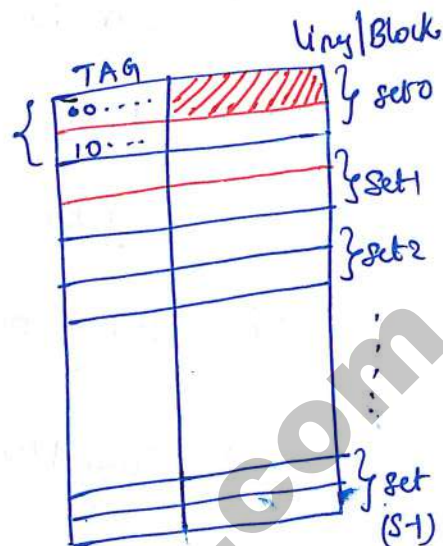
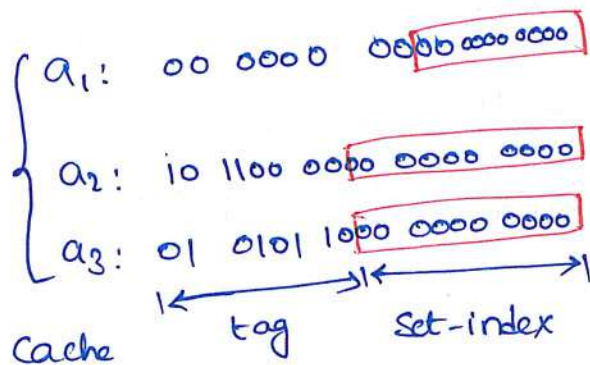
$S = \frac{m}{2} = \frac{2^{11}}{2} = 2^{10}$

a: B0

2-Comparisions $\leftarrow$ 2-way

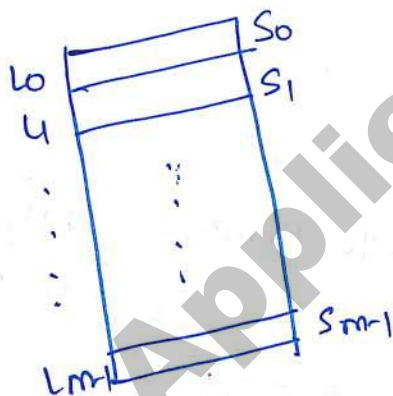
4-Comparisions $\leftarrow$ 4-way

PA: $\leftarrow 8b \rightarrow \leftarrow 10 \rightarrow \leftarrow 14b \text{ offset} \rightarrow$



Comparisons = 2 in 2-way SA-Mem
 = K in K-way SA-mem

1 way - Set - Associative $\Rightarrow$ Direct-Mapping



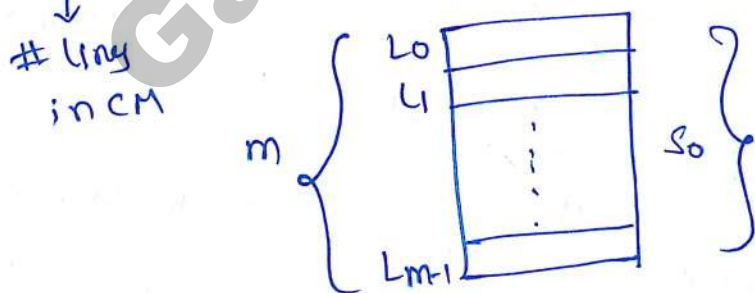
sets = # of Lines
 = 2¹¹

In practice

2-way

4-way

M-way Set-associative $\Rightarrow$ Fully - Associative



sets = 1 = 2⁰

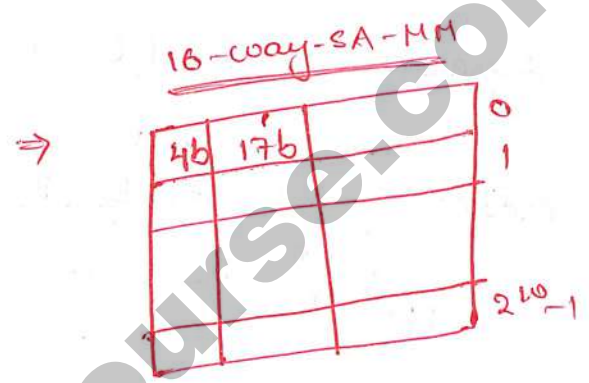
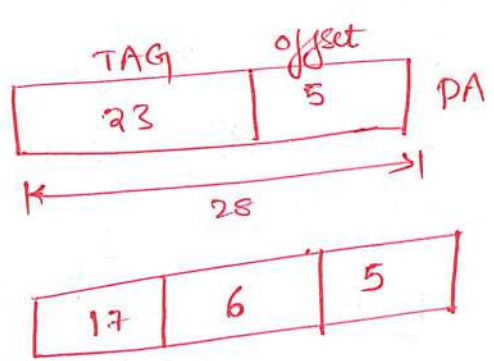
① CM-size = 32KB = 2^{15} B
Block-size = 32B = 2^5 B
MM-size = 2^{28} B

$\Rightarrow \frac{2^{15}}{2^5} = 2^{10}$ lines
 $S = \frac{2^{10}}{2^4} = 2^6$

$2^4 = 16$ -way-SA-MM

① addr-format

② Size of Tag-Mem with 4-additional bits/block.



$(4b + 17b) * 2^{10}$
21Kb

Replacement - Strategies:

V/I	TAG	CM	
1	✓	B_0	B_m L_0
1	✓	B_1	L_1
			$\vdots$
			L_{m-1}

Direct Addressing

addr: $B_0, B_1, B_m, B_2, \dots$

$m \bmod m \Rightarrow 0 \Rightarrow L_0$

V/I	TAG	CM	
1	0	B_0	L_0
1	m	B_1	$B_m L_1$
1	m	B_m	
1	2	B_2	
		$\vdots$	
✓	✓		
✓	✓		L_{m-1}

Associative Mapping

B_{3m}

V/I	TAG	CM	
1	✓	B_0	B_{2s} S_0
1	✓	B_s	S_1
			$\vdots$
			S_s

2-way Set-Associative Mapping

addr: $B_0, B_s, B_{2s}, B_1, \dots$

$0 \bmod s$

$2s \bmod s$ $s \bmod s \Rightarrow 0$

Replacement: FIFO Queue

addr: $B_0, B_s, B_1, B_{2s}, B_{s+1}, B_{2s+1}, \dots$

$$1 \bmod s \Rightarrow 1$$

$$2s \bmod s \Rightarrow 0$$

$$2s+1 \bmod s \Rightarrow 1$$

V/I TAG

1	✓	B_0	B_{2s}	} s_0
1	✓	B_s		
1	✓	B_1	B_{s+1}	} s_1
1	✓			
				} s_{s-1}

2-way set associative Mapping

Replacement: LRU (Least Recently used) → used in practical

addr: $B_0, B_s, B_1, B_{2s}, B_{s+1}, B_{2s+1}, \dots$

Annotations: LRU (under B_0), M (under B_s), Most (under B_{s+1})

$$2s \bmod s \Rightarrow 0$$

$$B_{2s+1} \bmod s \Rightarrow 1$$

V/I TAG

1	✓	B_0	B_{2s}	} s_0
1	✓	B_s		
1	✓	B_1	B_{s+1}	} s_1
1	✓			
				} s_{s-1}

2-way set associative Mapping

Replacement: optimal → Not practical

addr: $B_0, B_s, B_1, B_{2s}, B_{s+1}, \dots$

B_0, B_s

↑ time

→ Future

V/I TAG

1	✓	B_0	B_{2s}	} s_0
1	✓	B_s		
1	✓	B_1	B_{s+1}	} s_1
1	✓			
				} s_{s-1}

Eg 1:

4-block Cache (Initially Empty)

Order of main-memory block references

4, 5, 7, 12, 14, 5, 13, 4, 5, 7

Calculate hit-ratio if

① DAM

② 2-way-set-associative with LRU

①

V/I TAG		
1	✓	4 4
1	✓	5 13 5
1	✓	7

CM



$m=4$

$\Rightarrow 4 \bmod 4 = 0$

$5 \bmod 4 = 1$

$7 \bmod 4 = 3$

$12 \bmod 4 = 0$

$4 \bmod 4 = 0$

$5 \bmod 4 = 1 \Rightarrow \text{HIT} \checkmark$

$13 \bmod 4 \Rightarrow 1 \Rightarrow$

$4 \text{ HIT} \checkmark$

5 MISS

$7 \text{ HIT} \checkmark$

Hit Rate = $\frac{3}{10} = 0.3$
= 30%

②

V/I Tag		
✓		4
✓		12
✓		5
✓		13

CM

MM M M H H M H H M
~~4~~, ~~5~~, ~~7~~, 12, 4, 5, 13, 4, 5, 7

$\Rightarrow 4 \bmod 2 = 0 \text{ Set0}$

$\Rightarrow 5 \bmod 2 = 1 \text{ Set1}$

$\Rightarrow 7 \bmod 2 = 1 \text{ Set1}$

$\Rightarrow 12 \bmod 2 = 0 \text{ Set0}$

$13 \bmod 2 = 1$

$\Rightarrow \frac{4}{10} = \text{40\% Hit Ratio}$

Update Techniques:

Ph: 844-844-0102

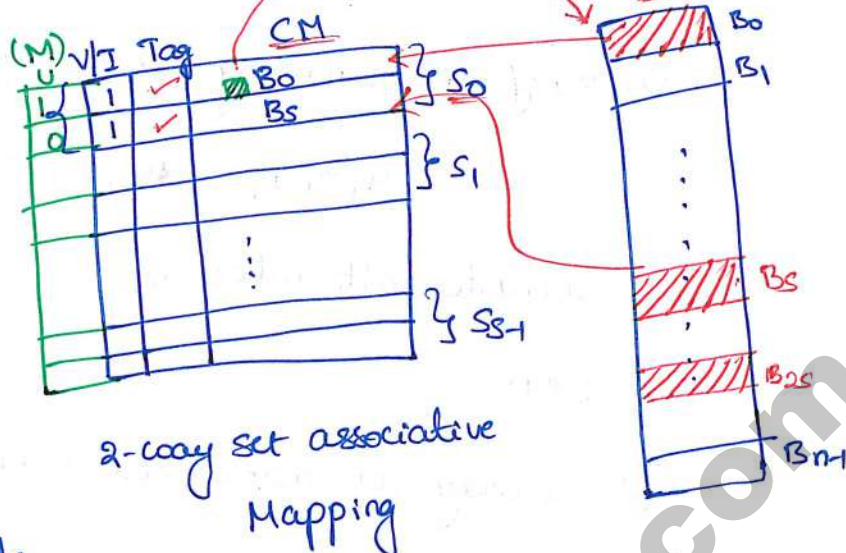
→ update / Modify bit

Cache-coherence:

→ Write-Through

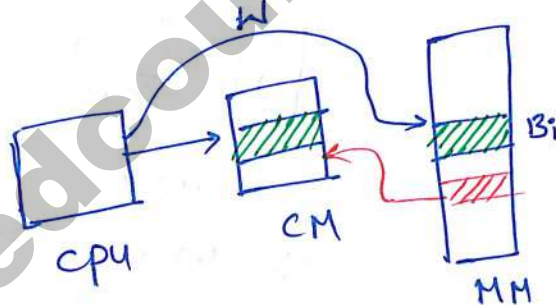
→ Write-back

Any Edit / updates in cache memory need to update in Main memory.



addr: $\underline{B_0}, B_2, B_3, B_1$
W R R

Write-Through



$$T_W = \max(CM_w, MM_w)$$

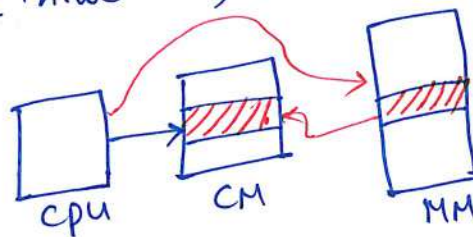
Simultaneous Write

Expected time

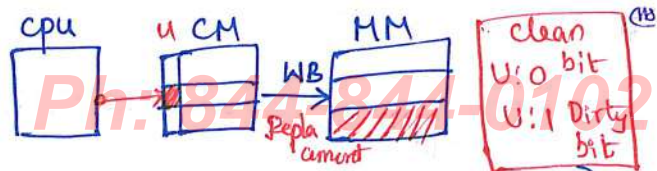
$$T_{avg-r} = H_r T_r + (1-H_r) (T_{cr} + T_{mr})$$

+ T_{aloc} → which block is to be Replaced

$$T_{avg-w} = \frac{H_w T_w}{Hit} + \frac{(1-H_r)}{Miss} (T_{aloc} + T_w) + T_{mr}$$



$$T_{avg-wt} = f_r * T_{avg-r} + f_w * T_{avg-w}$$



Write-back :

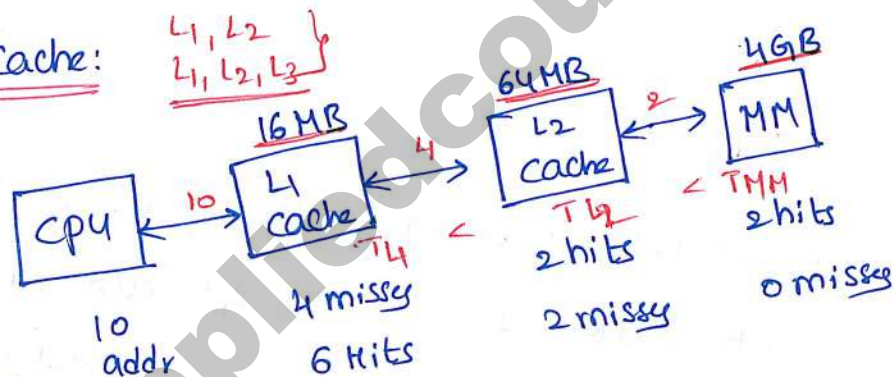
$$T_{avg-r} = H_r T_{cr} + (1-H_r) \left[P_d * (T_{WB} + T_{alloc} + T_{mr} + T_{cr}) + P_c * (T_{alloc} + T_{mr} + T_{cr}) \right]$$

c : clean bit
d : dirty bit

$$T_{avg-w} = H_w T_{cw} + (1-H_w) \left[P_d (T_{WB} + T_{alloc} + T_{mr} + T_{cw}) + P_c (T_{alloc} + T_{mr} + T_{cw}) \right]$$

$$T_{avg-WB} = f_r T_{avg-r} + f_w T_{avg-w}$$

Multi-Level cache:



Miss-penalty :

Time taken to service a miss

Global-miss-rate:

$$\frac{4}{10} \quad \frac{2}{10} \quad \frac{0}{10}$$

Local-miss-rate

$$\frac{4}{10} \quad \frac{2}{4} \quad \frac{0}{20}$$

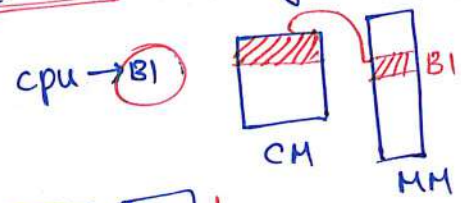
$$T_{avg} = (H_{L1} * T_{L1}) + \text{Miss rate}_{L1} * \text{Miss-penalty}_{L1}$$

$$\text{Miss-penalty}_{L1} = (H_{L2} * T_{L2}) + \text{Miss rate}_{L2} * \text{Miss-penalty}_{L2}$$

$$\text{Miss-penalty}_{L2} = T_{MM}$$

Types of cache-Misses:

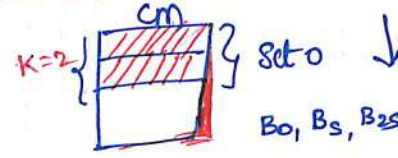
Ph: 844-844-0102

- ① Compulsory / Cold-start / First-ref miss


Size of block ↑

Size of cache ↑

K-way SA ↑
- ② Capacity - Miss

- ③ Conflict-Miss


Initially cache is Empty. we need to bring the data from MM and store in CM $\Rightarrow$ Compulsory Miss cold-start

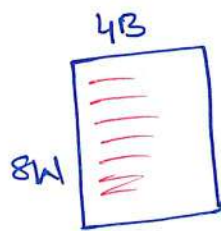
More Solved problems-1

- ① A certain processor employs a single-level cache. The cache block size is 8 words and the word size is 4 bytes. The memory system uses a 60 MHz clock. To service a cache-miss, the memory controller first takes 1 cycle to accept the starting address of the block, it then takes 3 cycles to fetch all the eight words of the block and finally transmits the words of the requested block at the rate of 1 word per cycle. The maximum bandwidth for the memory system when the program running on the processor issues a series of read operations $\frac{160 \times 10^6 \text{ bytes}}{\text{sec.}}$ $\Rightarrow$ 160

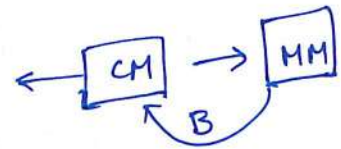
$$f = 60 \text{ MHz}$$

$$\Rightarrow 10^6 = \text{M}$$

$$t = \frac{1}{60 \text{ M}}$$



best-case
 $8W = 1 + 3 + 8 \times 1$
 $= 12 \text{ Cycles}$



$$\frac{4B \times 8W}{W} \rightarrow 12$$

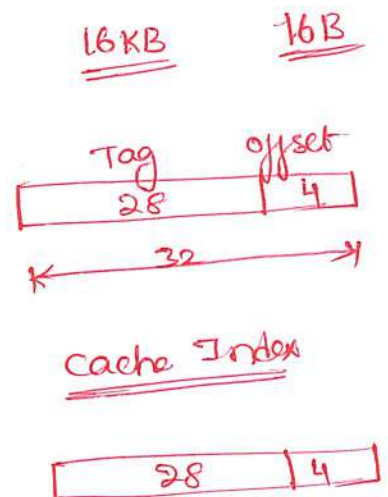
$$32B \rightarrow 12 \times \frac{1}{60 \text{ M}} \text{ sec}$$

$$= 160 \text{ MB/s}$$

2

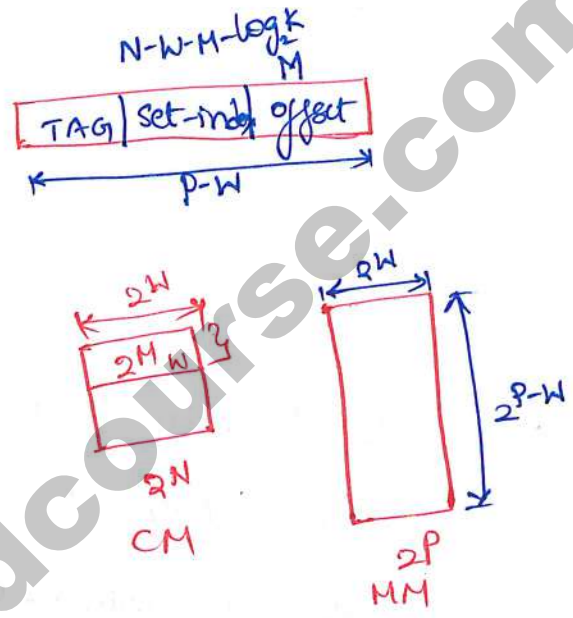
A certain processor uses a fully associative cache of size 16 KB. The cache block size is 16 bytes. Assume that the main memory is byte addressable and uses a 32-bit address. How many bits are required for the tag and the index fields respectively in the address generated by the processor?

- ☒ (A) 24 bits and 0 bits
- ☒ (B) 28 bits and 4 bits
- ☒ (C) 24 bits and 4 bits
- ☒ (D) 28 bits and 0 bits.



③ The size of the physical address space of a processor is 2^P bytes. The word length is 2^W bytes. The capacity of cache memory is 2^N bytes. The size of the each cache block is 2^M words. For a K-way set-associative cache memory, the Length (in number of bits) of the tag field is

- (A) $P - N - \log_2 K$
- (B) $P - N + \log_2 K$
- (C) $P - N - M - W - \log_2 K$
- (D) $P - N - M - W + \log_2 K$



$$CM_N = 2^{N-W}$$

$$2^{N-N-M} = \# \text{ Blocks in CM}$$

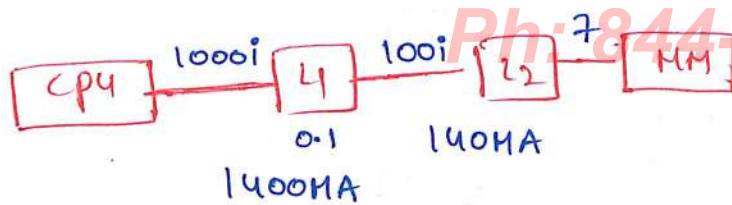
$$\# \text{ sets} = \left(\frac{2^{N-W-M}}{K} \right)$$

$$\Rightarrow P - W - N - M + \log_2 K - M$$

$$P - N + \log_2 K$$

$$\Rightarrow N - W - M - \log_2 K$$

④ Consider a two-level cache hierarchy L1 and L2 caches. An Application incurs 1.4 memory accesses per instruction on average. For this application, the miss rate of L1 cache 0.1, the L2 cache Experience on average. 7 misses per 1000 instructions. The miss rate of L2 expressed correct to two decimal places is 0.05



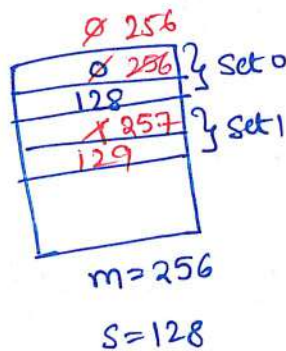
Global Miss-rate

$$LMR = \frac{7}{140}$$

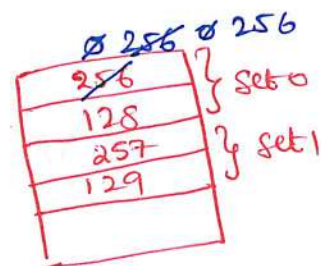
$$= \frac{1}{20}$$

$$= 0.05 = \boxed{5\%}$$

- ⑤ Consider a two-way set associative cache with 256 blocks and uses LRU replacement. Initially the cache is Empty. Conflict misses are those misses which occur due to the contention of multiple blocks for the same cache set. Compulsory misses occur due to first-time access to the block. The following sequence of accesses to memory blocks: (0, 128, 256, 128, 0, 128, 256, 128, 1, 129, 257, 129, 257, 129) is repeated 10-times. The number of Conflict misses experienced by the cache is $\underline{\hspace{2cm}}$



256 both Compulsory & Conflict miss



$$1: 0, 256, \underline{0}, 256 \rightarrow 4$$

$$2: 0, 256, 0, 256 \rightarrow 4$$

$$1, 129, 1, 129 \rightarrow 4$$

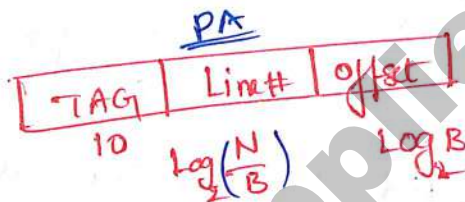
1 → 4
2 → 8
3 → 8
⋮

256	} S ₀
128	
257	} S ₁
129	

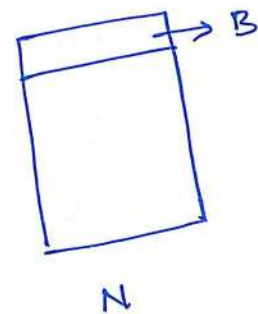
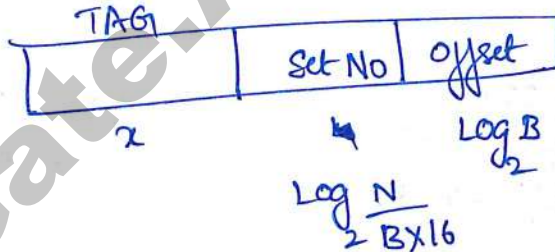
10 → 8 ⇒ $9 \times 8 + 4 = 72 + 4 = 76$ Conflict miss

- ⑥ A cache memory unit with a capacity of N words and block size of B words is to be designed. If it is designed as a direct mapped cache, the Length of the TAG field is 10 bits. If the cache unit is now designed as a 16-way set associative cache, the Length of the TAG field is _____ bits.

DAM:-



S.A:-



Number of Lines = $\frac{N}{B}$

$$10 + \log \frac{N}{B} + \log B = x + \log \frac{N}{16B} + \log B$$

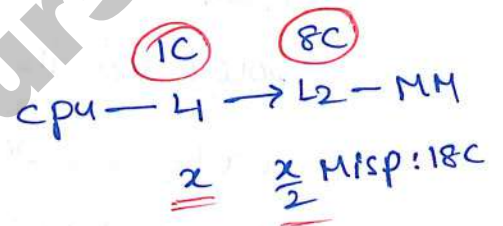
$$10 + \log \frac{N}{B} = x + \log \frac{N}{B} - \log 16$$

$$10 + 4 = x$$

Gatecse@appliedcourse.com ⇒ $x = 14$

- ① In a two-level cache system, the access time of L_1 and L_2 are 1 and 8 clock cycles, respectively. The miss penalty from the L_2 cache to main memory is 18 clock cycles. The miss rate of L_1 cache is twice that of L_2 . The average memory access time (AMAT) of this cache system is 2 cycles. The miss rates of L_1 and L_2 respectively are

- ✓ A. 0.111 and 0.056
 ✗ B. 0.056 and 0.111
 ✗ C. 0.0892 and 0.1784
 D. 0.1784 and 0.0892



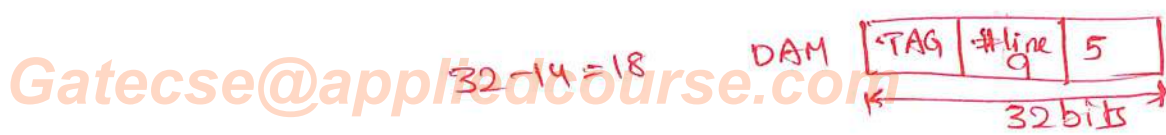
$$2 = 1 + x(8) + x \cdot \frac{x}{2} \cdot 18$$

$$\Rightarrow 9x^2 + 8x + 1 = 2$$

$$x = \frac{2}{18} = 0.111$$

- ② Consider a machine with byte addressable memory 2^{32} bytes divided into blocks of size 32 bytes. Assume a direct mapped cache having 512 cache lines are used with this machine. The size of the tag field in bits _____

- (A) 12 (B) 16 (C) 18 (D) 24



③ The read access times and the hit ratios for different caches in a memory hierarchy are as given below.

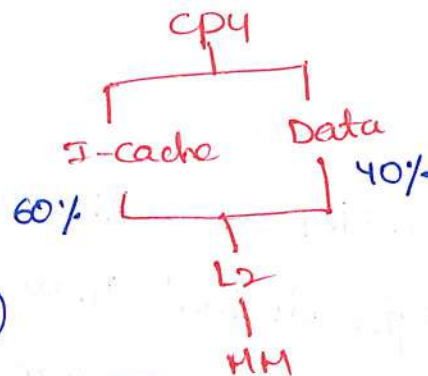
Accounts for Hit rate

Hit rate * read time

Cache	Read access time (in Nanoseconds)	Hit ratio
I-cache	2	0.8
D-cache	2	0.9
L2-cache	8	0.9

Instruction Data

The read-access time of main memory is 90 nano seconds. Assume that the caches use the staggered-word first read policy and the write back policy. Assume that all the caches are direct mapped address. Assume that the dirty bit is always 0 for all the blocks in the caches. In execution of a program, 60% of memory reads are for instruction fetch and 40% are for memory operand fetch. The average read access time in nano seconds (upto 2 decimal place) is _____



Inst: $2 + 0.2(8) + 0.2 \times 0.1(90)$

$= 2 + 1.6 + 1.8 = 5.4 \mu$

Data: $= 2 + 0.1 \times 8 + 0.1 \times 0.1 \times 90$

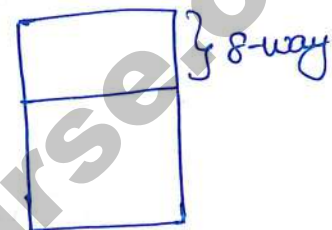
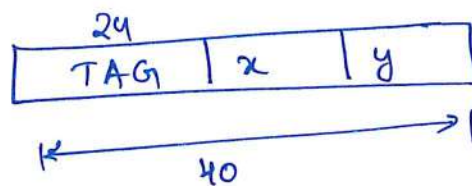
$= 3.7 \mu$

$$= 0.6 \times 5.4 + 0.4 \times 3.7$$

$$= 4.72 \text{ Sec}$$

- ④ The width of the physical address on a machine is 40 bits. The width of the TAG field in a 512KB 8-way set-associative cache is _____

- Ⓐ 24 Ⓑ 20 Ⓒ 30 Ⓓ 40



512KB

$$\Rightarrow 2^{19}$$

$$\# \text{ sets} = 2^x$$

$$\text{Block size} = 2^y$$

$$\Rightarrow 2^{19} = 2^x \times 2^3 \times 2^y$$

$$\Rightarrow 19 = x + y + 3$$

$$\Rightarrow x + y = 16$$

$$\Rightarrow 40 - 16 = 24$$

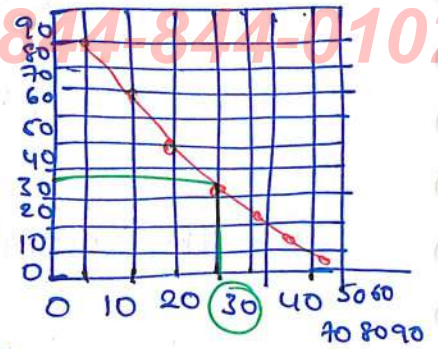
- ⑤ A file system uses an in-memory cache to cache disk blocks. The miss rate of the cache is shown in the figure. The latency to read a block from the cache is 1 ms and to read a block from the disk is 10 ms. Assume that the cost of checking whether a block exists in the cache is negligible. Available cache sizes are in multiple of 10MB. The smallest cache size is required to ensure an average

read Latency less than 6ms is 30MB MB.

Ph: 844-844-0102

Cache 1ms
Disk 10ms

Miss rate(%)



miss-rate = x

Hit-rate = $1-x$

cache size(MB)

$$\Rightarrow (1-x)1 + x(10)$$

$$9x + 1 < 6\text{ms}$$

$$x < 0.5556$$

$$\Rightarrow 55.56\%$$

$$\Rightarrow \boxed{30\text{MB}}$$

- ⑥ Assume that for a certain processor, a read request takes 50 nanoseconds on a cache miss and 5 nanoseconds on a cache hit. Suppose while running a program, it was observed that 80% of the processor read requests result in a cache hit. The average read access time in nanoseconds is _____

Ⓐ 10

Ⓑ 12

Ⓒ 13

Ⓓ 14

$$.80 \times 5 + 0.2(50)$$

$$= 4 + 10$$

$$= 14$$

⑦ In designing a Computer cache system, the cache block (or cache line) size is an important parameter. which one of the following statements is correct in this context?

X(A) A smaller block size implies better spatial locality.

X(B) A smaller block size implies a smaller cache tag and hence lower cache tag overhead.

X(C) A smaller block size implies a larger cache tag and hence lower cache hit time.

✓(D) A smaller block size incurs a lower cache mis penalty

MM
CM

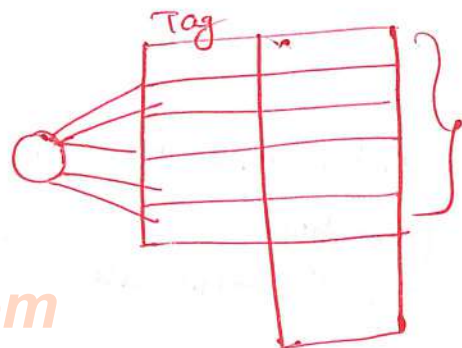
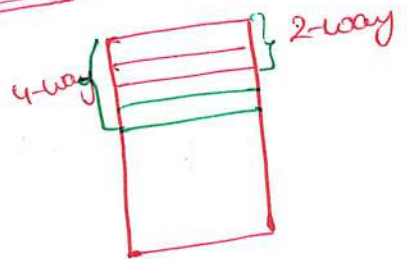
⑧ If the set associativity of a processor cache is doubled while keeping the capacity and block size unchanged, which one of the following is guaranteed to be NOT affected?

X(A) width of TAG Comparator

X(B) Width of set index decoder

X(C) Width of way selection multiplexer.

✓(D) Width of processor to main memory data bus word-size

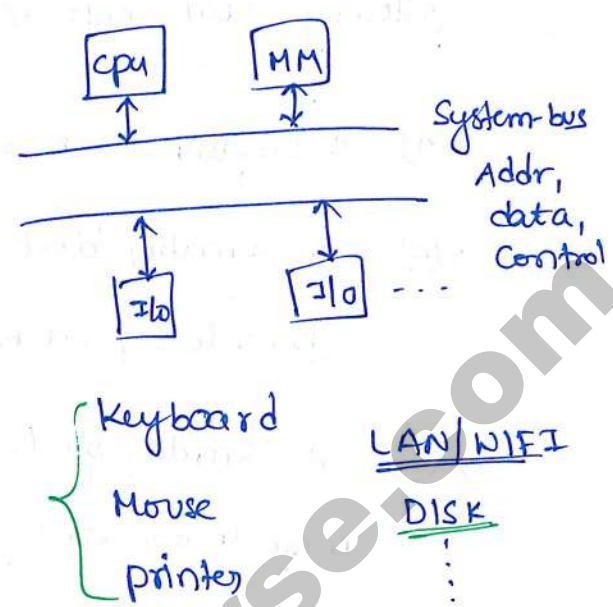


I/O Introduction:

Input-output-organization

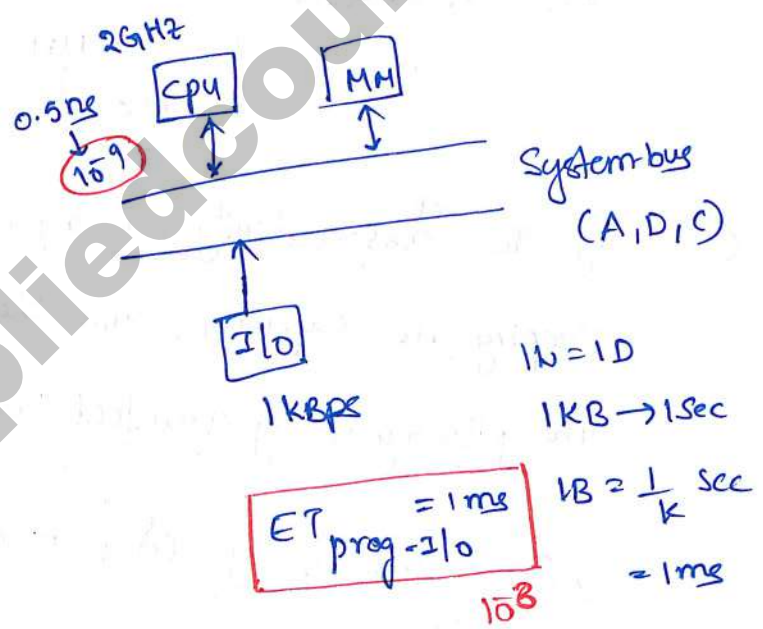
I/O Transfer-modes

1. programmed I/O
2. INT-Driven I/O
3. DMA



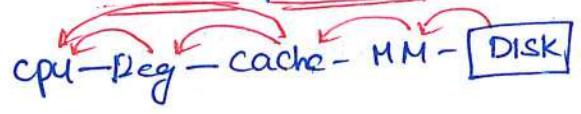
Programmed I/O:

- $I_1 : \dots$
 $I_2 : \text{LDA I/O-addr}$
 $I_3 : \dots$
 $I_4 : \dots$



CPU-wait-time

Interrupt-driven I/O:

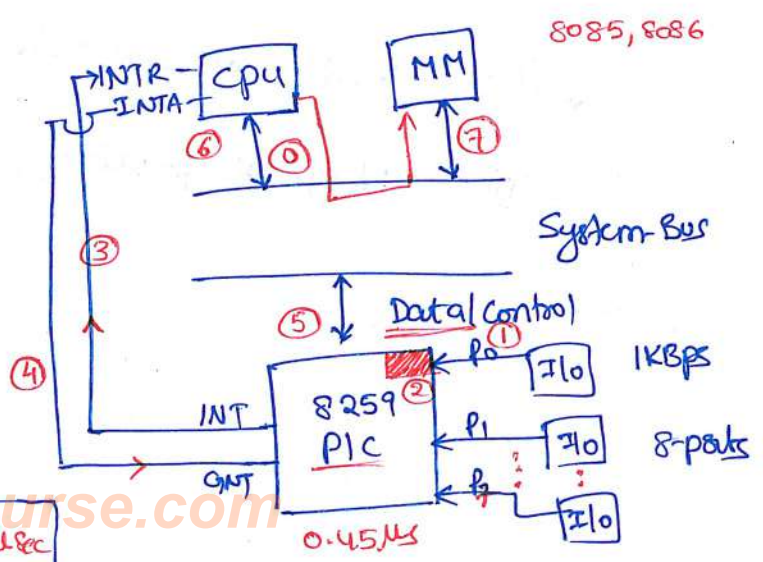


PLC

programmable Interrupt

Controller
I/O Interface

$ET_{INT-I/O} = 0.45 \mu\text{Sec}$

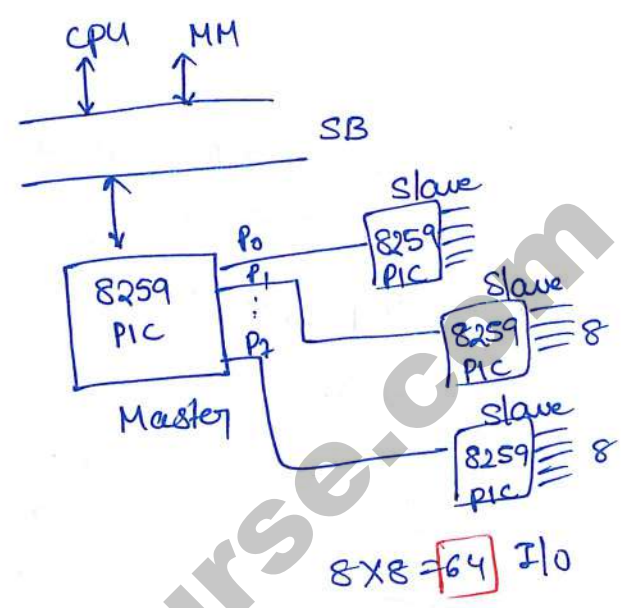
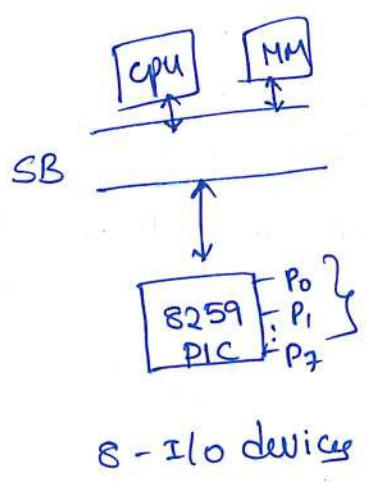


problem:

HDD → MM

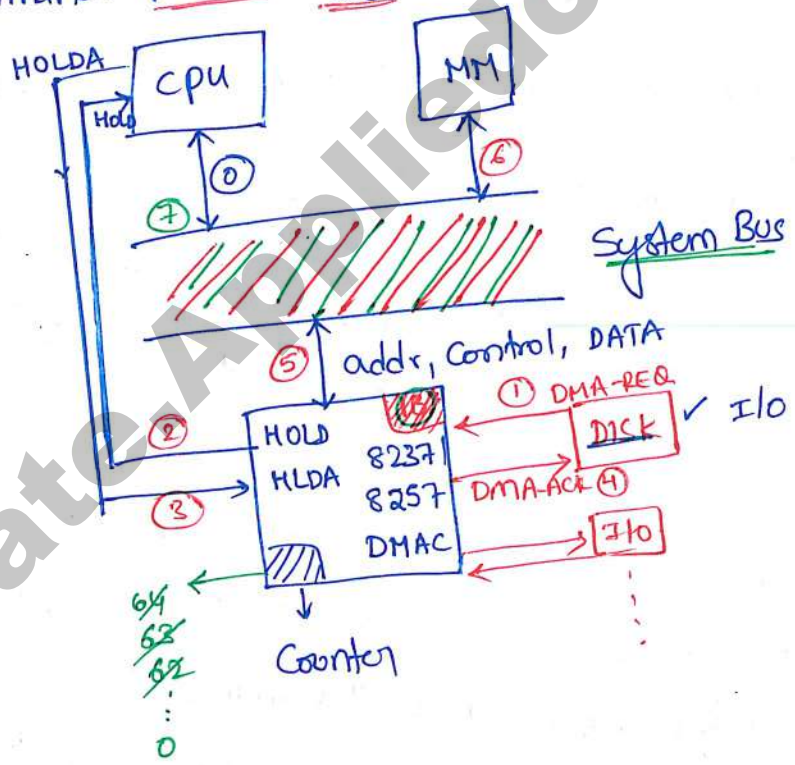
Ph: 844-844-0102

Single-buffer vs cascading Mode



DMA (Direct Memory Access):

I/O Command: port #, mem addr, count, control-signals



DMA-Modes:

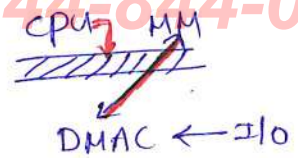
cycle-stealing: Byte by Byte word by word

Burst-Mode: Batch Mode Bulk mode

Timings:

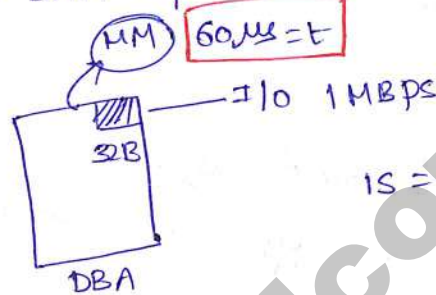
(*) Preparation-time (CPU-Busy-State)

(*) Transfer-time (CPU-Blocked-State)



- ① Consider a 1 MBPS I/O device interfacing with a CPU using DMA in cycle-stealing mode. whenever 32B of data present in the Buffer, it is transferred to the main memory. Main memory cycle time is 60 micro-sec. Percentage of time CPU is in Blocked state during the DMA operation is _____

Sol:



$$1S = 1MB \Rightarrow 1B = \frac{1}{1M} S$$

$$= 1 \mu s$$

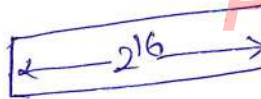
$$\text{Preparation time} = 32 \mu s - \text{Busy}$$

$$\Rightarrow 32B = 32 \mu s$$

$$\text{Trans time} = 60 \mu s \Rightarrow \text{Blocked}$$

$$= \frac{60}{92} = \boxed{\frac{60}{(60+32)}}$$

- ② The size of the data count register of a DMA Controller is 16 bits. The processor needs to transfer a file of 29,154 KB from the disk to main memory. The memory is byte addressable. The minimum number of times the DMA Controller needs to get Control of the system bus from the processor to transfer the file from the disk to main memory is 456

2^{16} 

64KB

 $1 \rightarrow 64 \text{ KB}$ $? \leftarrow 29154 \text{ KB}$

$$\left\lceil \frac{29154}{64} \right\rceil$$

$$\Rightarrow \lceil 455.53 \rceil$$

$$\Rightarrow \boxed{456}$$

== 0 ==

Gate-Appliedcourse.com